African Scientific Research and Innovation Council

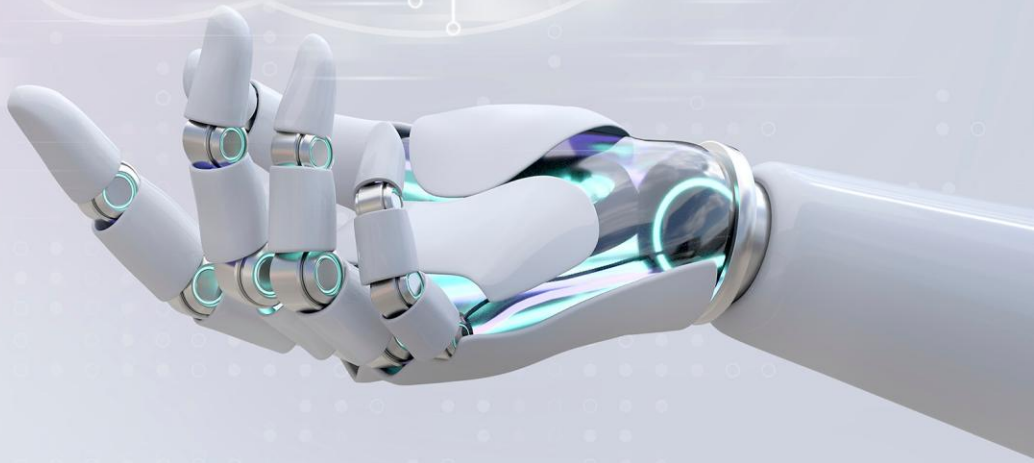
BAAI 智源研究院

北京大学 PEKING UNIVERSITY

中国人民大学 RENMIN UNIVERSITY OF CHINA

Global AI Compute Faculty Development Initiative

Module 04: Distributed Parallel Training and Communication Primitives



Guojie Luo gluo@pku.edu.cn

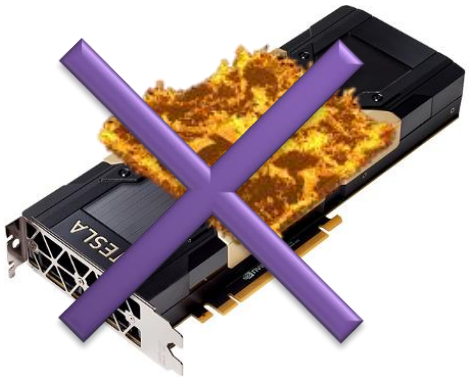
Jing Wang jwang@ruc.edu.cn

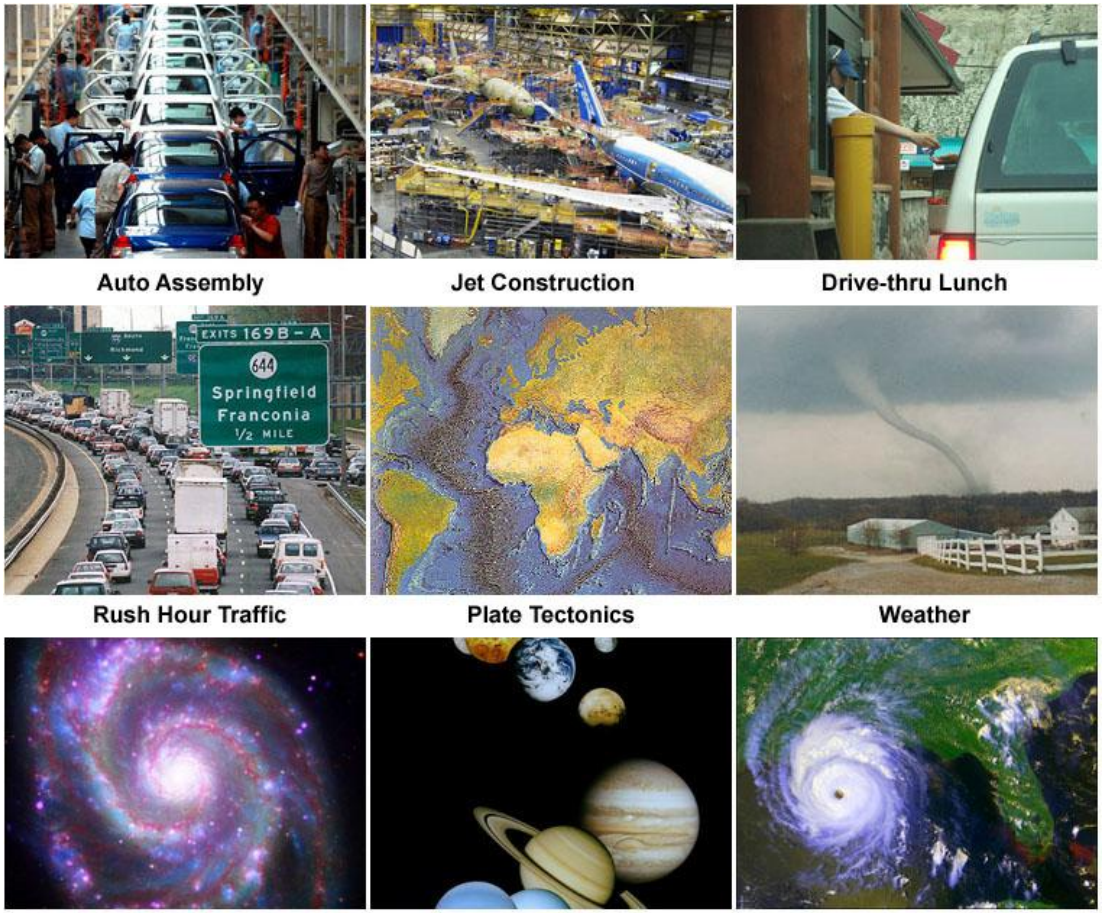
October 2026 · 2nd AICFD

Supported by UNDP

From Serial to Parallel Computing

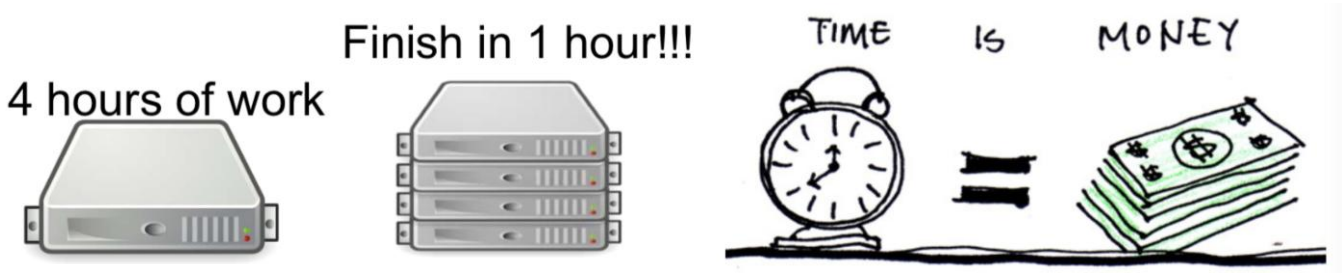
- Growing data volumes and computation exceed the capacity of a single device





Why Parallel Computing?

Save Time
Use more resources to reduce runtime and cost
Shorter runs allow more experiments and debugging cycles



- **Developer and researcher time is more valuable than hardware.**

Suppose one training run takes 10 GPU-days
Parallelize with distributed training

- **With 1,024 GPUs, ideally finish in 14 minutes!**
- **Greatly accelerate development and research cycles**

Large Models Take Longer to Train



Boss: What did you do last month?



You: Trained the model for one epoch.



Boss: OK. What is your plan for next month?



You: Train it for... another epoch?

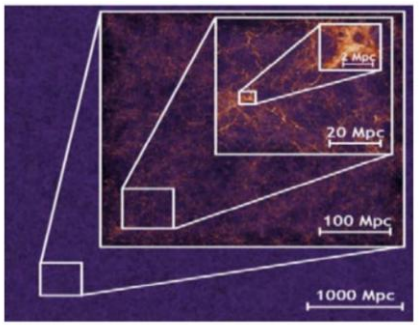




Why Parallel Computing?

Solve Larger Problems

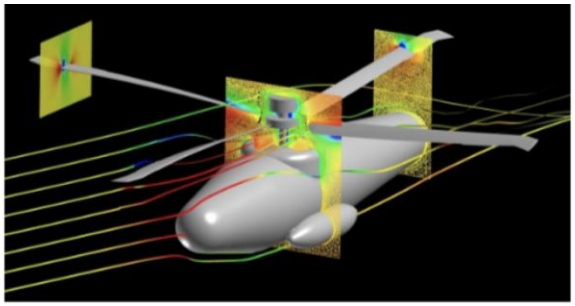
- Impossible or impractical on a single machine
- Scientific computing
  - Trillions of simulation elements
  - Thousands of parameters to tune
  - Terabytes of data to process
  - Hours on million-core, petaflop-scale clusters



1 trillion particles, 4.225 Gpc box-size simulation, and 6 kpc force resolution.



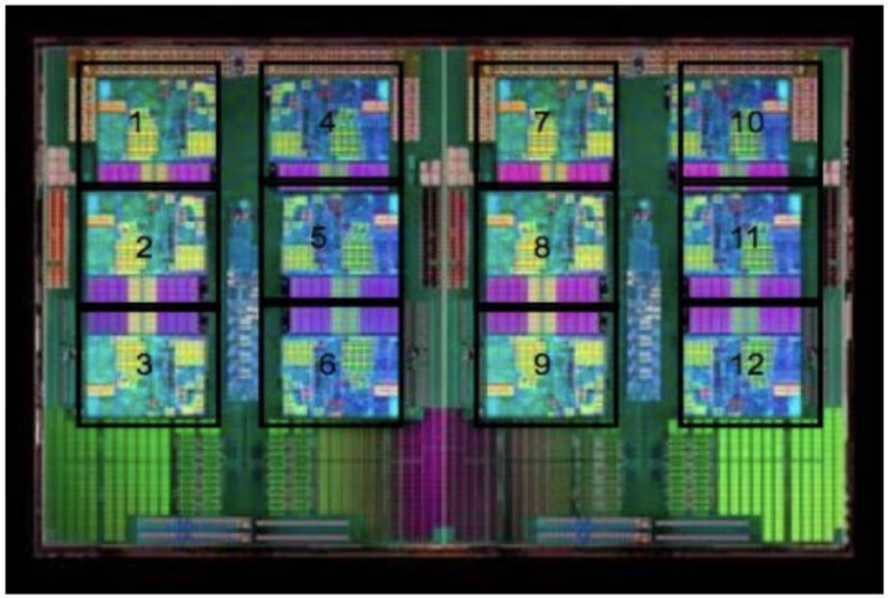




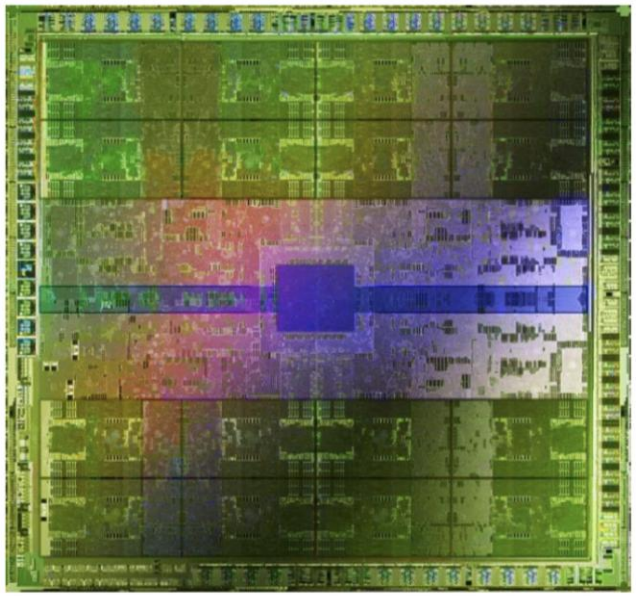
Science drives progress!

Why Parallel Computing?

Unlock the Performance of Parallel Hardware



12 Cores IBM Blade Multi-core CPU



512 Cores NVIDIA Fermi GPU

Large Models Take Longer to Train

Model	Params. (M)	Training Time (GPU-hours)
ResNet-50	26	31
ResNet-101	45	44
BERT-Base	108	84
Turing-NLG 17B	17,000	TBA
GPT-3 175B	175,000	3,100,000

Measured on NVIDIA A100

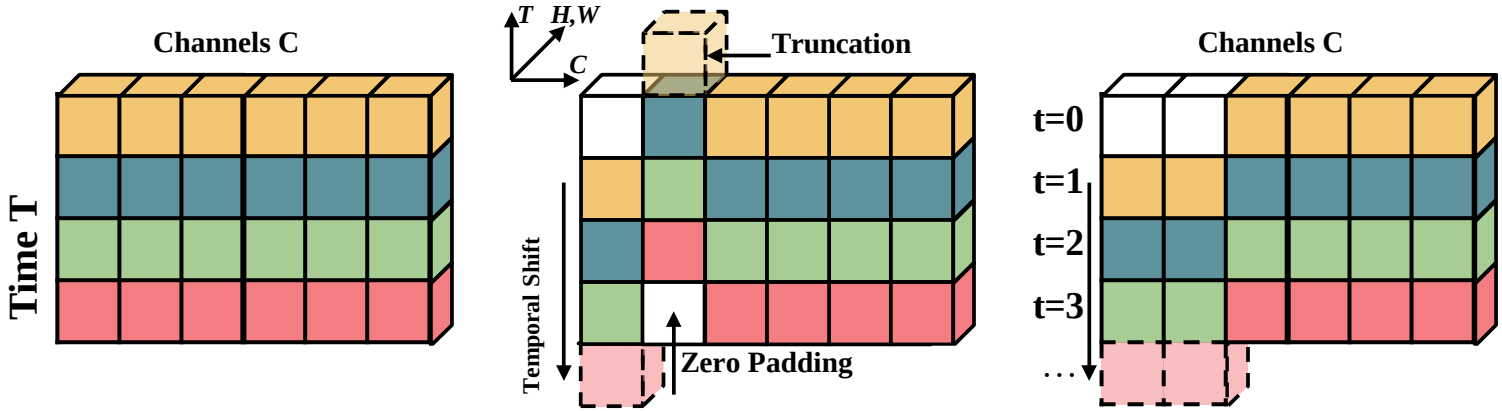
The SUMMIT Supercomputer

CPU: 2 × 16-core IBM POWER9 (dual NVLink bricks, 25 GB/s per side)
Interconnect: dual-rail EDR InfiniBand, 200 Gbit/s
Memory: 512 GB DDR4
GPU: 6 NVIDIA Tesla V100 GPUs
Storage: 250 PB



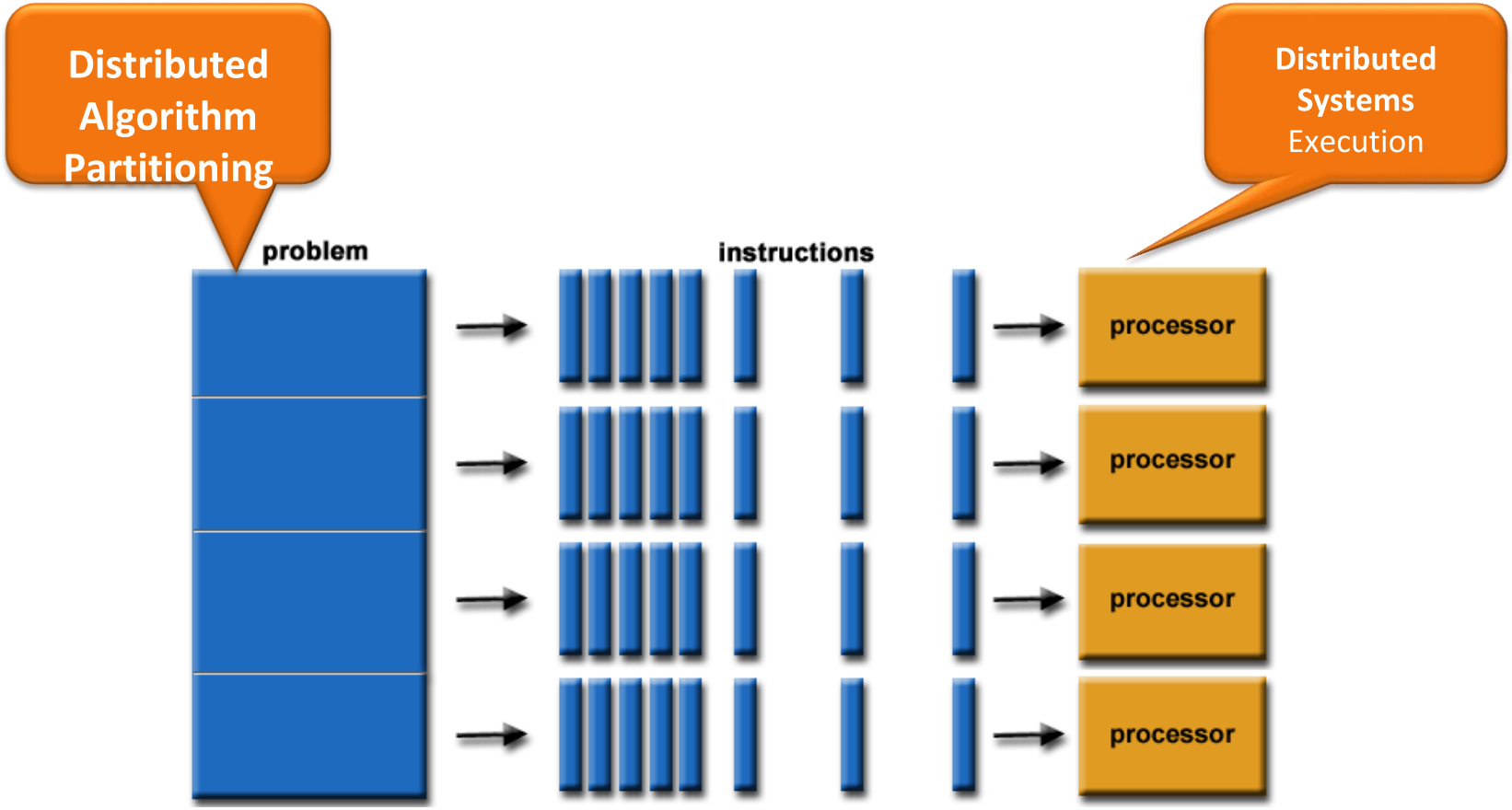
Training a Model on 660 Hours of Video

Model	Training Time	Accuracy
1 SUMMIT node (6 GPUs)	49h 50min	74.1%
128 SUMMIT nodes (768 GPUs)	28min	74.1%
256 SUMMIT nodes (1536 GPUs)	14min (**211x**)	74.0%



TSM: Temporal Shift Module for Efficient Video Understanding [Lin 2019]

Distributed Processing



The Value of Distributed Learning

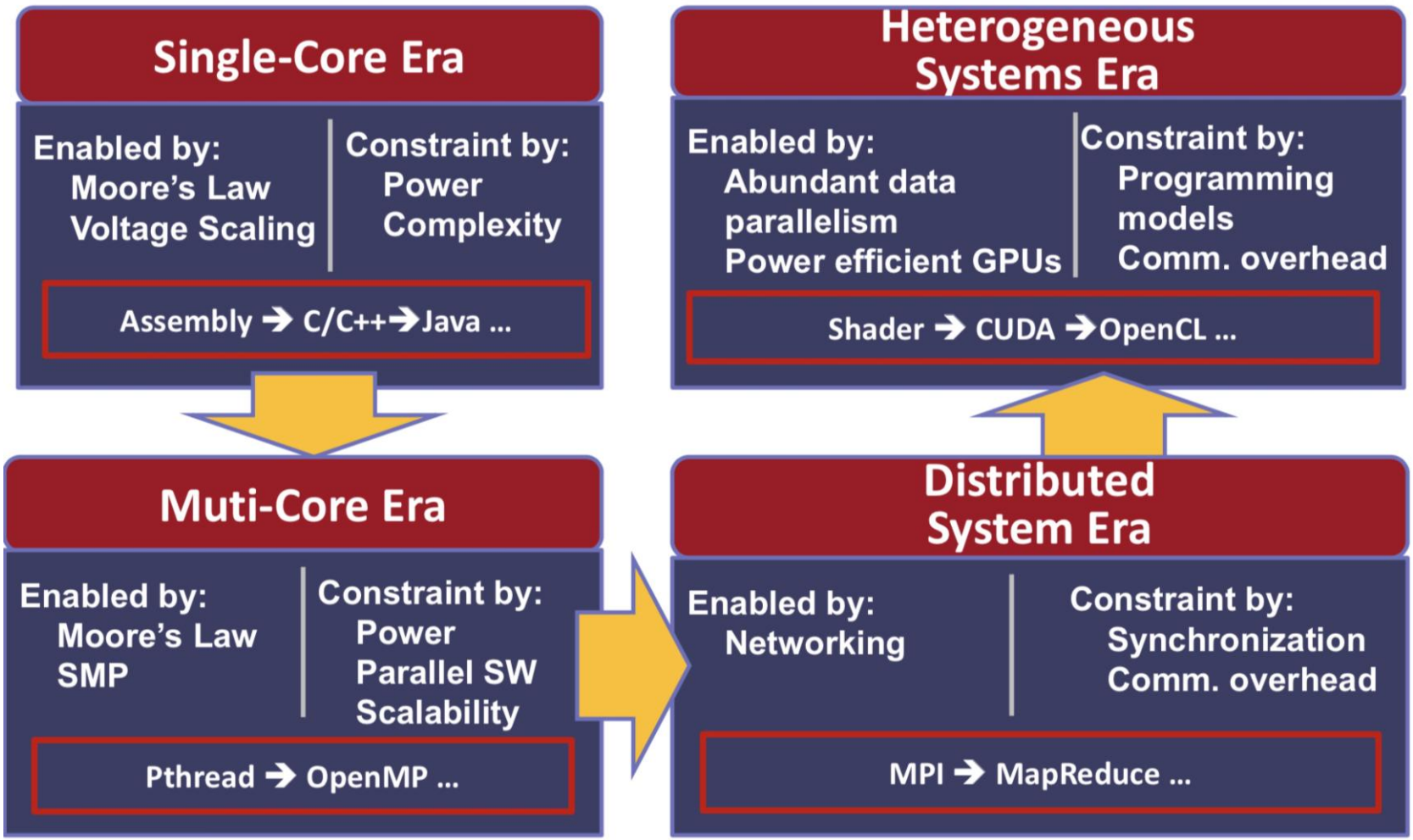
- Training time:
- Data size × work per step / compute rate

Model-dependent | Variable

- Compute rate:
- Rate per device × devices × efficiency

Moore's law, limited gains | Variable | Main focus

Trends in Parallel Computing



Limits of Scalar Pipelines

- A scalar pipeline has multiple stages in a single linear sequence

Three Key Limitations

Scalar pipeline throughput limit

- Only one instruction per cycle
- Deeper pipelines have costs

One uniform pipeline is inefficient

- Different hardware requirements
- Long or variable latencies

Rigid execution reduces efficiency

- A stall blocks later instructions

Extensions to Address the Limits

- 1. Parallel (superscalar) pipelines
  - Fetch multiple instructions per cycle
  - Decode instructions in parallel
- 2. Diversified execution units
  - Use multiple heterogeneous FUs in EX
  - Choose the number and mix of FUs
- 3. Dynamic execution
  - Later instructions bypass stalls
  - Buffer in-flight instructions

Superscalar Pipelines and Instruction-Level Parallelism

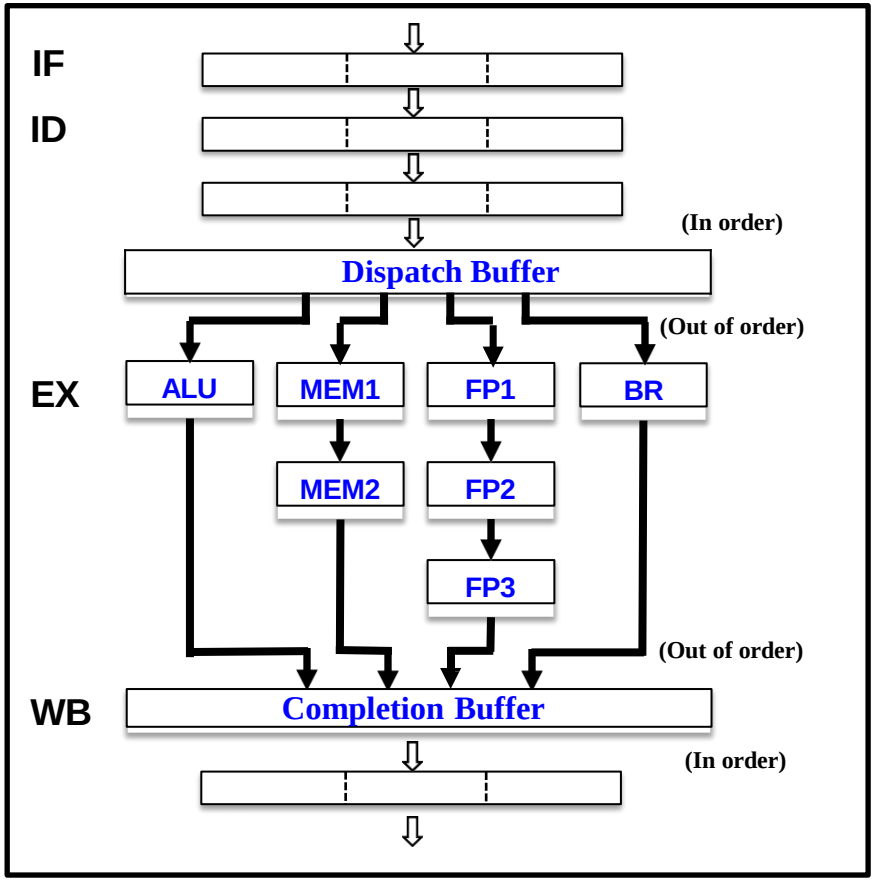
Width = 3
4 pipelined FUs
2 multi-entry buffers:

- Dispatch buffer (DB)
  - Centralized
    - Pentium Pro
  - Distributed
    - PowerPC 604



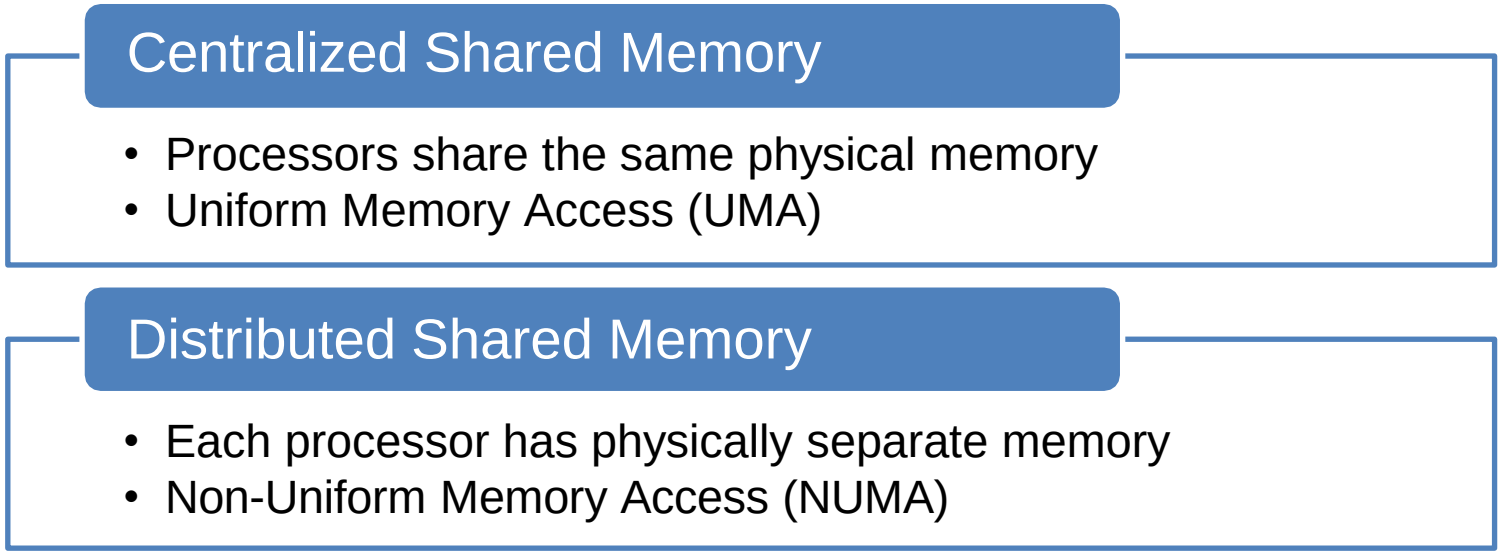


- Completion Buffer



Parallel, Diversified Pipelines

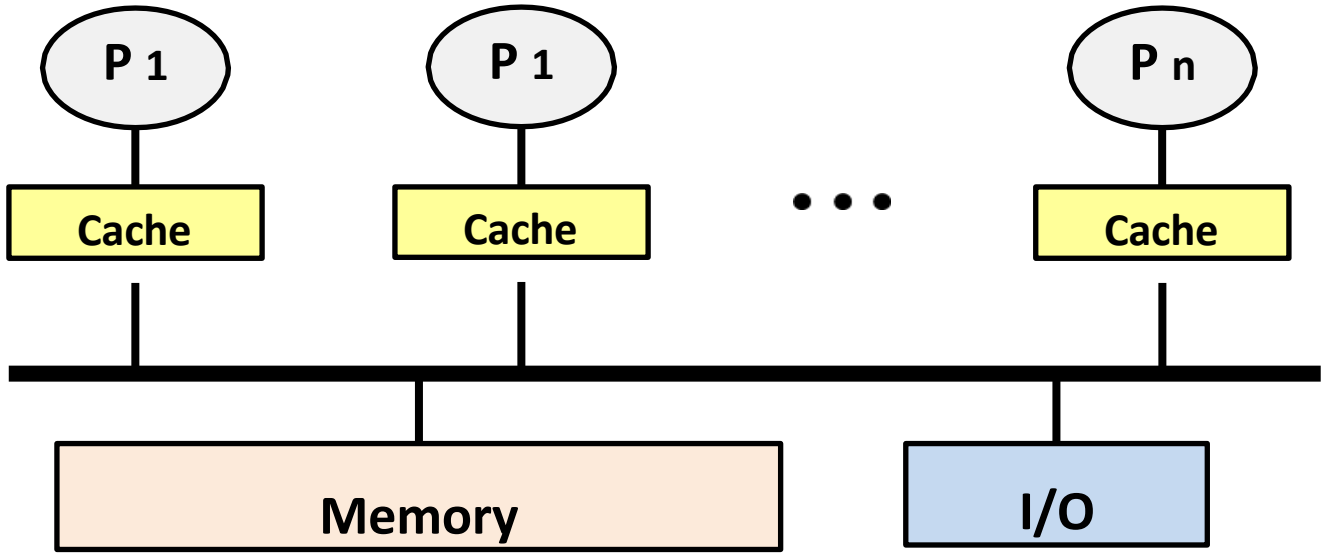
Two Types of Multiprocessors



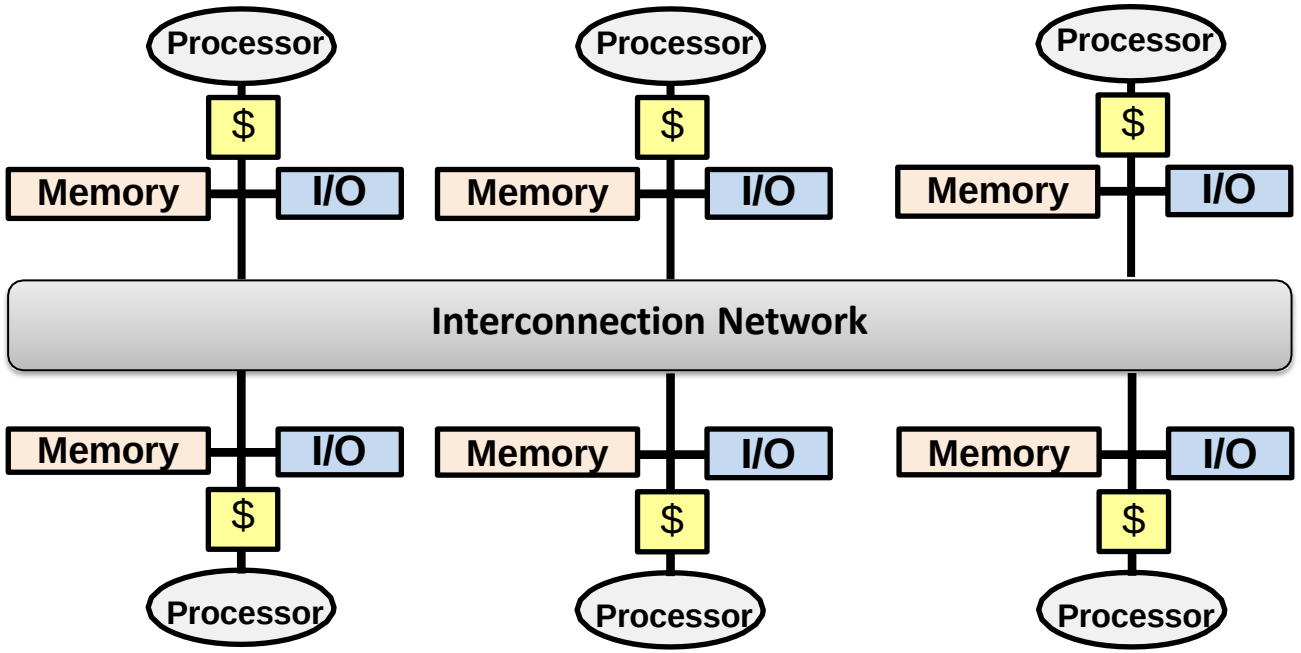
- Classification by Memory Organization
- Shared memory means a shared address space

Centralized Shared Memory

- Often called symmetric multiprocessing (SMP)
  - Equal access time to every memory location



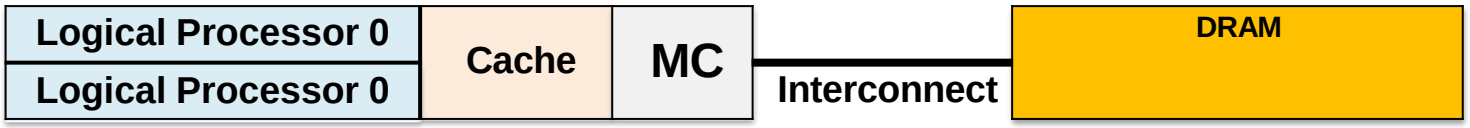
Distributed Shared Memory (DSM)



- Two Advantages
  - Scalable memory bandwidth
  - Low-latency local access
- Main Drawbacks
  - Complex communication
  - Higher latency between nodes

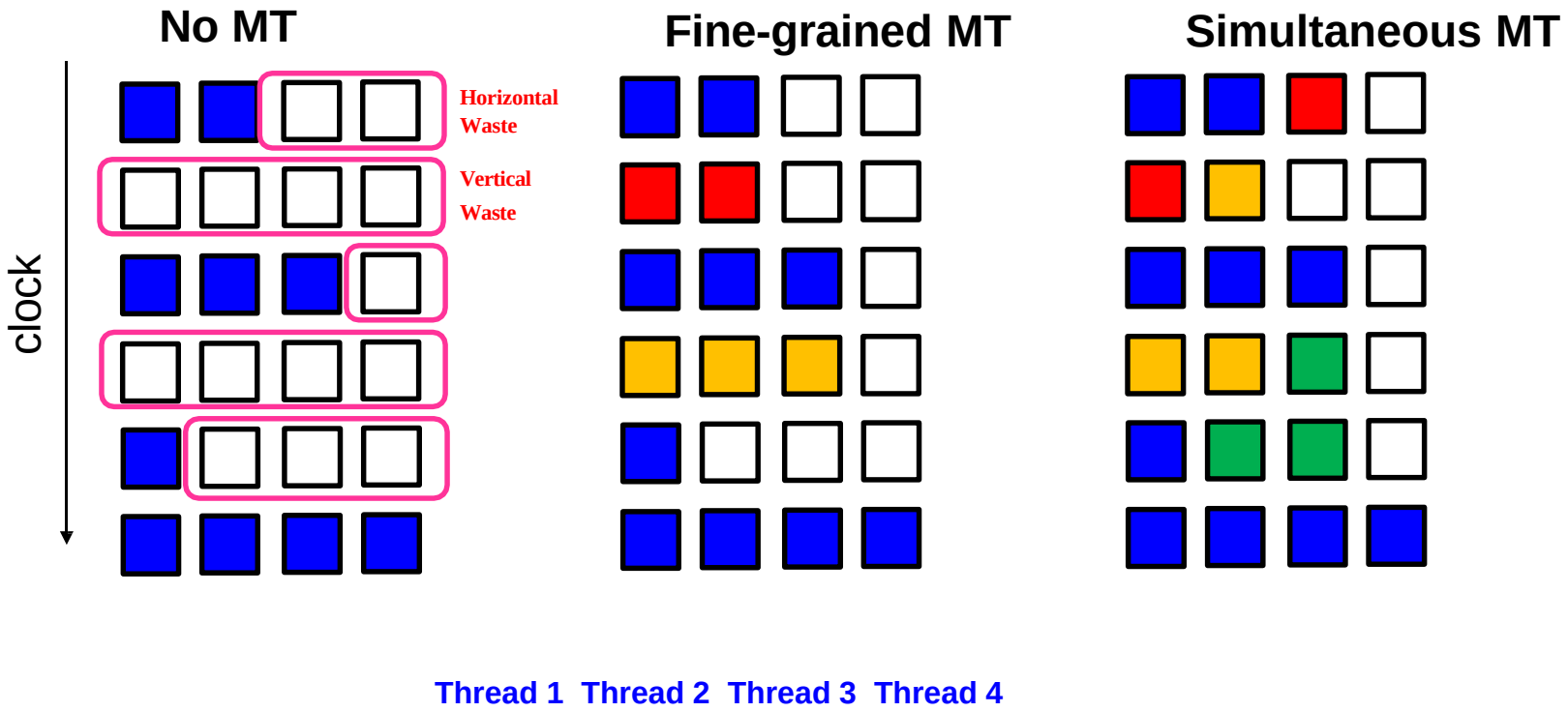
Thread-Level Parallelism

- Available ILP cannot hide long memory stalls, reducing FU utilization
- Hardware multithreading keeps otherwise idle processor resources busy
- **Multithreaded processors execute instruction streams from multiple threads in parallel**
  - Replicate resources such as registers and the PC
  - Add logic to switch between threads in the pipeline



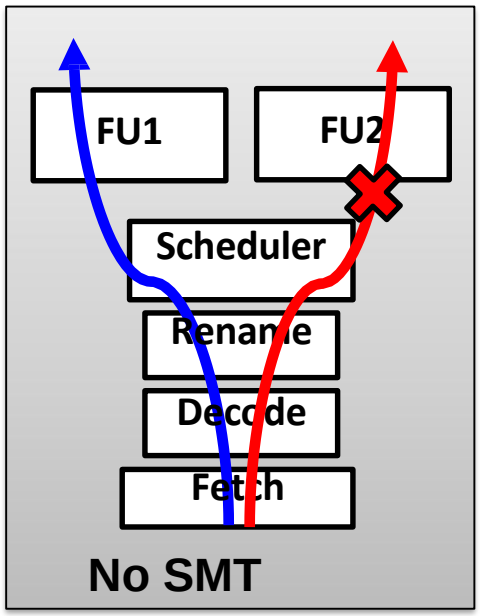
SMT and Hardware Utilization

- Multithreading improves hardware utilization in different ways

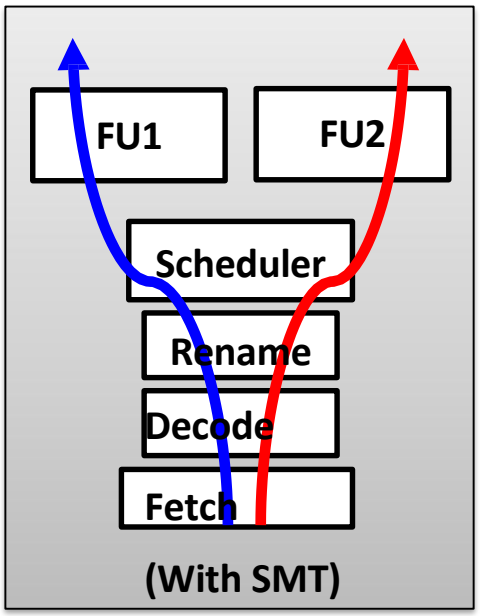


Why Multiple Cores?

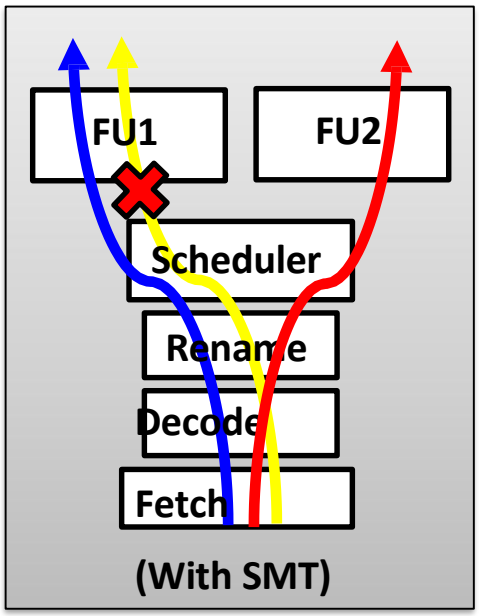
- A single superscalar core cannot fully exploit TLP



Only one thread runs at a time



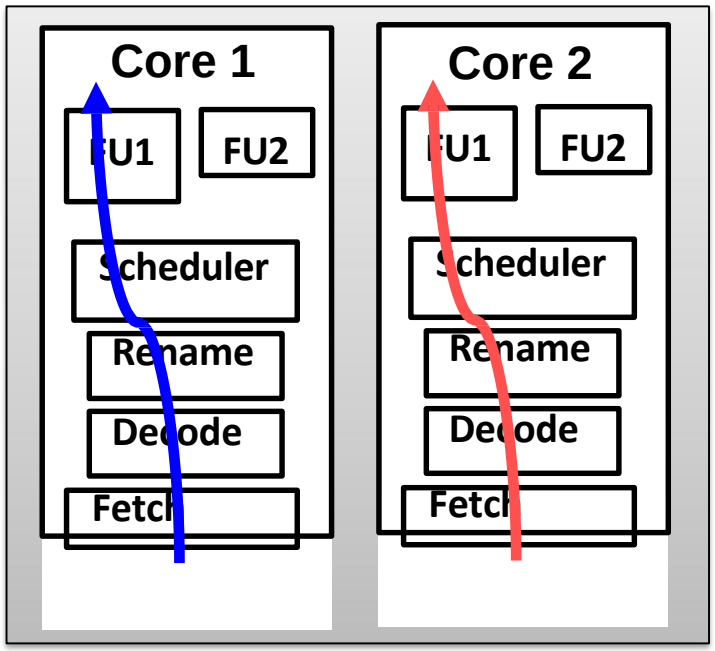
Two threads can run concurrently



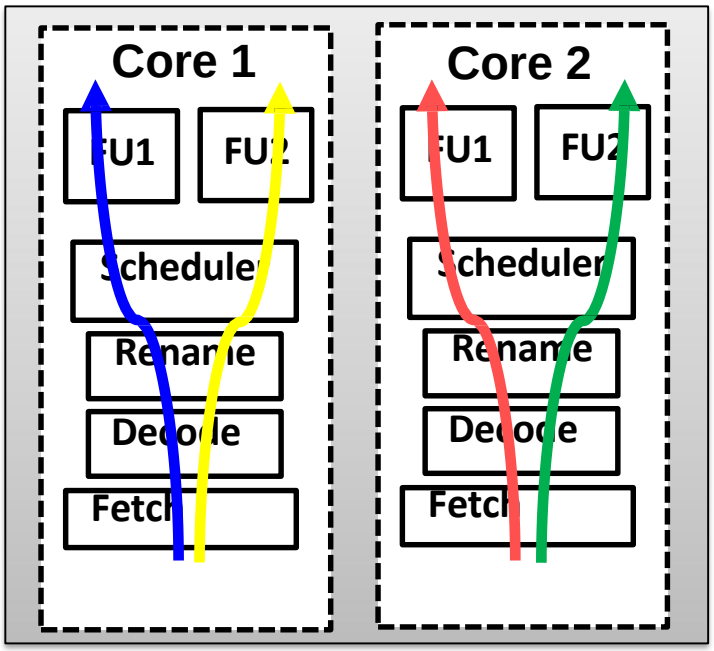
Threads cannot use the same FU simultaneously

Why Multiple Cores? (continued)

- Multicore Architectures Explicitly Exploit TLP

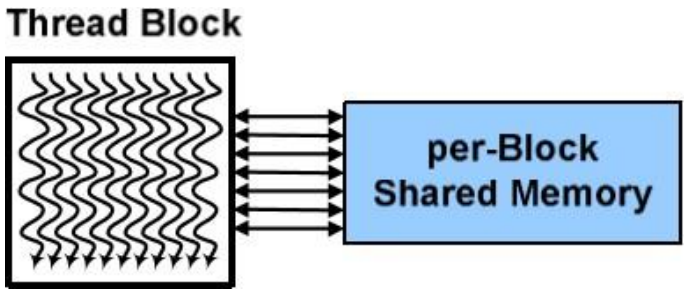


Threads can run on separate cores

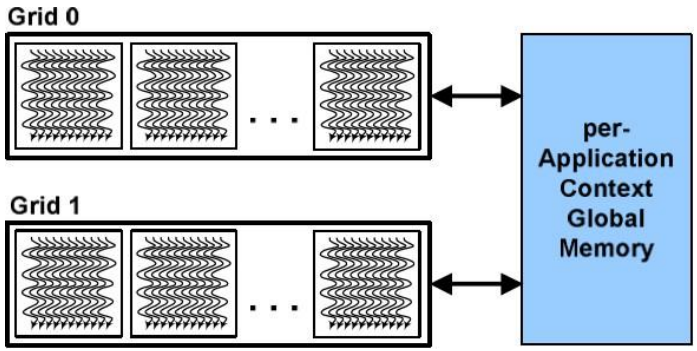


Dual core with SMT: 4 concurrent threads

The CUDA Parallel Computing Model



- **A thread block is a group of cooperating threads**
  - 1–512 concurrent threads
  - Each thread has a unique ID



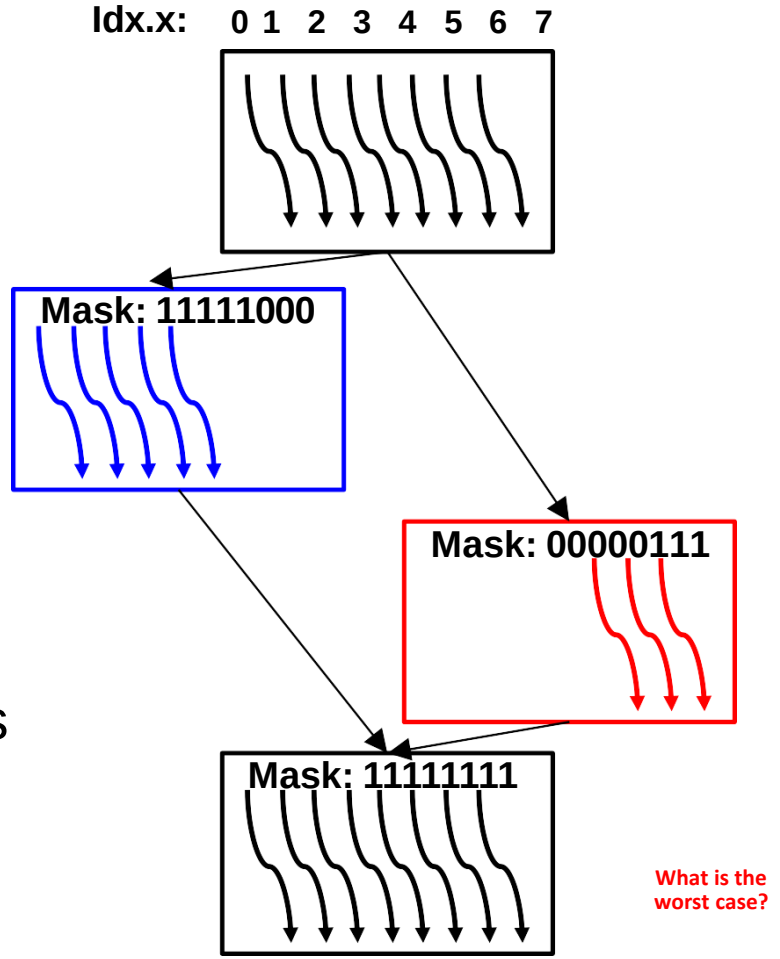
- Grid: an array of thread blocks executing the same kernel
  - Read and write global memory
  - Synchronize between dependent invocations

Stack-Based Reconvergence

- Divergent Branch

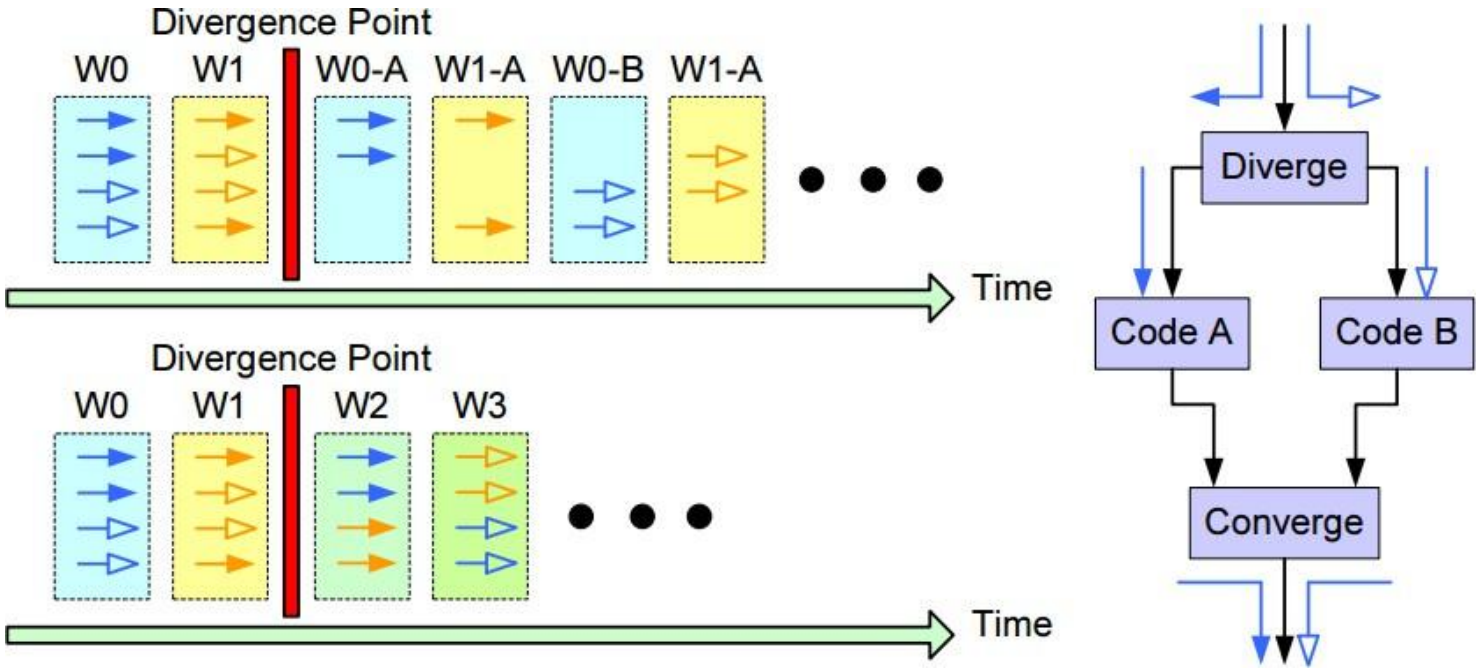
```
If (IDx.x < 5) {
my_data[IDx.x] += a
}
else {
my_data[IDx.x] *= b
}
```

- A reconvergence stack brings threads back together
  - Stores the reconvergence PC, active mask and execution PC



Dynamic Warp Formation

- Form new warps from ready threads with the same PC



Throughput-Oriented Workloads

- Focus on the number of tasks completed
  - Finance, social media, gaming, etc.
- Multicore CPUs suit these workloads
- GPU adoption has grown rapidly in recent years

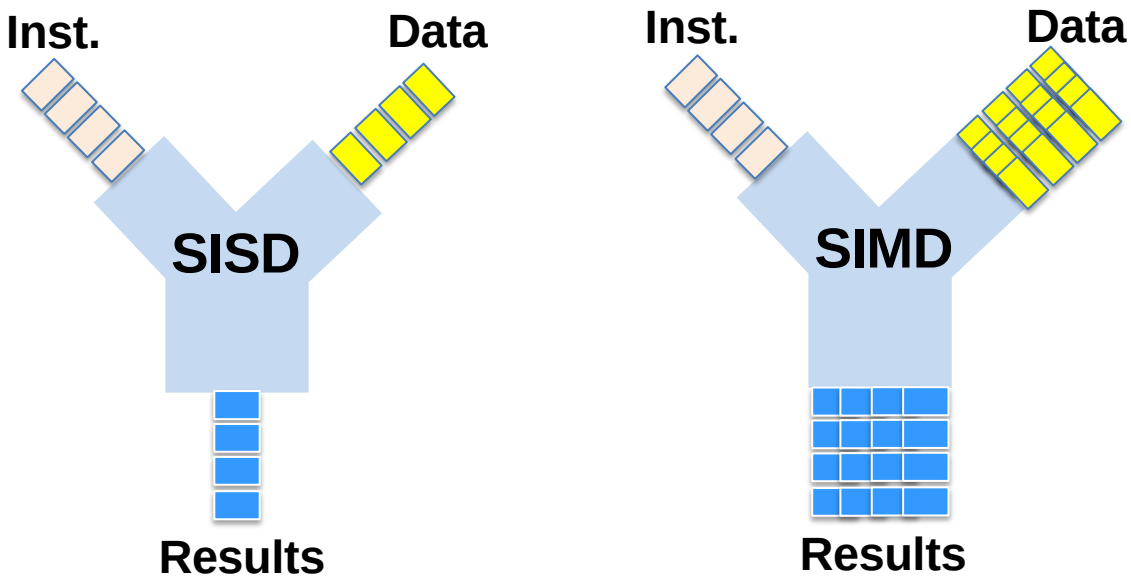




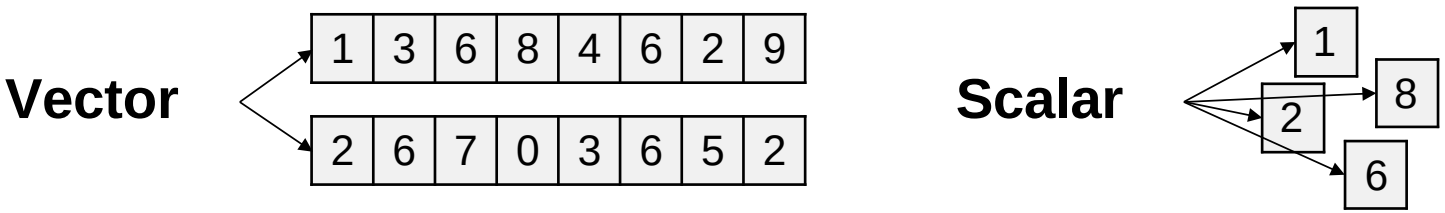
Data-Level Parallelism

- **DLP: parallelism from simultaneous operations on large datasets, rather than multiple threads**

- Especially suited to large scientific and engineering workloads



Vector Operations

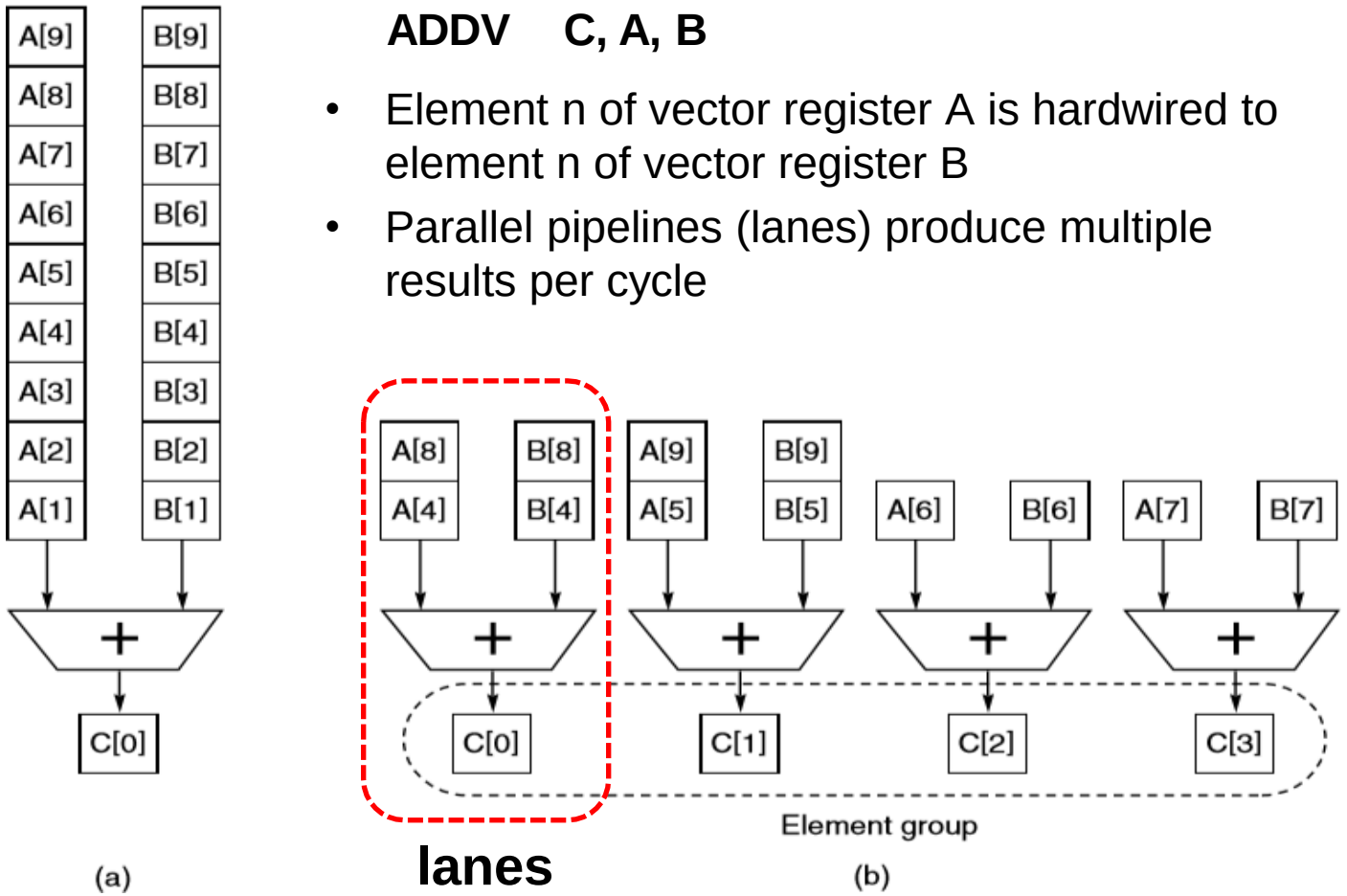


A vector is a linear array of numbers or scalars

- Can a processor support a vector data type?

- The idea behind vector operations:
  - Read a set of data elements
  - Operate on the array of elements

Multiple Lanes

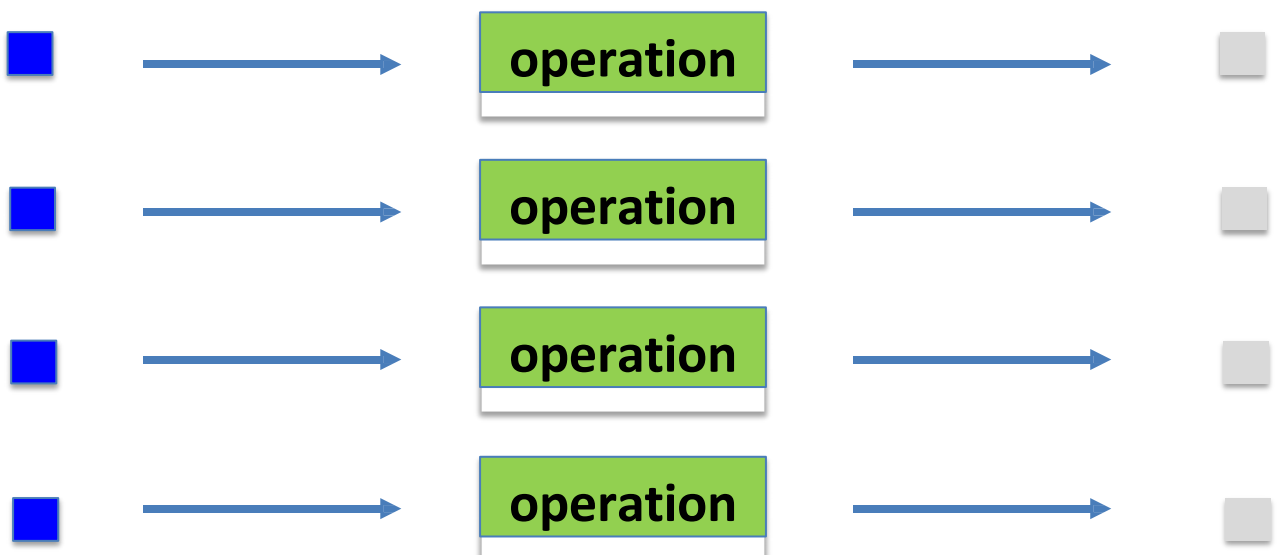


ADDV C, A, B

- Element n of vector register A is hardwired to element n of vector register B
- Parallel pipelines (lanes) produce multiple results per cycle

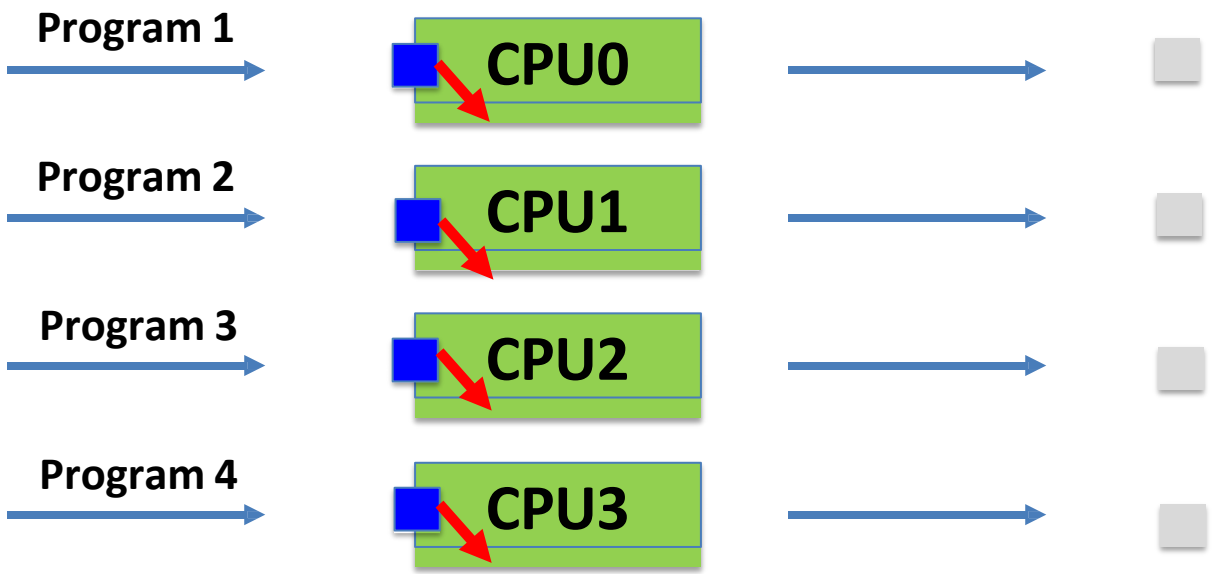
Data-Parallel Workloads

Apply the same independent computation to multiple inputs



Processing Data-Parallel Workloads

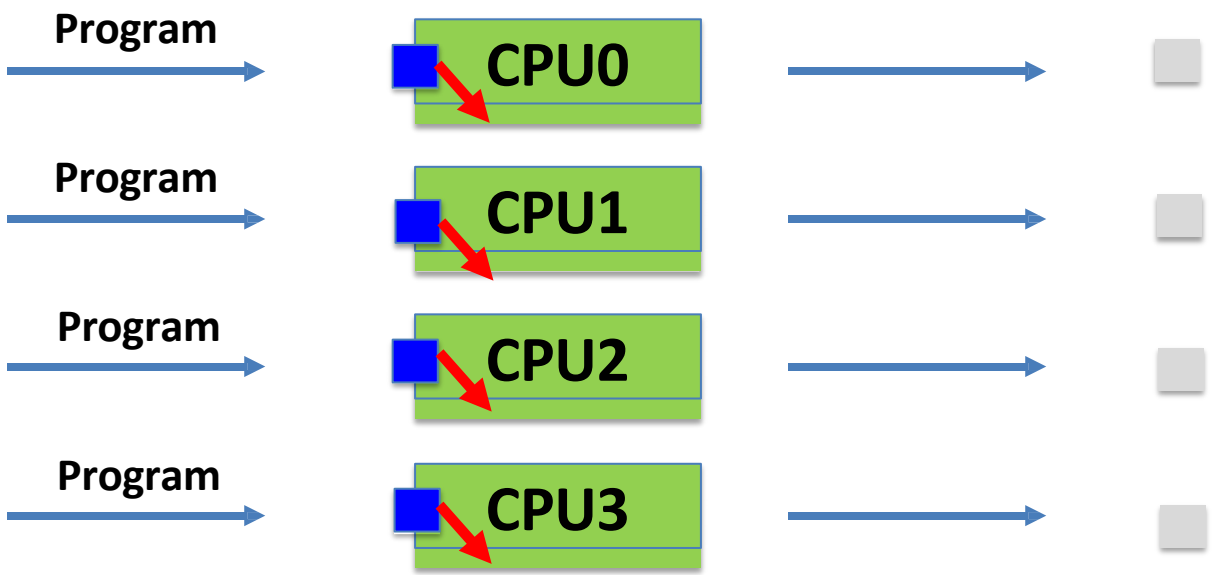
Multiprocessor approach (MIMD)



Assign independent tasks to multiple CPUs

Processing Data-Parallel Workloads

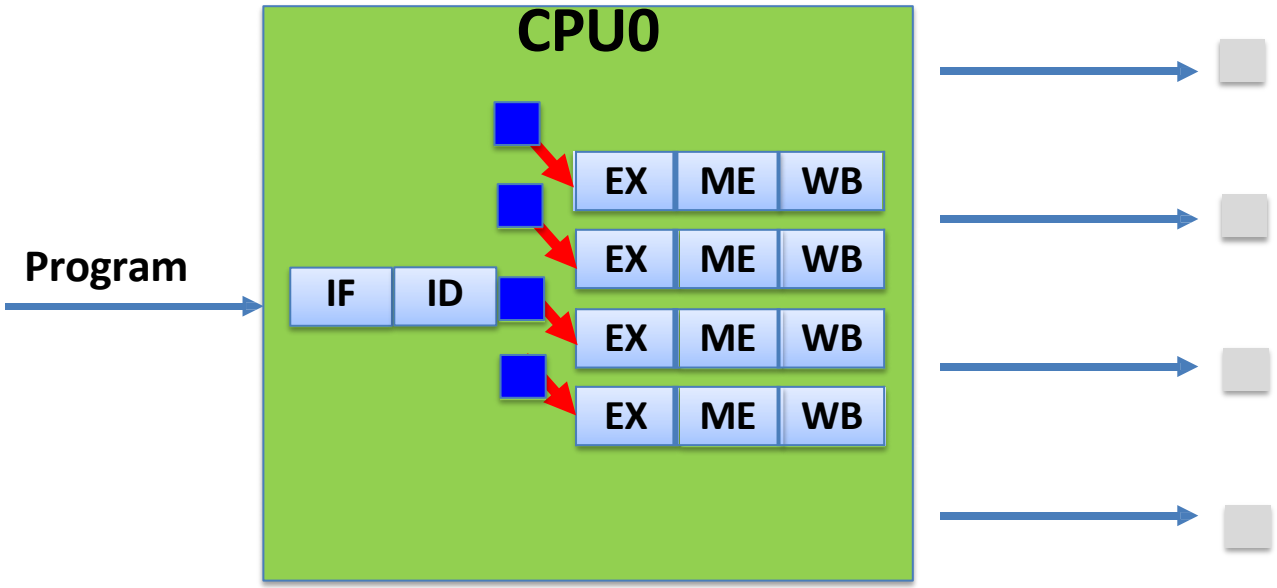
SPMD: Single Program, Multiple Data



Assign identical, independent tasks to multiple CPUs

Processing Data-Parallel Workloads

SIMD/vector approach: eliminate redundant units

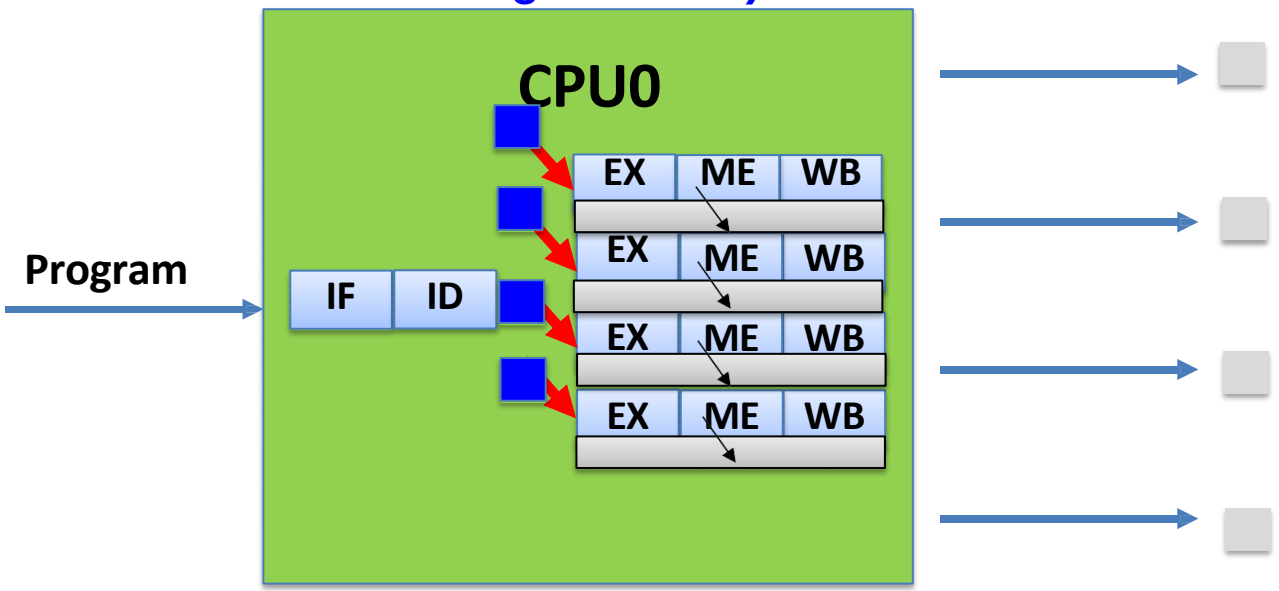


Assign identical, independent tasks to multiple execution units

Processing Data-Parallel Workloads

SIMT approach:

Multiple threads + scalar operations (one PC, multiple register files)



Assign identical, independent tasks to threads in lockstep

Outline

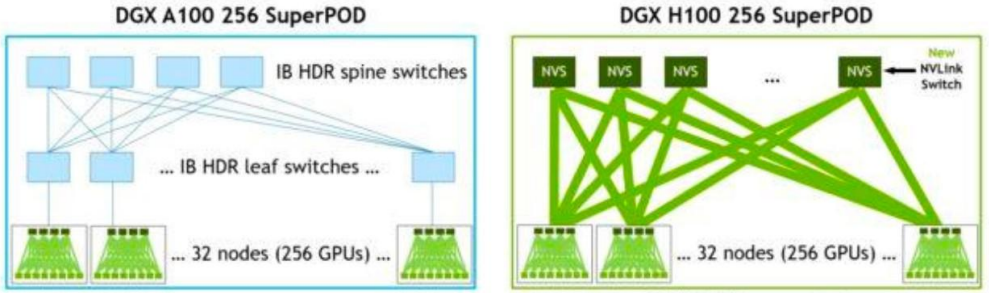
- **Data Parallelism**
  - ZeRO-1/2/3, FSDP
  - Memory/Communication Comparison
- **Tensor Parallelism**
- **Pipeline Parallelism**
  - F-then-B
  - 1F1B
- **Hybrid Parallelism**
  - 3D/5D Parallelism
  - Automatic Model Parallelism

1. Data Parallelism and ZeRO/FSDP

- ZeRO-1/2/3, FSDP
- Memory/Communication Comparison

1.3. Introduction to Distributed LLM Training

- Transition from foundational LLM inference to the computational realities of training.
  - Inference memory is dominated by model weights and transient KV cache.
  - Training introduces a massive memory footprint due to the need to preserve the entire computational graph and maintain high-precision optimizer states.
  - A single hardware accelerator is insufficient; computation and memory must scale horizontally.



	A100 SuperPod			H100 SuperPod			Speedup	
	Dense PFLOP/s	Bisection [GB/s]	Reduce [GB/s]	Dense PFLOP/s	Bisection [GB/s]	Reduce [GB/s]	Bisection	Reduce
1 DGX / 8 GPUs	2.5	2,400	150	16	3,600	450	**1.5x**	**3x**
32 DGXs / 256 GPUs	80	6,400	100	512	57,600	450	**9x**	**4.5x**

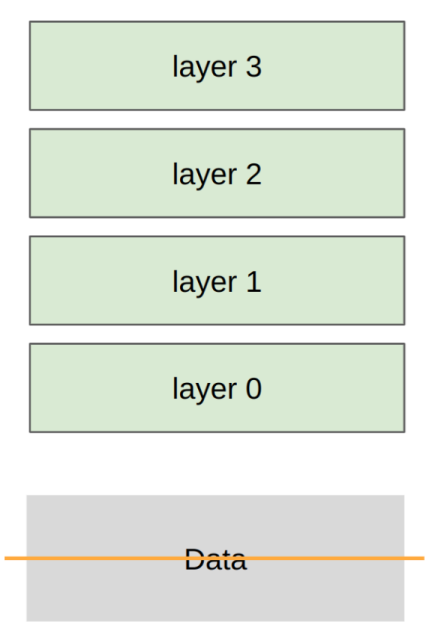
Split memory/compute across GPUs and machines

1.15. Revisiting GPU Memory Calculation

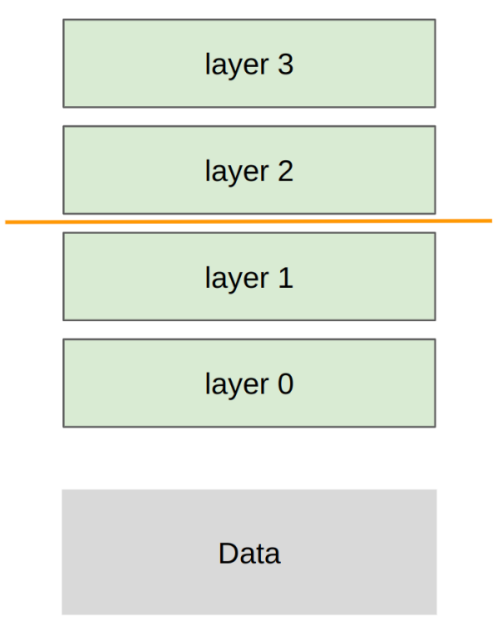
- **GPU memory consumption :=**
  - Model parameters (in FP16) +
  - Gradients (FP16) +
  - Master weights (FP32) +
  - Adam momentum (FP32) +
  - Adam variance (FP32) +
  - Model Activations (This scales with the batch size)!

1.1. Model parallel along the different axes

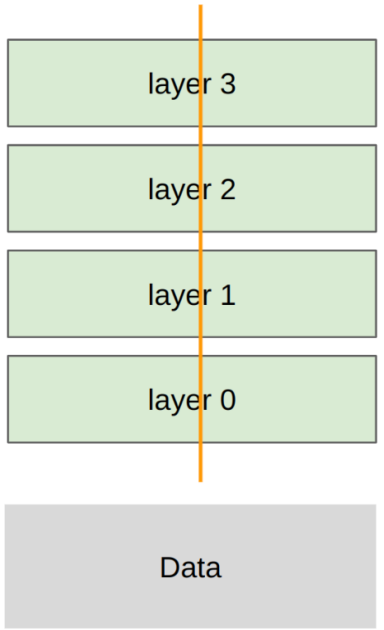
- Introduction to the three core paradigms
  - Data Parallelism, Tensor Parallelism, and Pipeline Parallelism.



Data Parallel



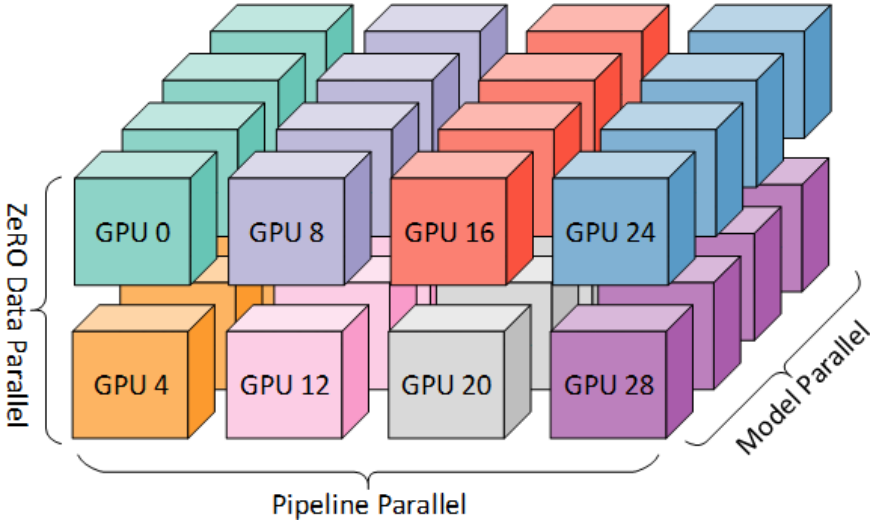
Pipeline Parallel



Tensor Parallel

1.2. Mathematical Orthogonality of Hybrid Parallelism

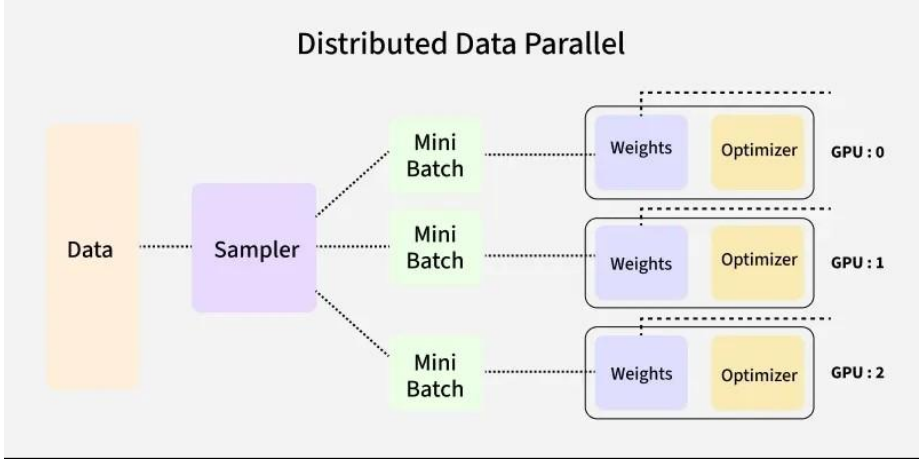
- **Orthogonal Axes:** DP (batch), TP (intra-layer/hidden dims), PP (inter-layer/depth).
- **The Scaling Wall:** Isolated paradigms fail. DP hits memory limits; TP hits network bottlenecks; PP requires massive micro-batches, exploding memory.
- **Resource Scaling:** Multiplicative combination: $\mathrm{TotalGPUs} = \mathrm{TP} \times \mathrm{PP} \times \mathrm{DP}$.
- **Synergy:** TP reduces layer memory → fits larger micro-batches → lowers PP bubble ratio.



https://www.microsoft.com/en-us/research/blog/deepspeed-extreme-scale-model-training-for-everyone/

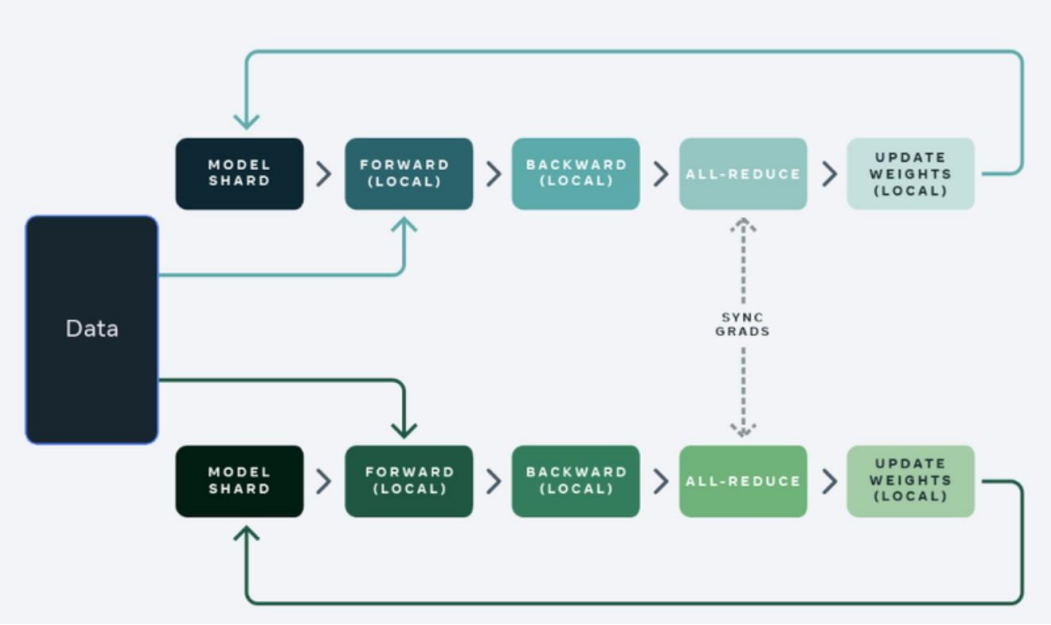
1.4a. Introduction to Data Parallelism (DP)

- Data Parallelism (DP) is the foundational distributed training strategy.
  - Operates under the Single-Program Multiple-Data (SPMD) paradigm.
  - The training dataset is partitioned into non-overlapping mini-batches.
  - Every GPU receives a complete replica of the model weight and optimizer states.
  - Each accelerator computes forward and backward passes independently on its respective data shard.
- DistributedDataParallel (DDP) is the standard module for DP in PyTorch.



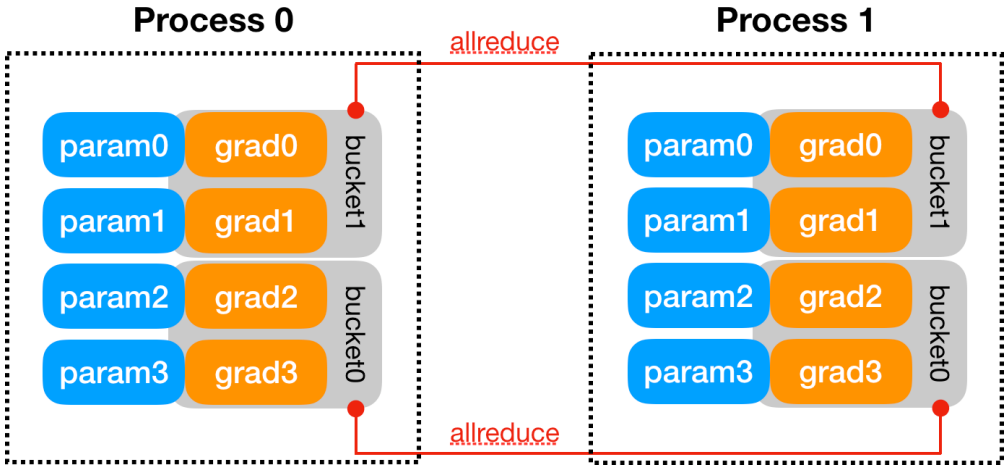
1.4b. Introduction to Data Parallelism (DP)

- Because each GPU computes gradients based on a different data shard, replicas would diverge without synchronization.
  - The gradients computed by each GPU must be aggregated and averaged after every step.
- All-Reduce efficiently sums the gradients across all nodes and broadcasts the result back, ensuring all replicas update identically.



1.5. Gradient Bucketing in DDP

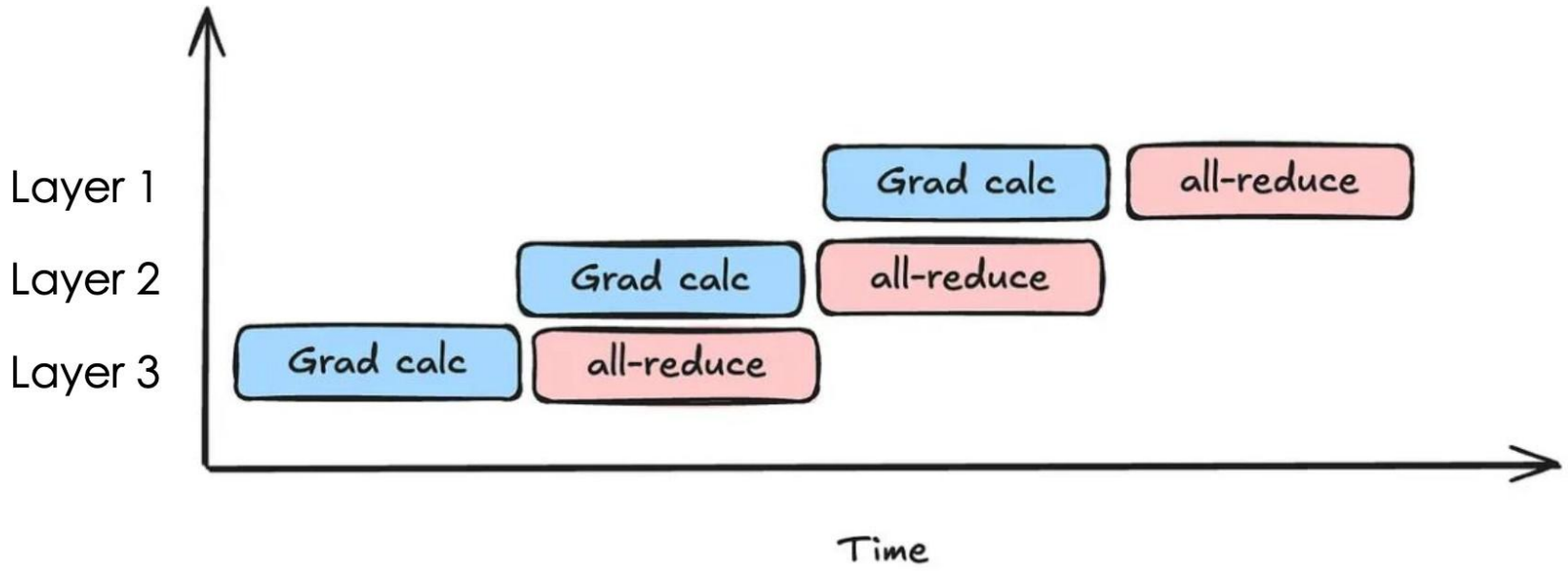
- Launching independent All-Reduce operations for every individual parameter tensor results in network congestion.
- DDP mitigates this via gradient bucketing.
  - Parameter gradients are organized into configurable memory buckets
  - The All-Reduce is only triggered when a bucket is completely filled during the backward pass.
  - This dramatically reduces the total number of network calls and optimizes bandwidth.



https://docs.pytorch.org/docs/stable/notes/ddp.html

1.6. Compute-Communication Overlap in DDP

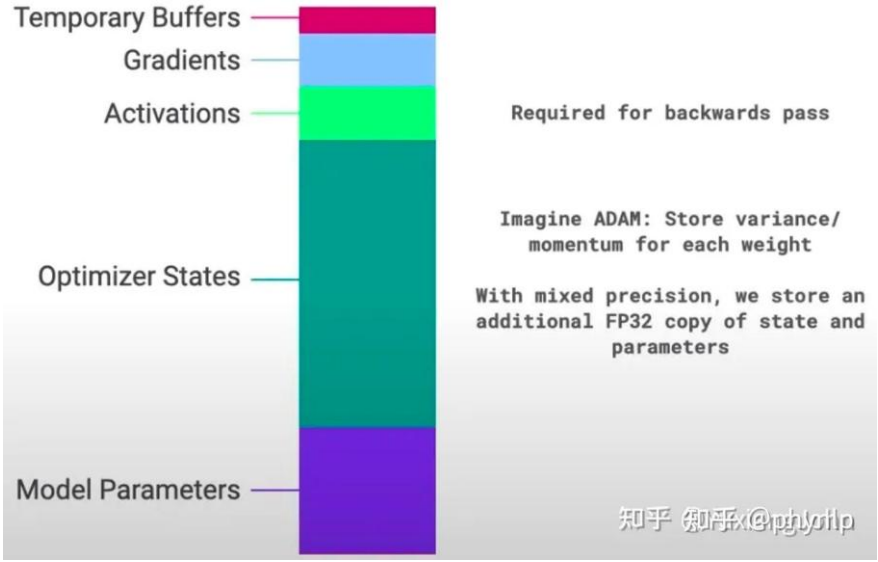
- Communications (red blocks) are not on the backward compute critical path (blue blocks). Thus good compute communication **overlap capability**.
  - 1. grad_calc(layer i, gpu k) depends only on grad_calc(layer i −1, gpu k)
  - 2. All-reduce(layer i, gpu k) depends on grad_calc(layer i, all gpus)



1.7a. The Memory Crisis in LLM Training

- Memory consumption comes from two folds:
  - Storing the models weights in memory (usually in FP16)
  - Storing the optimizer states (momentum and variance) in memory (usually in FP32, for accuracy purpose)
- For a 7B model, storing the model's weights in FP16 requires 14GB memory. The backward pass generates gradients requiring an additional 14GB.
- Storing the optimizer states in FP32 requires 56G memory.
- In total 70G memory used! Note that H100 GPU only has 80G memory

Memory composition during LLM training

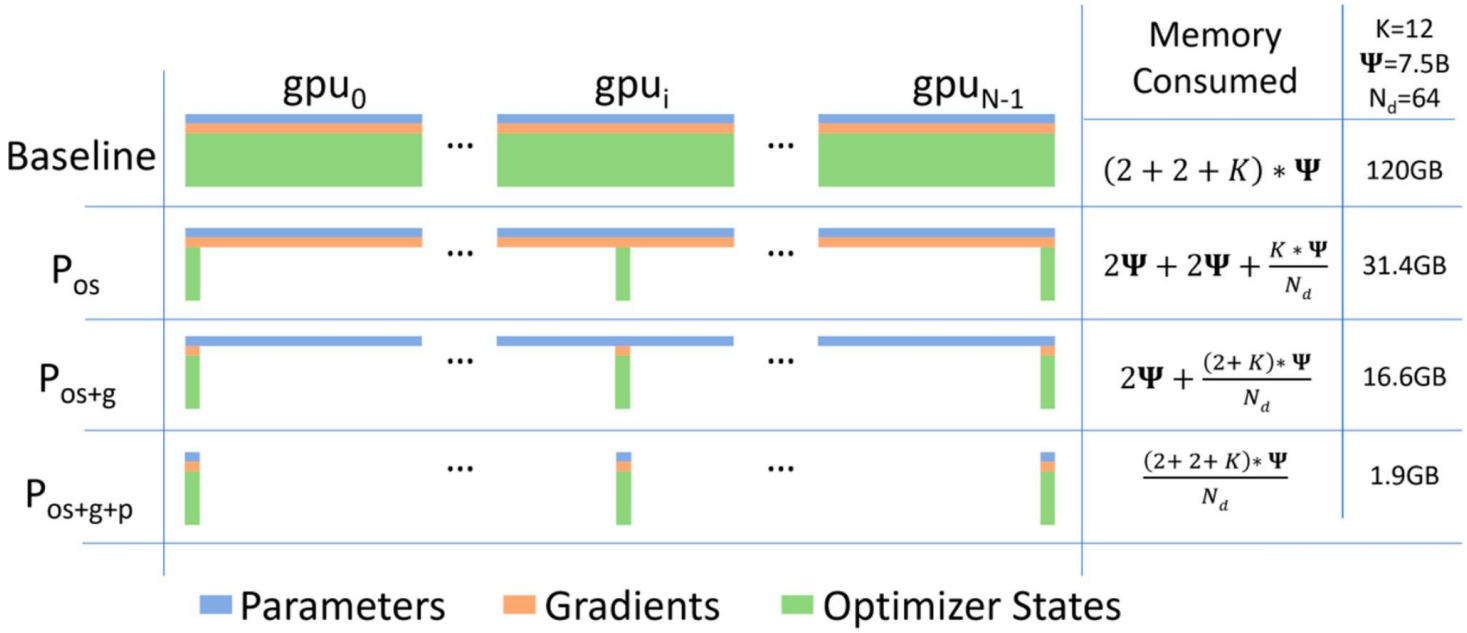


1.7b. The Memory Crisis in LLM Training

- Standard DDP suffers from severe memory redundancy.
- Training across 8 GPUs forces the cluster to maintain 8 complete copies of model weights, gradients, and massive FP32 optimizer states.
- Redundant replication consumes High Bandwidth Memory without providing additional computational capacity.
- Strictly limits the maximum trainable model size to the capacity of a single GPU, creating an insurmountable barrier for LLMs.

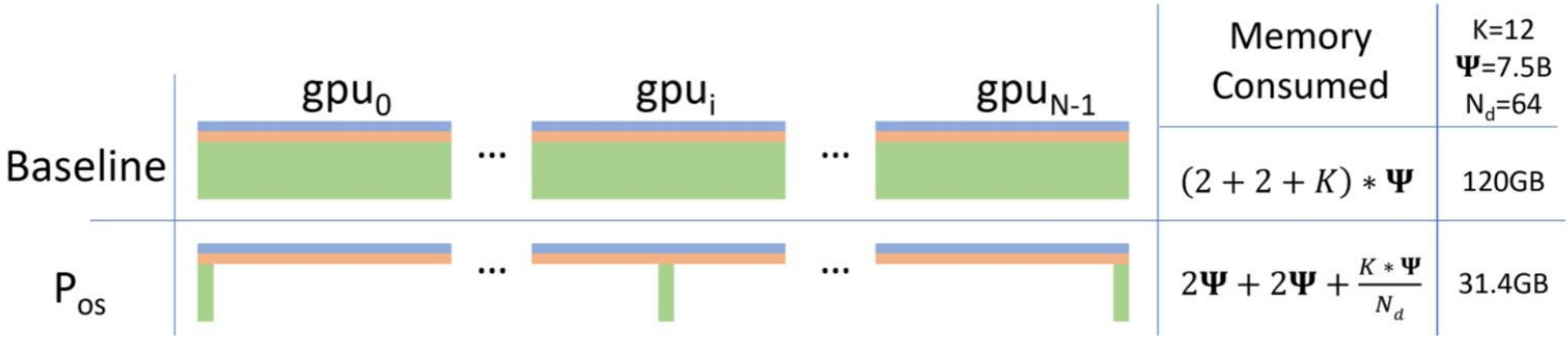
1.8. ZeRO – Solving the Memory Overhead Issue of DP

- Core idea of ZeRO (Zero Redundancy Optimizer)
  - Split up the expensive parts (state) and use the reduce-scatter equivalence.



1.9a. ZeRO Stage 1: Optimizer State Sharding (Pos)

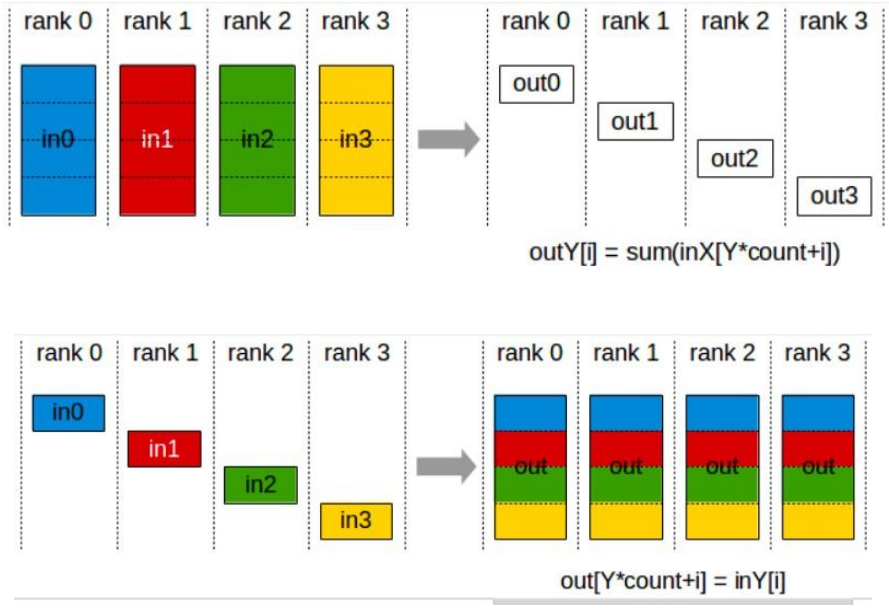
- Split up the optimizer state (first + second moments) across GPUs
- Each worker has the full parameters, and computes gradient on its subset of data.
- Each worker is responsible for updating a shard of the full parameters



	Memory Consumed	K=12 Ψ=7.5B N_d=64
Baseline	$(2+2+K)*\Psi$	120GB
P_{os}	$2\Psi+2\Psi+\frac{K*\Psi}{N_d}$	31.4GB

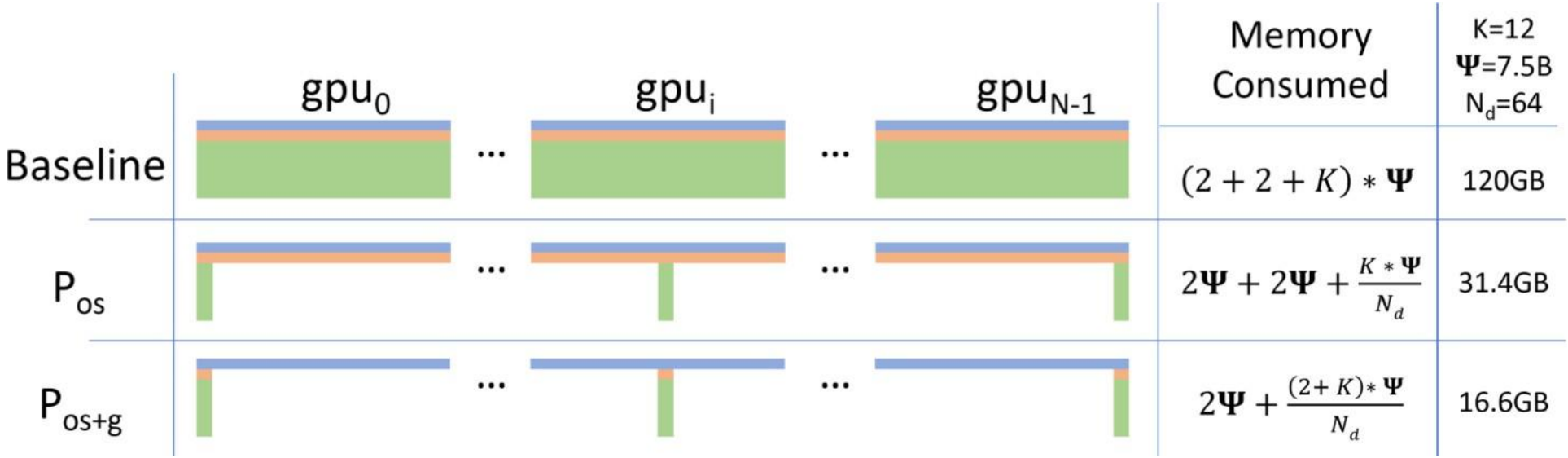
1.9b. ZeRO Stage 1: Optimizer State Sharding (Pos)

- Each worker computes a full gradient on their subset of the data
- ReduceScatter the gradients: each worker gets the full gradient corresponding to their parameter shard (#params communication cost).
- Each worker updates their param using their gradient + state.
- Perform an AllGather to synchronize params
  - #params communication cost



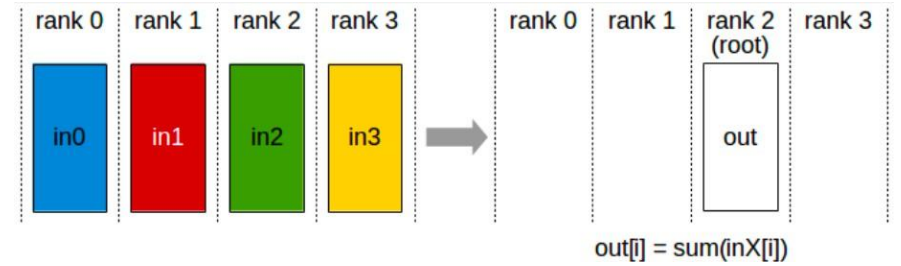
1.11a. ZeRO Stage-2: Optimizer State + Gradient Sharding (Pos+g)

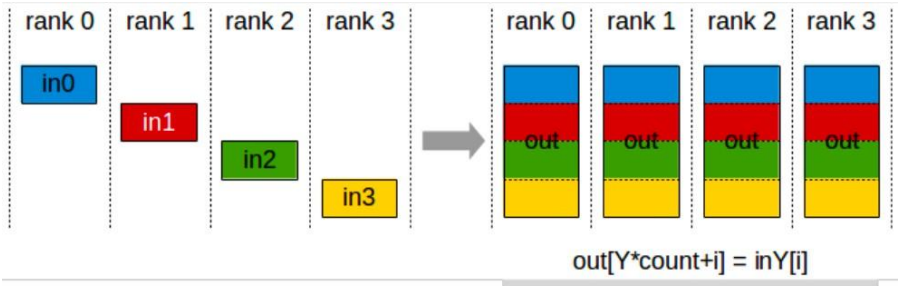
- High-level idea:
  - Also keep the gradients (pink slices) sharded across the machines.
  - Use the same (rough) tricks as stage 1.
- Complexity: we can never instantiate a full gradient vector, but each worker must compute a full gradient (since we're data parallel)



1.11b. ZeRO Stage-2: Optimizer State + Gradient Sharding (Pos+g)

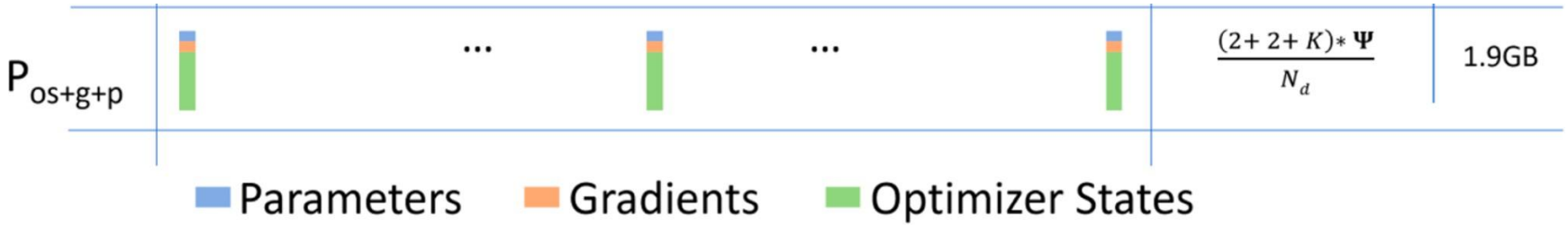
- Step 1. Everyone incrementally goes backward on the computation graph
  - Step 1a. After computing a layer's gradients, immediately reduce to send this to the right worker.
  - Step 1b. Once gradients are not needed in the backward graph, immediately free it.
- Step 2. Each machine updates their param using their gradient + state.
- Step 3. All Gather the parameters.





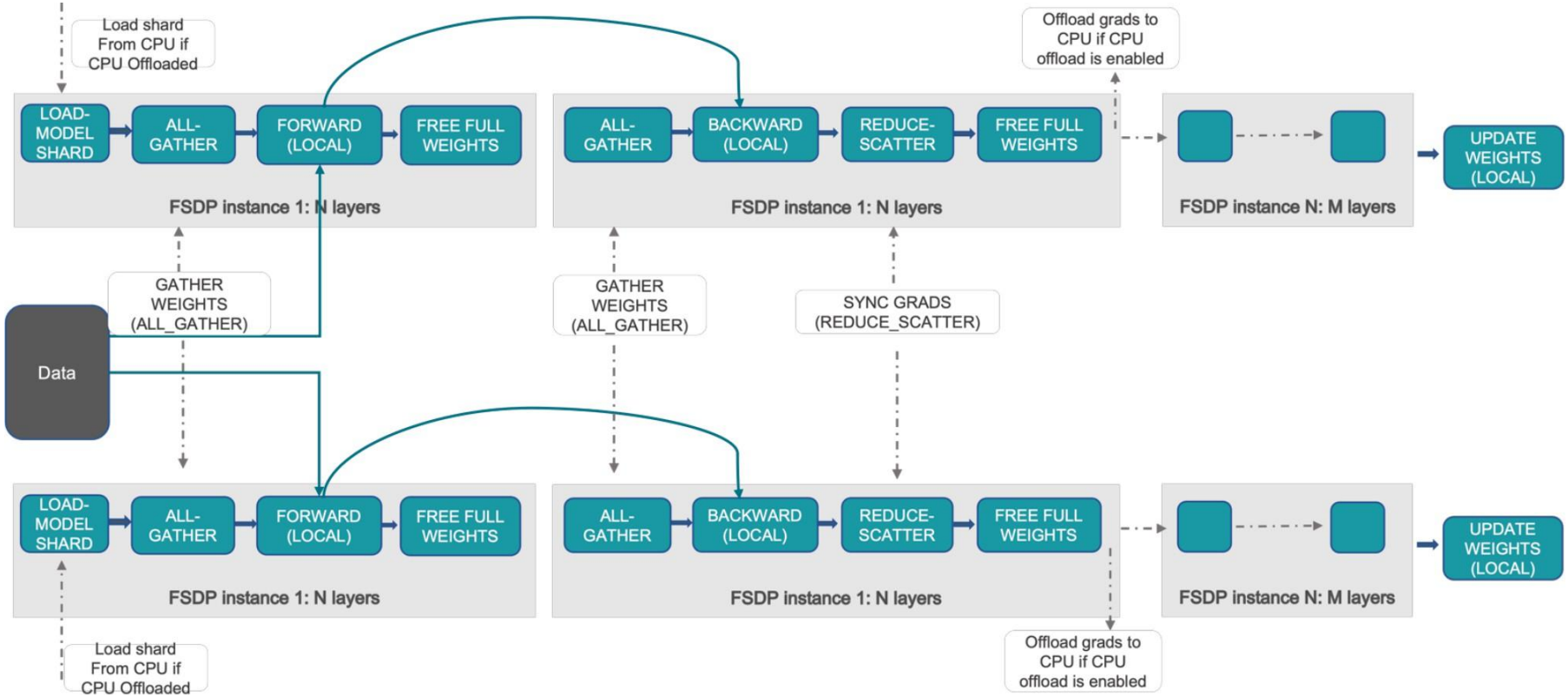
1.12a. ZeRO Stage-3 (aka FSDP) Shard Everything

- High-level ideas
  - Shard everything, including parameters!
  - Use the same “incremental communication / computation” ideas
  - Send and request parameters on demand while stepping through the compute graph.



1.12b. ZeRO Stage-3 (aka FSDP) Shard Everything

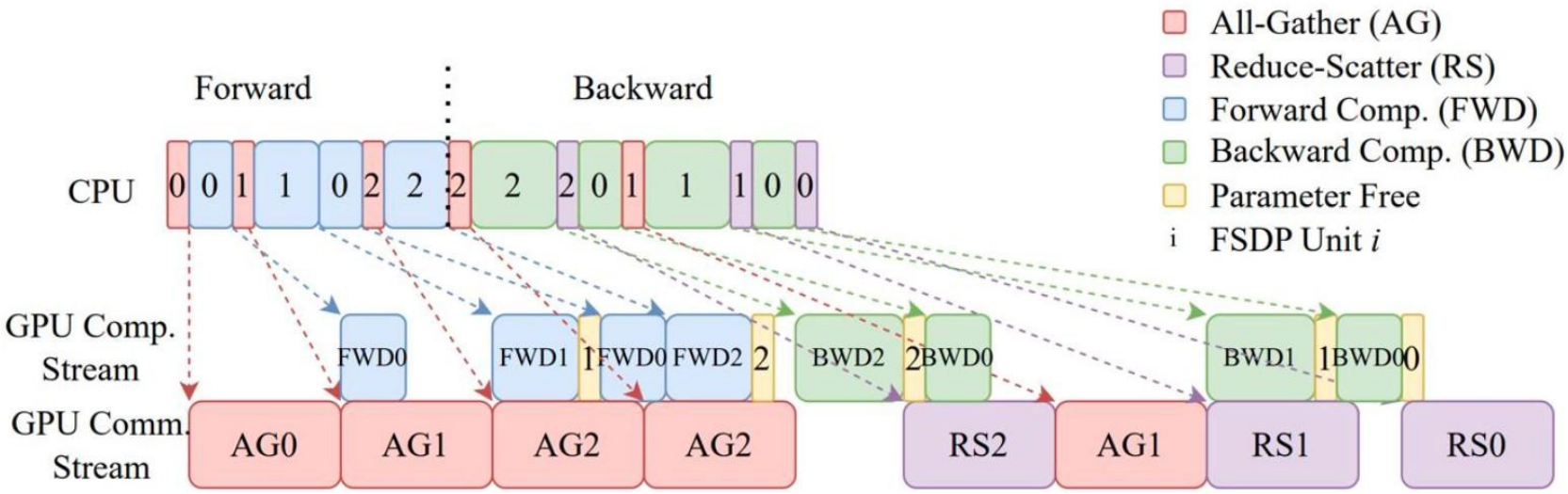
- Communication cost – 2 all gather (#param), 1 reduce-scatter (#param).



https://pytorch.org/tutorials/intermediate/FSDP_tutorial.html

1.13. FSDP / ZeRO-3: Actual Picture

- Incremental computation / communication
  - Parameters / gradients are requested / sent and then immediately freed
- Overlapping communication and computation $(W_1W_0 + W_2W_0)x = y$
  - The all-gathers happen all at once while forward happens, masking the comm cost.



Y. Zhao et al., PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel, PVLDB, 16(12):3848–3860, 2023.

1.14. Communication Overhead Recap

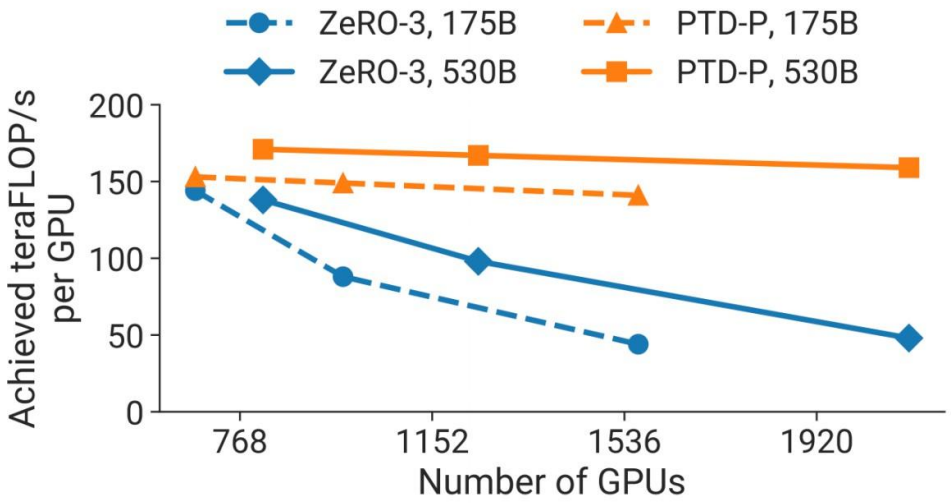
- DDP (All reduce): 2x #params
- ZeRO Stage-1 / 2 (Reduce-Scatter + All-gather): 2x #params
  - Memory saved for free
- ZeRO Stage-3 / FSDP (All-gather + Reduce-scatter + All-gather): 3x #params
  - More overhead
- ZeRO stage 1 is free (in the bandwidth limited regime) memory wins.

	Naïve DDP	ZeRO-1
Communication Primitive	One all-reduce (full gradients)	One reduce-scatter (collect gradient slices) + one all-gather (sync the params)
Communication Cost	2x #params	2x #params
Memory	(4 + K) × #params	(4 + K / nGPUs) × #params

K is the bytes used by optimizer states for each param (typically 12 for FP32 momentum)

2.2. Issues remain with data parallel – models don’t fit

- ZeRO-1 and ZeRO-2 don’t let you scale memory
- ZeRO-3 is nice in principle, but can be slow and does not reduce activation memory



Activations ∝
seq_len × hidden_size ×
num_layers × global_batch_size

Out-of-memory (OOM) explosion !

Better ways to split up the model is needed…

1.16. Section Conclusion

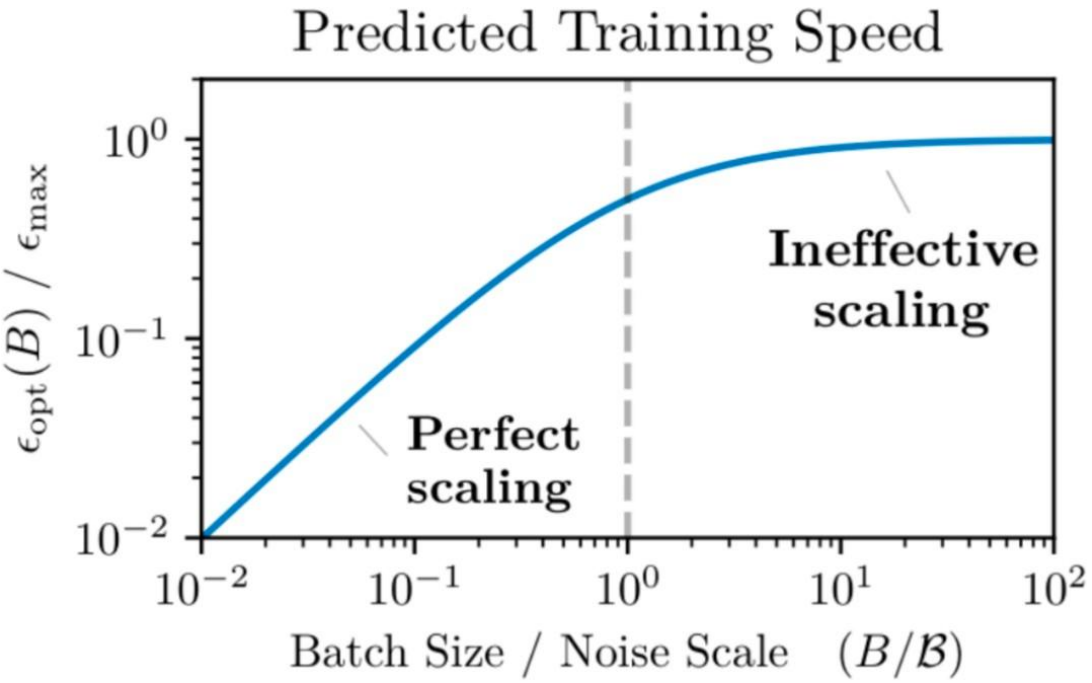
- Summary: Training massive LLMs requires breaking the single-GPU memory barrier via state sharding.
- ZeRO and FSDP elegantly solve capacity issues by partitioning states without sacrificing data-parallel simplicity.
- The Next Bottleneck: As models reach hundreds of billions of parameters, even the pipelined layer-by-layer communication of FSDP bottlenecks the training cluster.
- Solution: We must slice the actual matrix mathematics themselves → Tensor Parallelism.

2. Tensor Parallelism

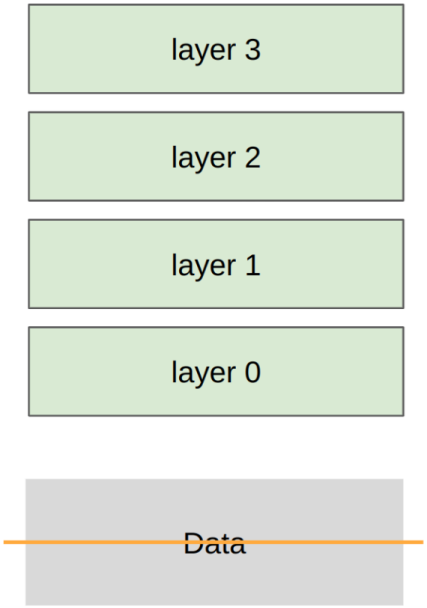
- Basic Concept
- MLP & Attention Partitioning
- When to use TP / Limitations

2.1. Issues remain with data parallel – compute scaling

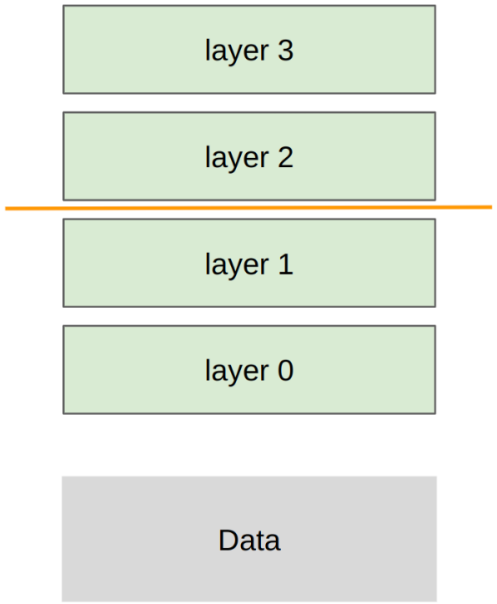
- With data parallel, #machines < batch size (and near this, comm overhead is high)
- And there's diminishing returns to batch sizes



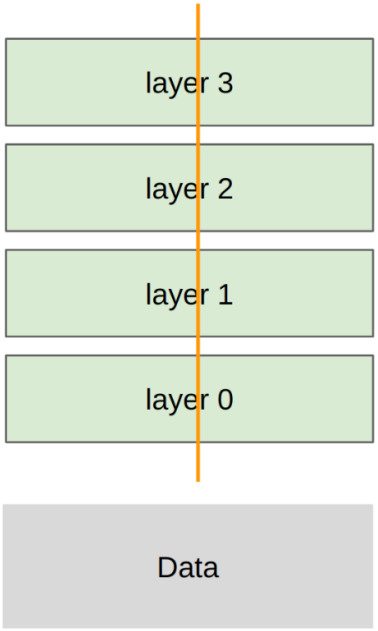
2.3. Model parallel along the different axes



Data Parallel



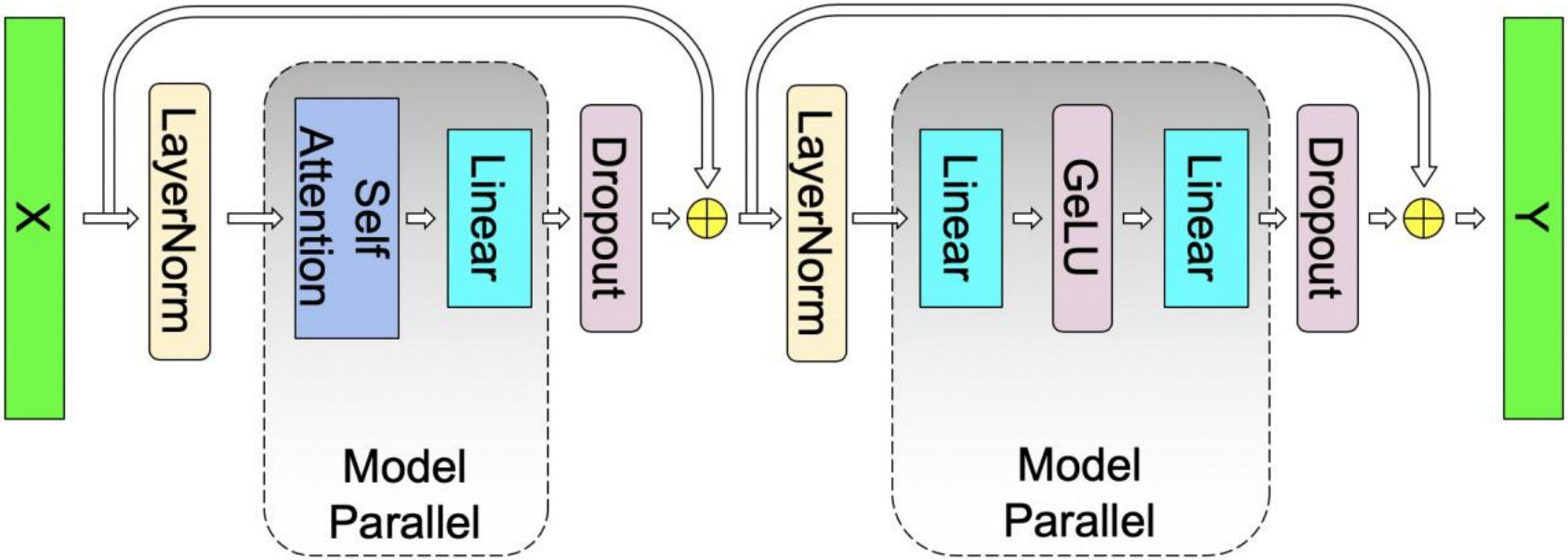
Pipeline Parallel



Tensor Parallel

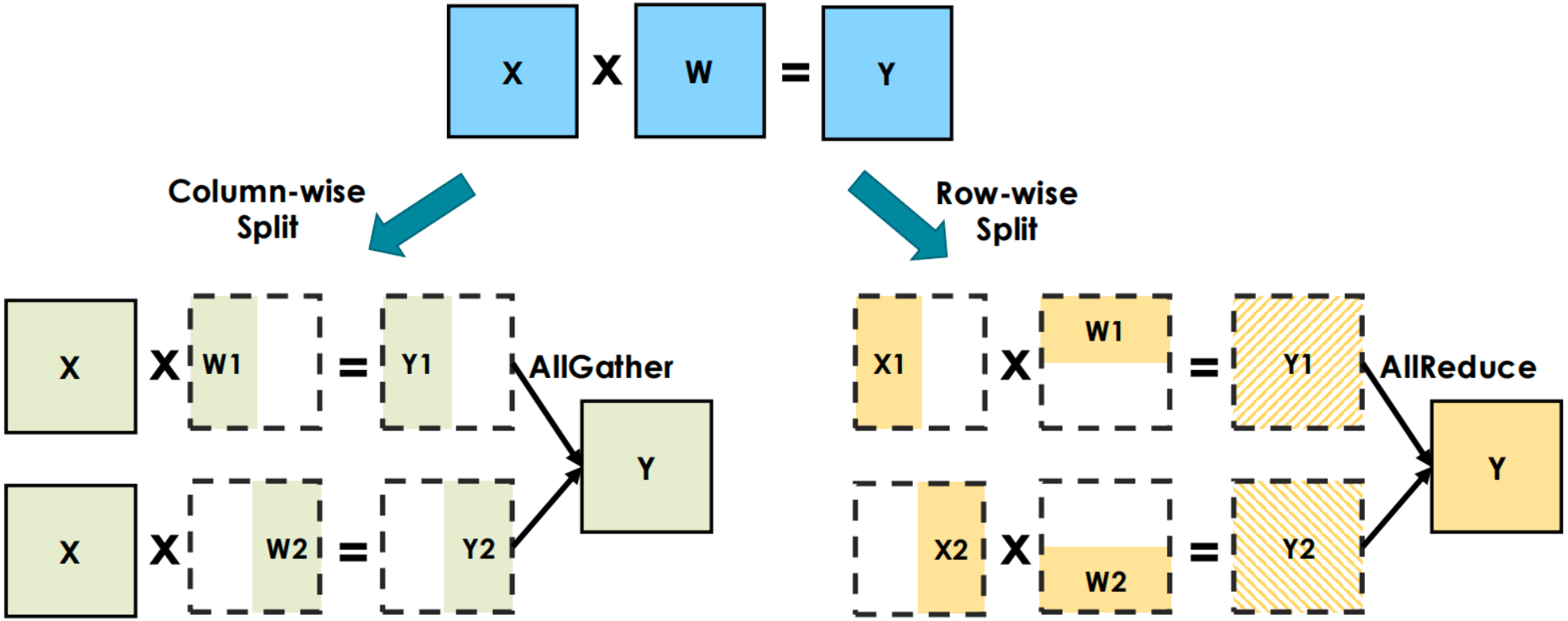
2.4a. Tensor parallel – Basic Concept

- The vast majority of LLM parameters reside in **Linear Projections:**
  - **Attention:** Q, K, V, and Output projections.
  - **MLP:** Up and Down projections.
- These projections are **General Matrix Multiplications (GEMM)** ($Y = XW$)



2.4b. Tensor parallel – Basic Concept

- Two ways to partitioning the GEMM to different GPUs：



2.5a. Tensor parallel – Deconstructing the MLP Block

- An MLP block consists of a linear expansion, a non-linear activation (GeLU), a linear projection back to hidden dimension and a dropout.
- Mathematically,

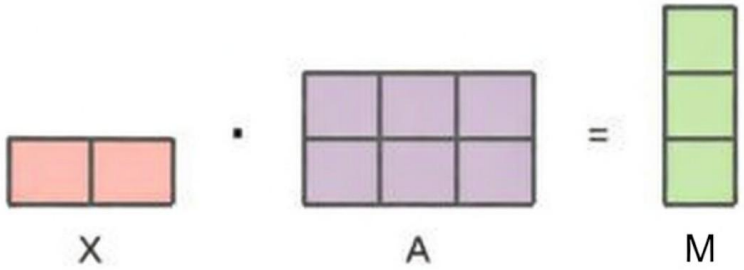
$$Y = \mathrm{ReLU}(xA)$$

$$Z = \mathrm{Dropout}(Y\,B)$$

2.5b. Tensor parallel – Deconstructing the MLP Block

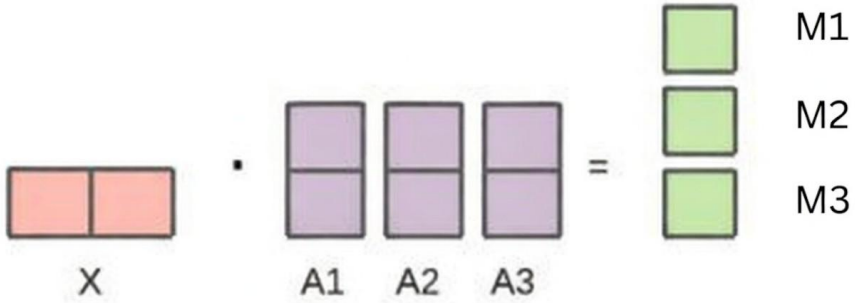
- Row-wise split of A: requires synchronization
- Column-wise split of matrix A: the output can be fed into GeLU independently

$$Y = \mathrm{ReLU}(xA)$$



is equivalent to

$$[Y_1, Y_2] = \mathrm{ReLU}(x[A_1, A_2])$$

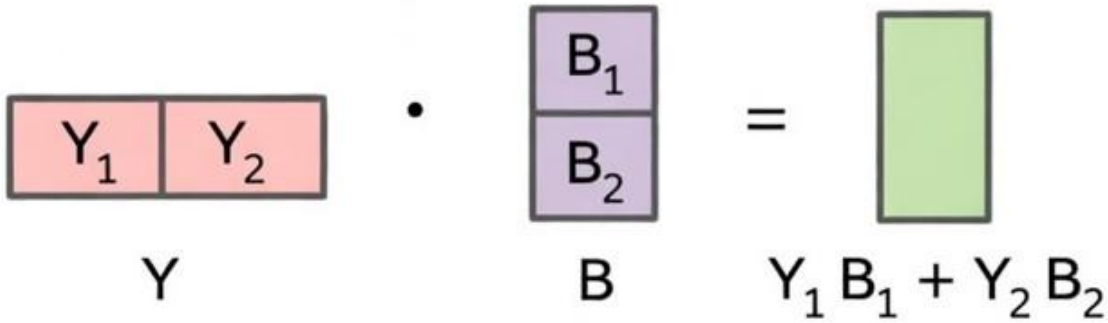


2.5c. Tensor parallel – Deconstructing the MLP Block

- Column-wise split of matrix A → Column-wise split of Y
- Column-wise split of Y + Row-wise split of B: block-wise matrix multiplication requires no synchronization

$$[Y_1, Y_2] = \mathrm{ReLU}(x[A_1, A_2])$$

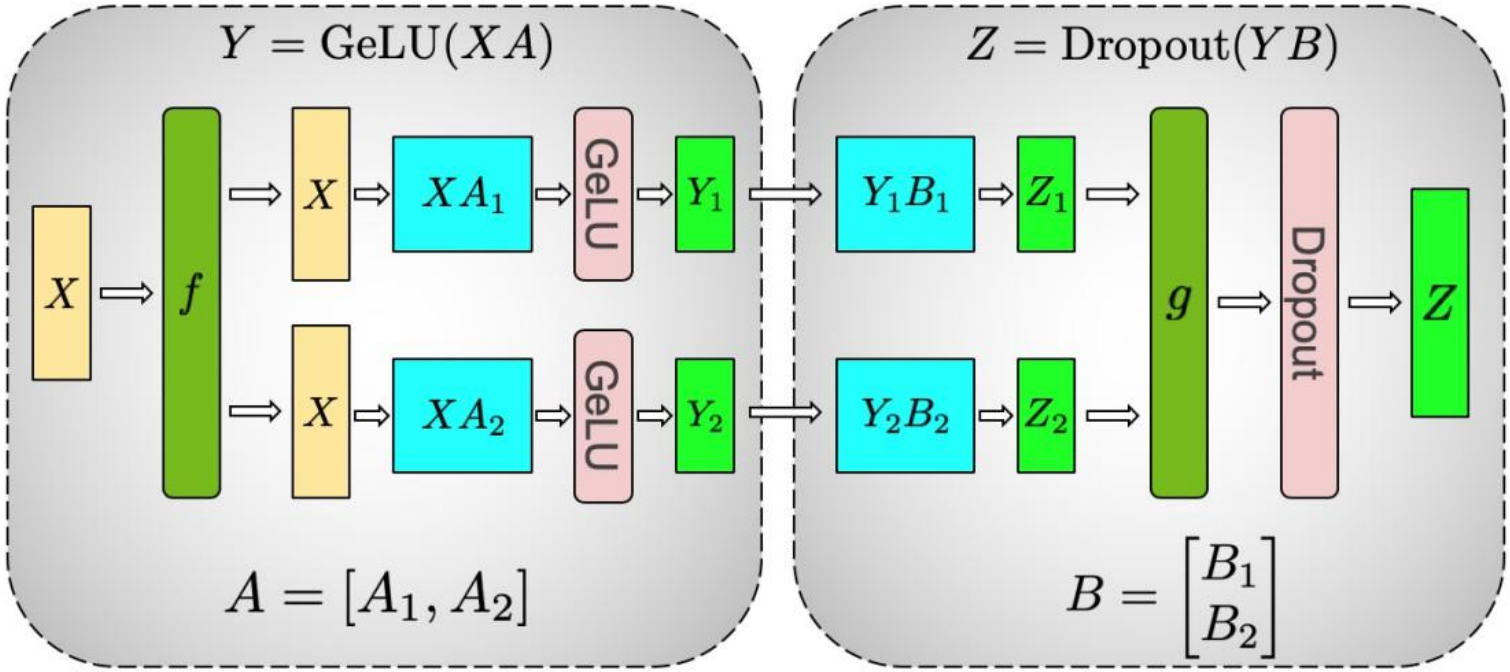
$$Z = \mathrm{Dropout}(Y\,B)$$



2.5d. Tensor parallel – Deconstructing the MLP Block

- Assign columns (A_1, A_2) and rows (B_1, B_2) to separate GPUs.
- In the forward pass, f is the identity, and g is an all-reduce.
- In the backward pass, f is an all-reduce, g is the identity.

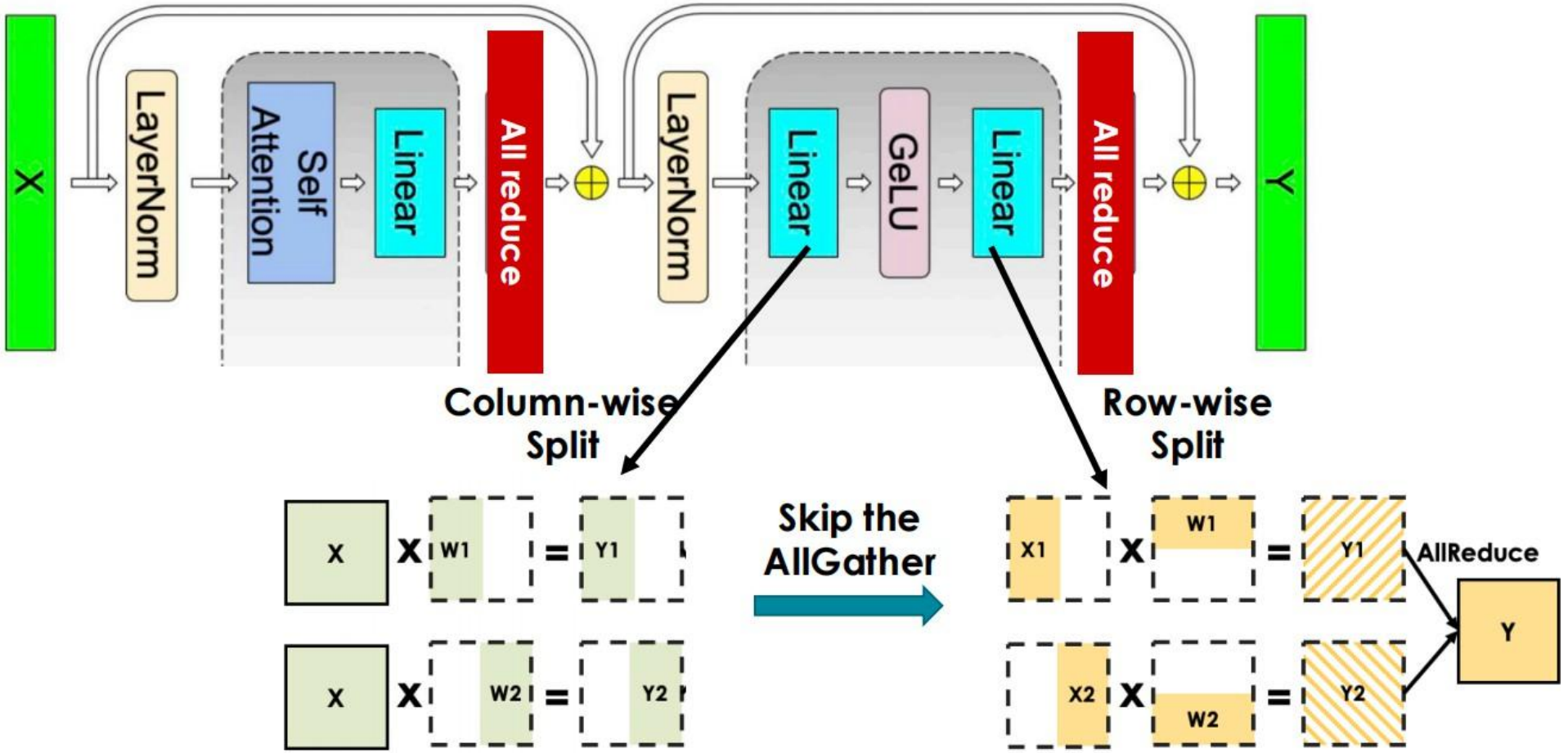
MLP w.o. TP $\Longrightarrow$



MLP w. TP $\Longrightarrow$ $[Y_1, Y_2] = \mathrm{GeLU}(x[A_1, A_2])$ $Z = \mathrm{Dropout}([Y_1, Y_2]({}_{B}{}^{1}))$

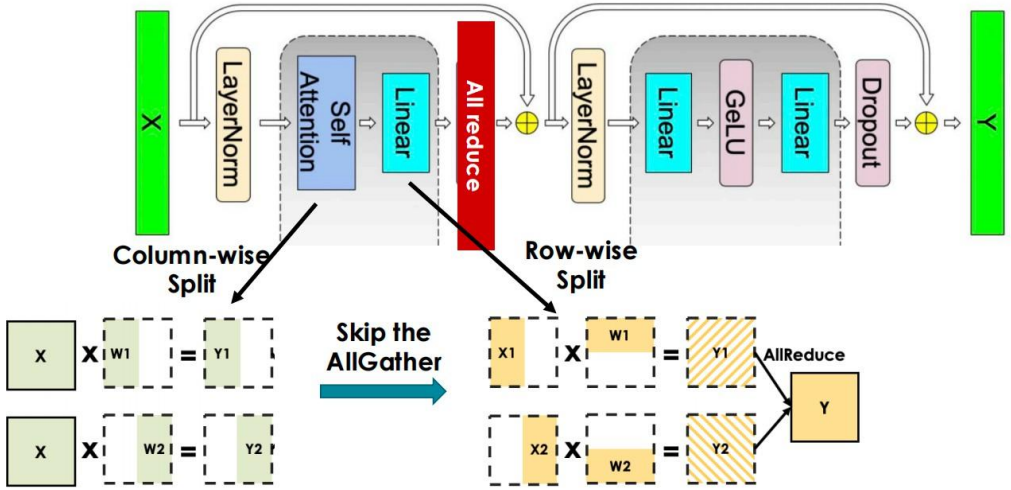
2.6. Tensor parallel – Tensor Parallelism for MLP

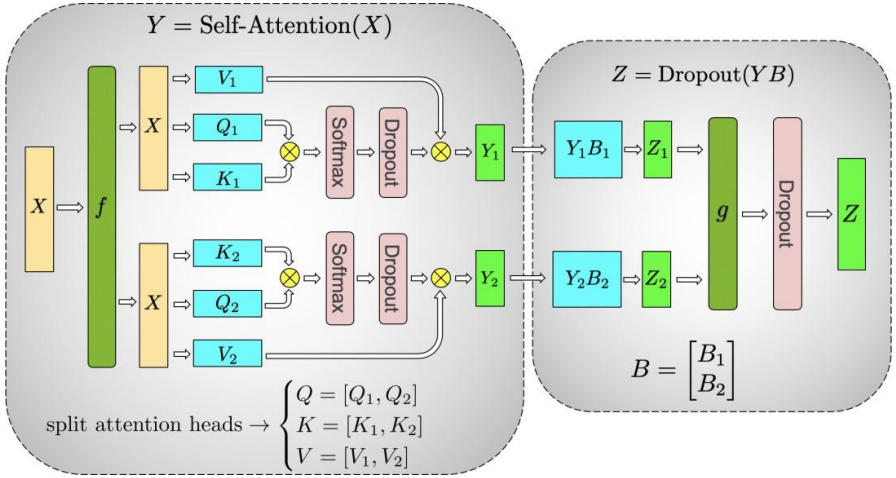
- Interleaving split to minimize communication:



2.7. Tensor parallel – Tensor Parallelism for Attention

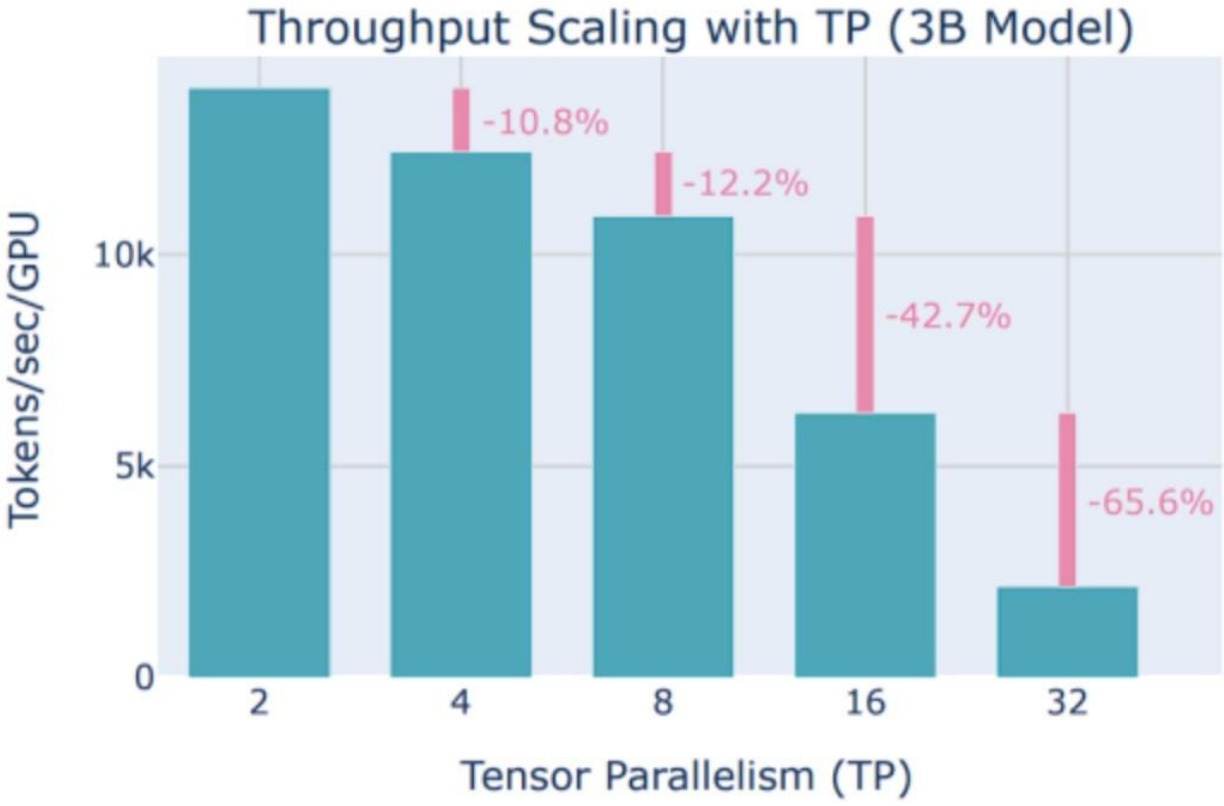
- Interleaving split to minimize communication:





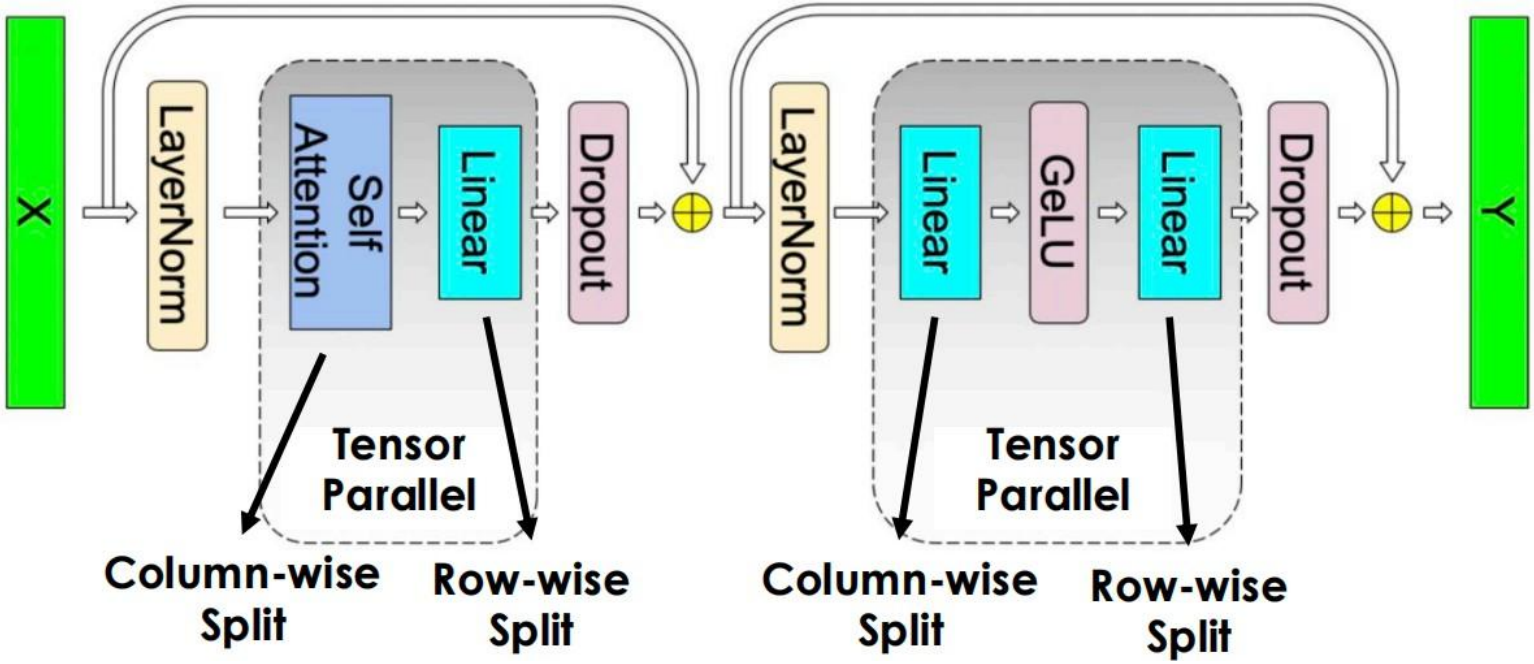
2.8. When to use TP?

- On GPUs, tensor parallel within a node (up to 8 GPUs) due to high speed interconnects.



2.9. Limitation of TP

- TP only splits GEMM
- LayerNorm and DropOut are still replicated
  - **Full activation memory overhead** [batch_size*seq_len, hidden_dim] on each TP partition

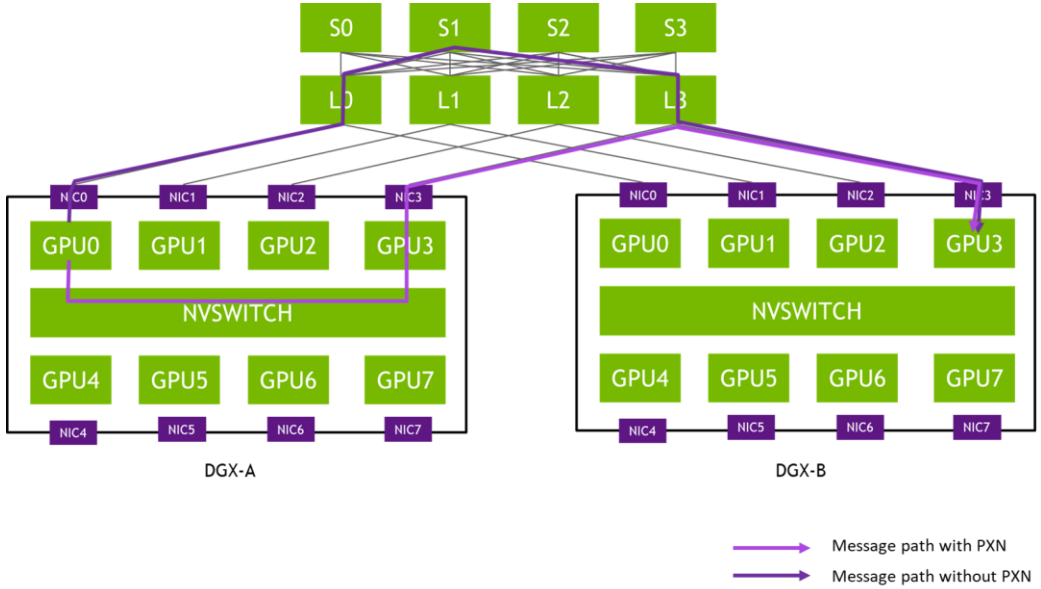


3. Pipeline Parallelism

- F-then-B
- 1F1B / Interleaved 1F1B
- Advanced PP schedules

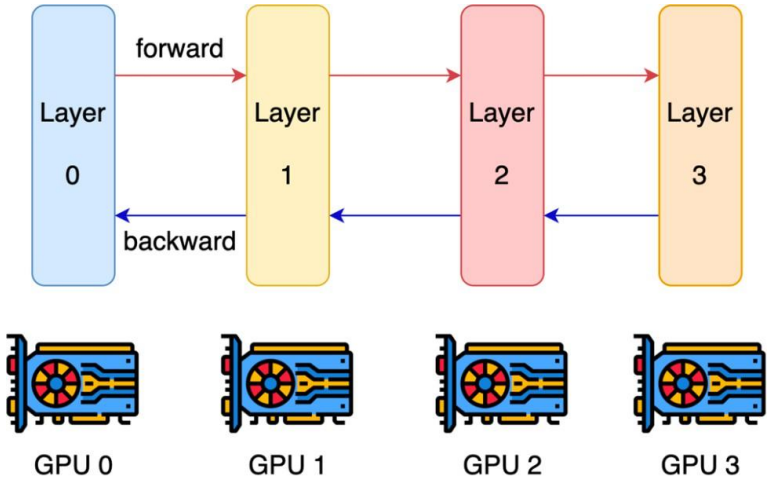
3.1. Introduction to Inter-Node Scaling

- TP is computationally efficient but mathematically requires synchronous All-Reduce operations, rendering it unsuitable for inter-node scaling.
  - Ethernet/Infiniband latency would catastrophically bottleneck GPU utilization.
  - To scale deep LLMs across multiple nodes, the system must slice the model longitudinally.
- Pipeline Parallelism (PP) assigns sequential blocks of layers to different physical nodes.

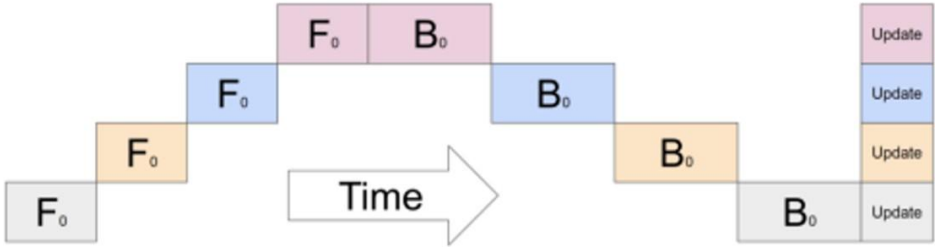


3.2. Pipeline Parallelism (PP)

- Layer-wise parallel cuts up layers, assigns some subset to GPUs (K stages)
- Activations and partial gradients are passed back and forth



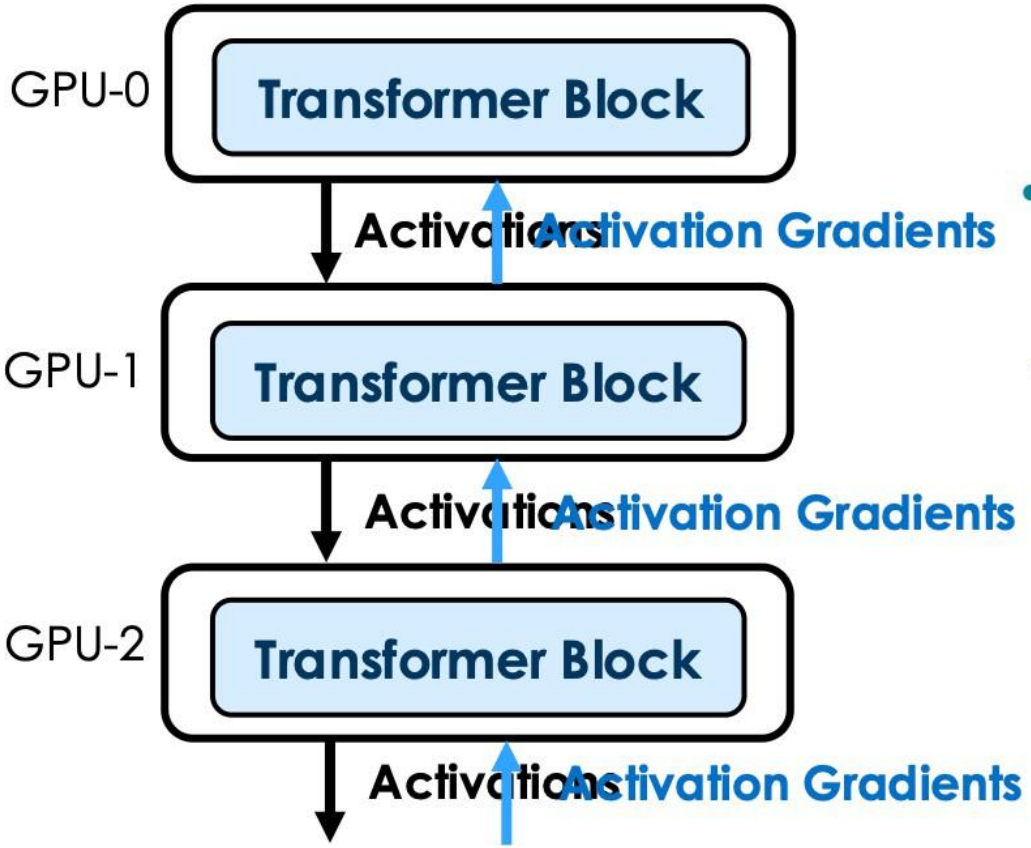
With n gpus, each gpu is active $\frac{1}{n}$ of the time.



Each GPU is idling most of the time, waiting for the backward pass to propagate back

3.3. Pipeline Parallelism

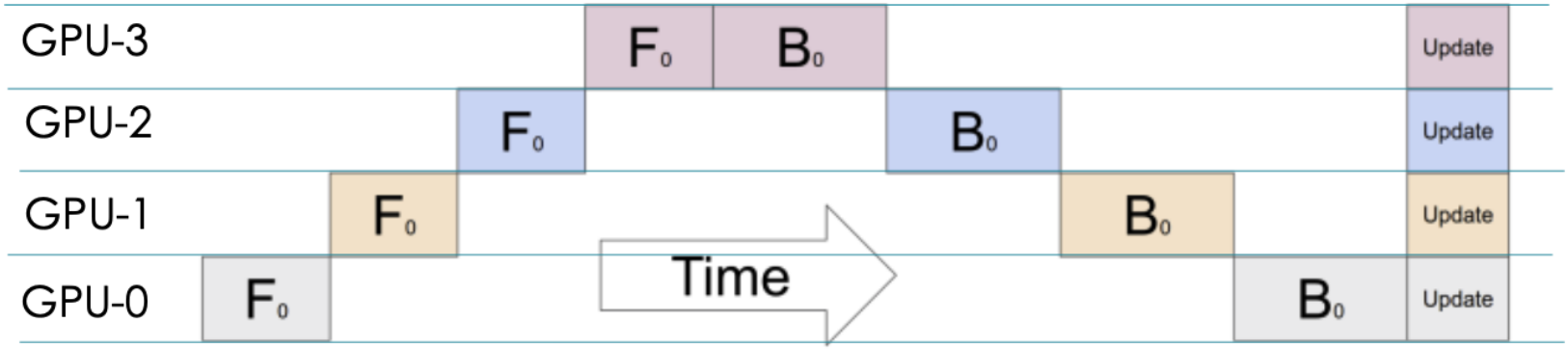
- Communication pattern



- P2P communication for activations in the forward pass
- P2P communication for activation gradients in the backward pass

3.4a. Pipeline Parallel: Forward-then-Backward

Execution timeline

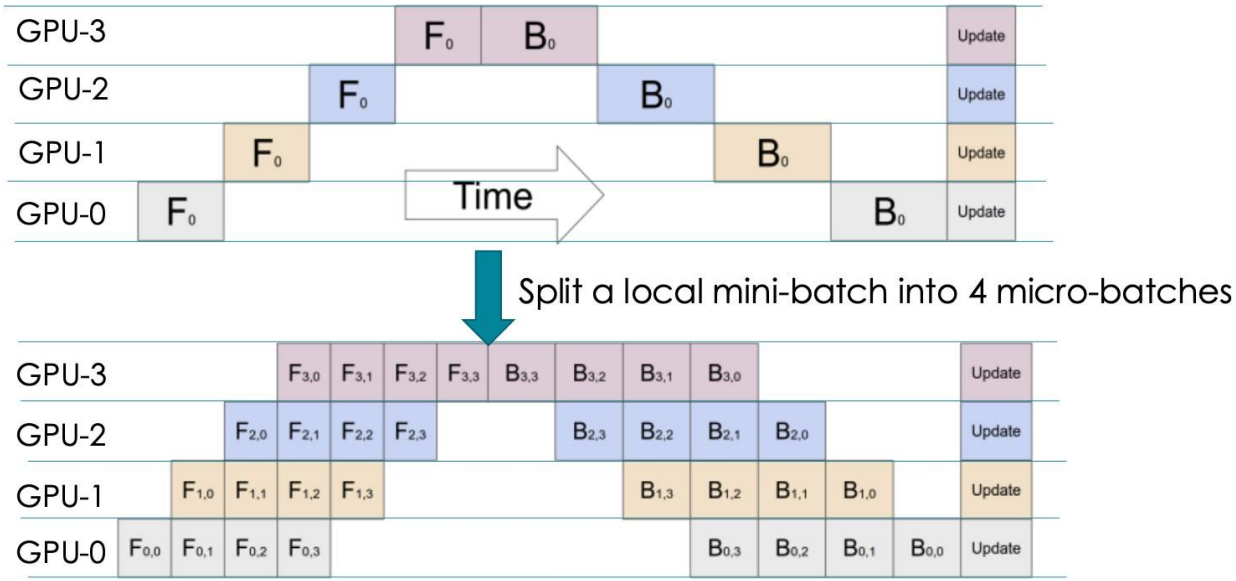


How to make GPUs busy?

Batch split!

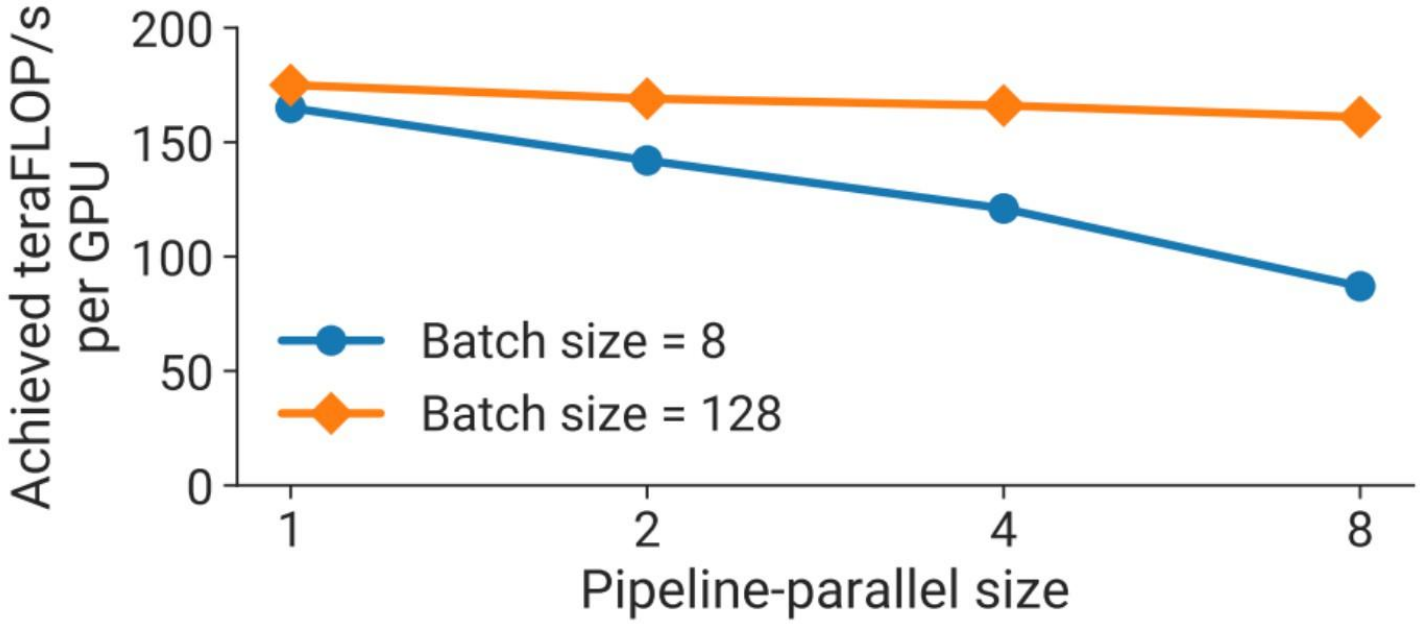
3.4b. Pipeline Parallel: Forward-then-Backward

- GPipe (Google, 2019)
- Divide the global mini-batch into M smaller “micro-batches”
- To maximize TFLOPS, increase Global Batch Size to fit more micro-batches into the pipe.



3.4c. Pipeline Parallel: Forward-then-Backward

- GPipe bubble ratio analysis
  - Pipeline must “fill up” at the start and “drain” at the end, causing the inherent bubble.
  - GPipe bubble rate defined mathematically as: $\frac{K-1}{M}$ (K: PP degree; M: #micro-batches)
  - To minimize the bubble rate, systems engineers must set $M \gg K$
  - Example: If $K = 4$ and $M = 16$, the bubble rate drops dramatically to approximately 18%.



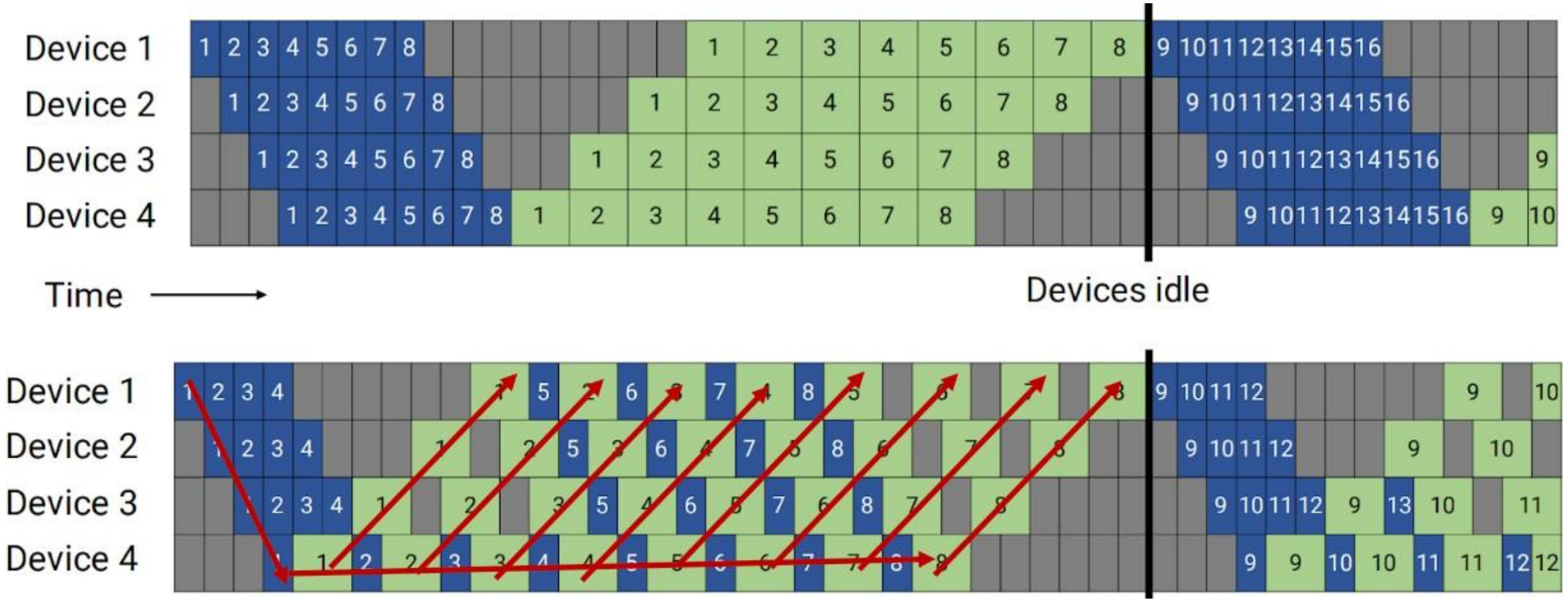
3.5. Why pipeline parallel?

1. Pipelines save memory (compared to DDP)
2. Pipelines can have good communication properties (compared to FSDP and TP)
    - It depends only on activations and is point to point

Generally, we will use pipelines on slower network links (i.e. inter-node) as a way to get better memory-wise scaling.

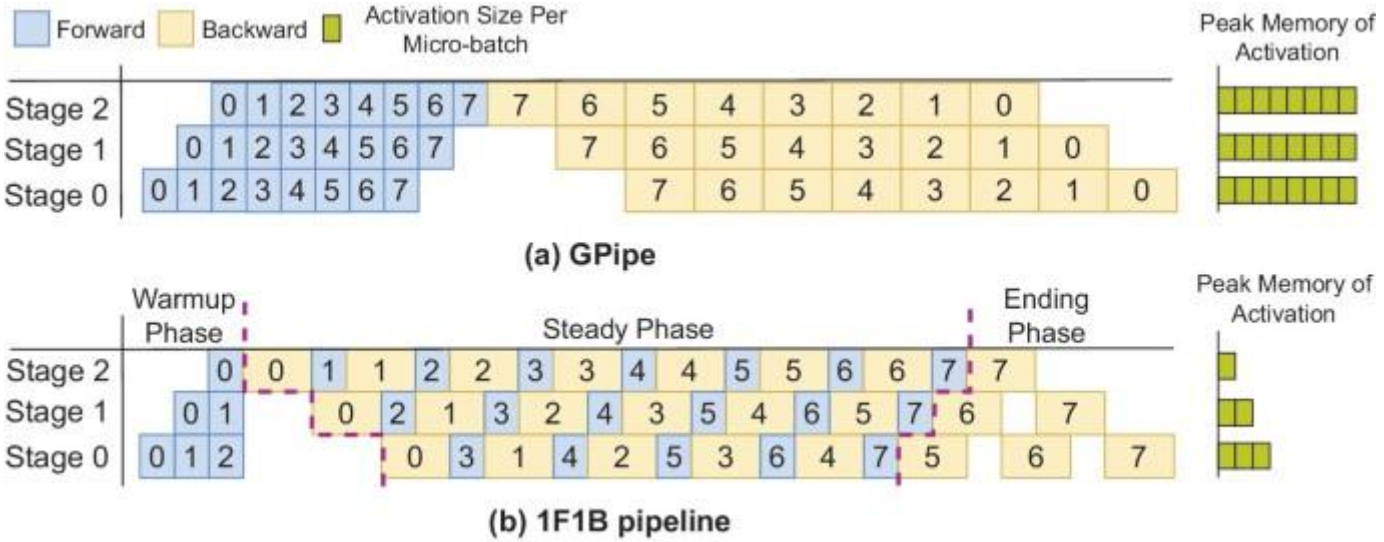
3.6a. Pipeline Schedule Optimization

- PipeDream proposes a “1F1B” schedule
  - Once the warm-up phase concludes, GPUs alternate: compute one forward pass, immediately compute one backward pass.
  - Prioritizes backward passes to change the lifecycle of cached data.



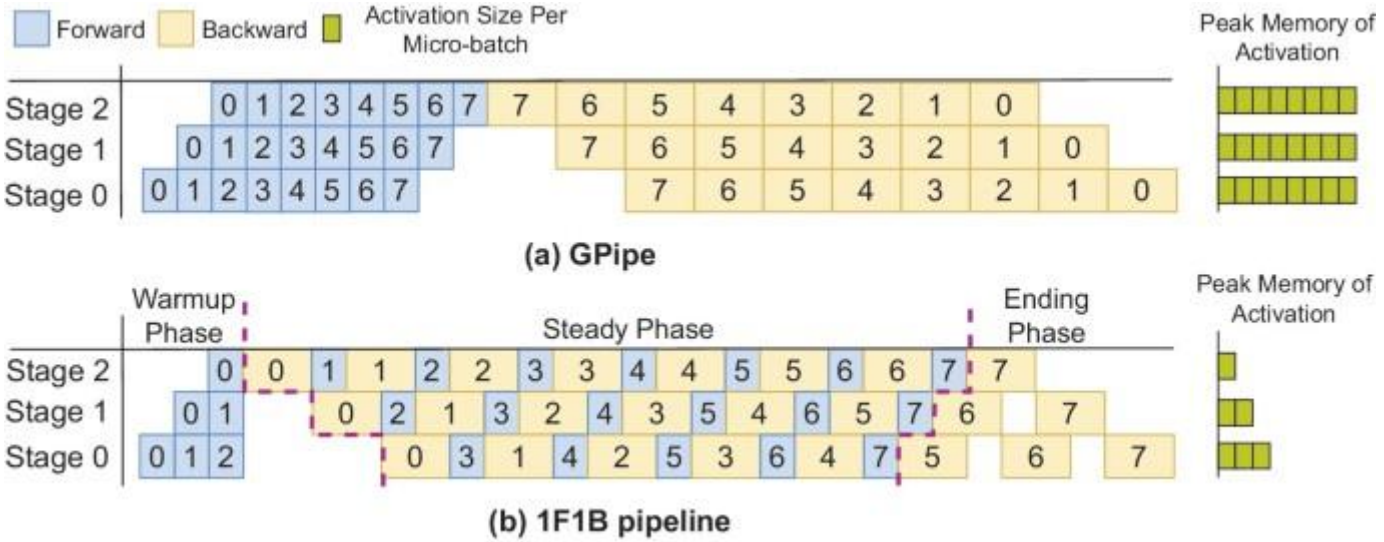
3.6b. Pipeline Schedule Optimization

- GPipe memory crisis
  - GPipe is strictly synchronous: all M forward passes must complete before any backward pass begins.
  - Backward passes absolutely require intermediate activations.
  - Every pipeline stage must cache activations for all M micro-batches simultaneously.
  - As M scales to shrink the bubble, activation memory scales linearly O(M), rapidly triggering OOM



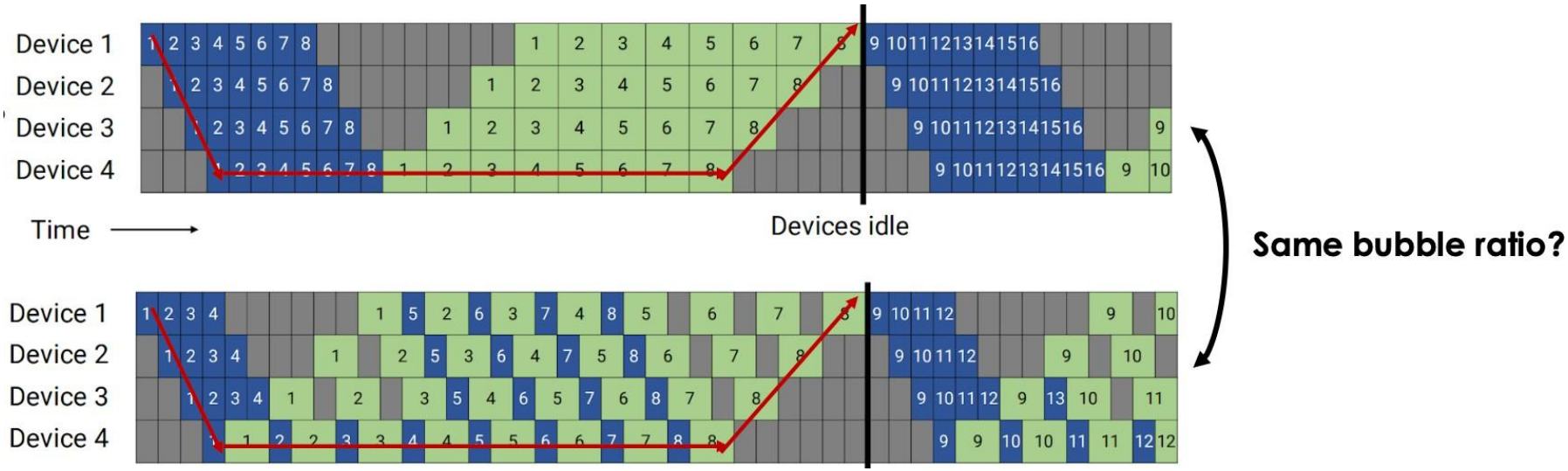
3.6c. Pipeline Schedule Optimization

- 1F1B memory footprint
  - 1F1B executes backward passes as early as mathematically possible.
  - Heavily cached activations are consumed for gradients and instantly discarded.
  - Max in-flight activations any GPU must store is strictly bounded by pipeline depth K, rather than M.
  - Decouples peak activation memory from micro-batch count, allowing infinite M scaling (memory permitting).



3.6d. Pipeline Schedule Optimization

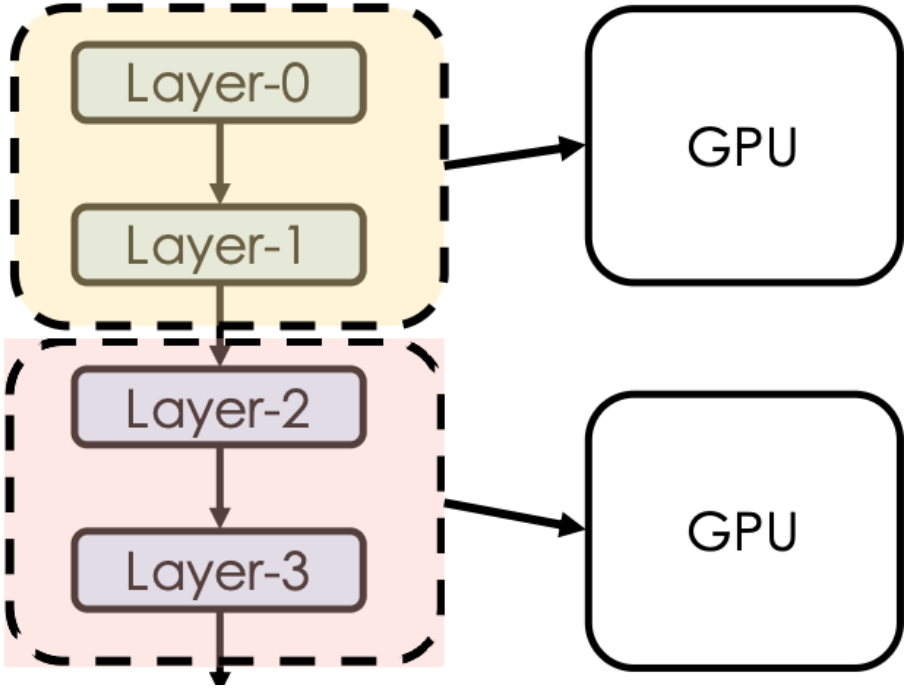
- 1F1B bubble ratio
  - A common misconception is that 1F1B is faster than GPipe.
  - Mathematically, 1F1B contains the exact same aggregate amount of idle time (fill and drain phases).
  - The bubble rate remains strictly: $\frac{K-1}{M}$
  - 1F1B's sole advantage over GPipe is the algorithmic reduction in peak activation memory footprint, not raw theoretical throughput.



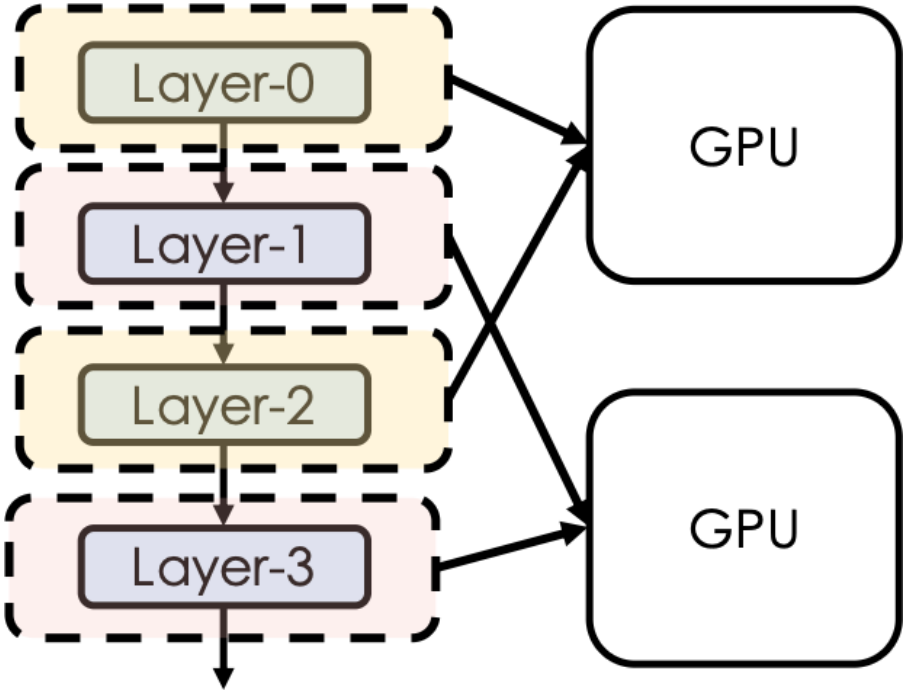
3.7. Limitations of 1F1B and Larger Batch Sizes

- To achieve negligible bubble rates (<5%), M must be exceedingly high.
- Global batch size is mathematically constrained by optimizer convergence limits.
- If global batch size is fixed, scaling M shrinks individual micro-batch sizes toward 1.
- Small micro-batches underutilize modern GPU Tensor Cores, destroying arithmetic intensity.
- A novel schedule is required to reduce bubbles without increasing M.

3.8a. Interleaved 1F1B (Megatron-LM)



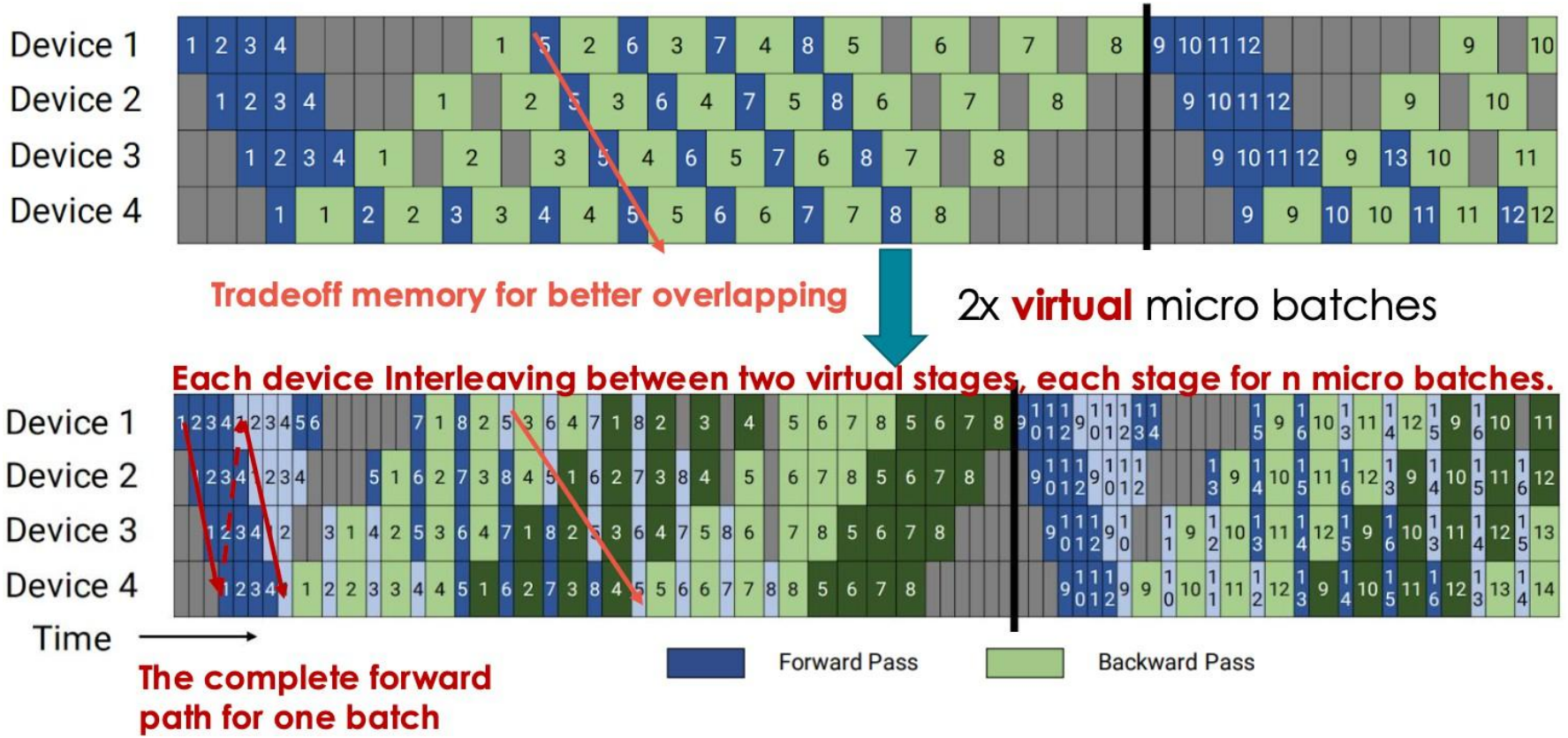
One micro-batch will go through both GPU but only once



One micro batch will go through both GPU but now twice.
GPU are now busy working, just like we have two batches

3.8b. Interleaved 1F1B (Megatron-LM)

- Divides the model into a larger number of “virtual stages” (V stages per GPU).
  - GPU 1 is assigned Virtual Stage 1 (e.g., layers 1-2) and Virtual Stage 5 (e.g., layers 9-10) simultaneously



3.8c. Interleaved 1F1B (Megatron-LM)

- Interleaving V virtual stages compresses the bubble time proportionally.
- The new bubble rate formulation: $\frac{K-1}{MV}$
  - MV is limited by number of the layers in the model
- Trade-offs
  - V times more communication
  - Peak activation memory increases slightly due to overlapping life-cycles.

3.8d. Interleaved 1F1B (Megatron-LM)

- Megatron-core recommends Interleaved 1F1B by default
  - #layers in virtual pipeline state typically set to 2-4.

```
from megatron.core import parallel_state
from megatron.core.pipeline_parallel import p2p_communication

----------------------------------------------------------

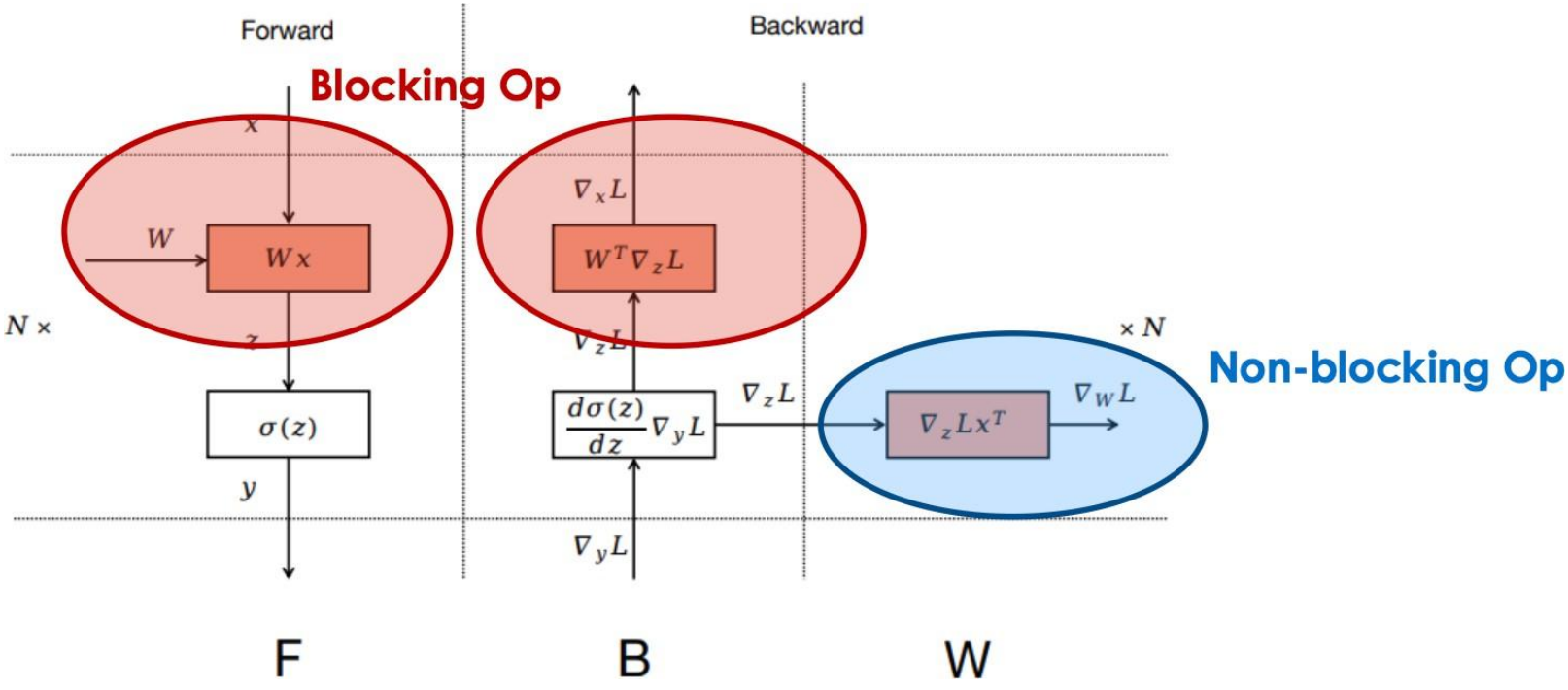
# 1. Pipeline Parallel Size
# A value of 4 means the model is split across 4 GPUs/Groups.
--pipeline-model-parallel-size 4 \

# 2. Interleaved Pipeline Schedule
# Enables "Virtual Pipeline Stages" to reduce the pipeline bubble.
# Setting this to 2 or more splits each stage into smaller chunks
# to overlap computation and communication more effectively.
--num-layers-per-virtual-pipeline-stage 2 \

# 4. Micro-batch & Global-batch Settings
--micro-batch-size 1 \
--global-batch-size 1024 \
```

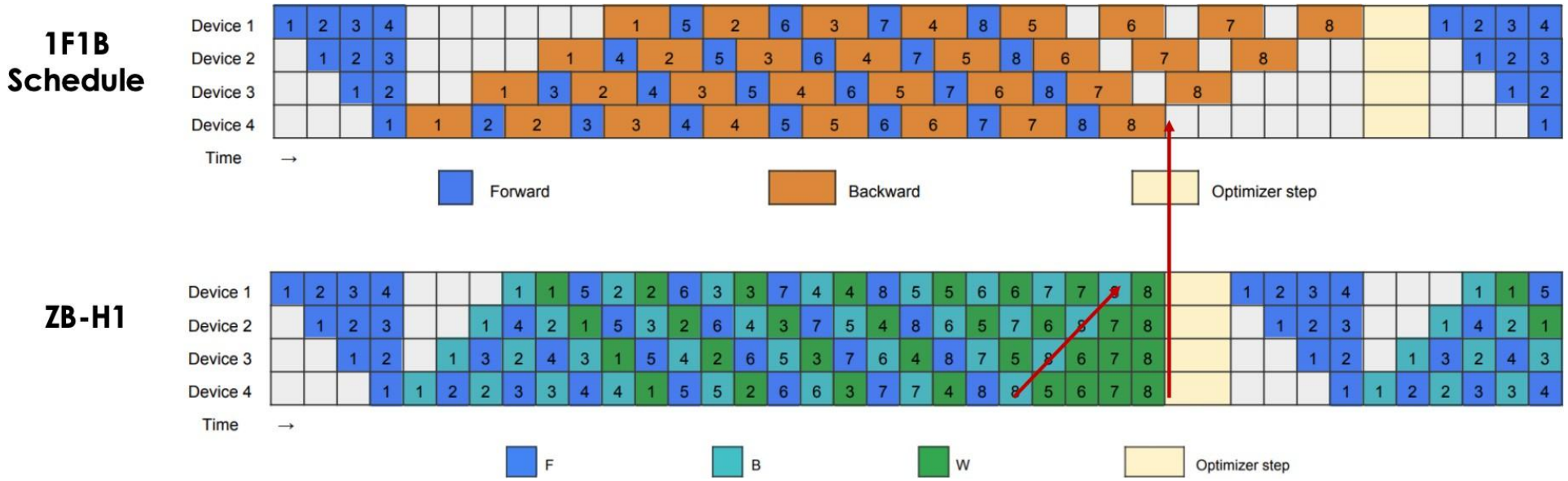
3.9a. Advanced PP: Zero-bubble Pipeline (ICLR'24)

- Key insight:
  - Separate blocking and non-blocking computation to enable more flexible pipeline schedule.



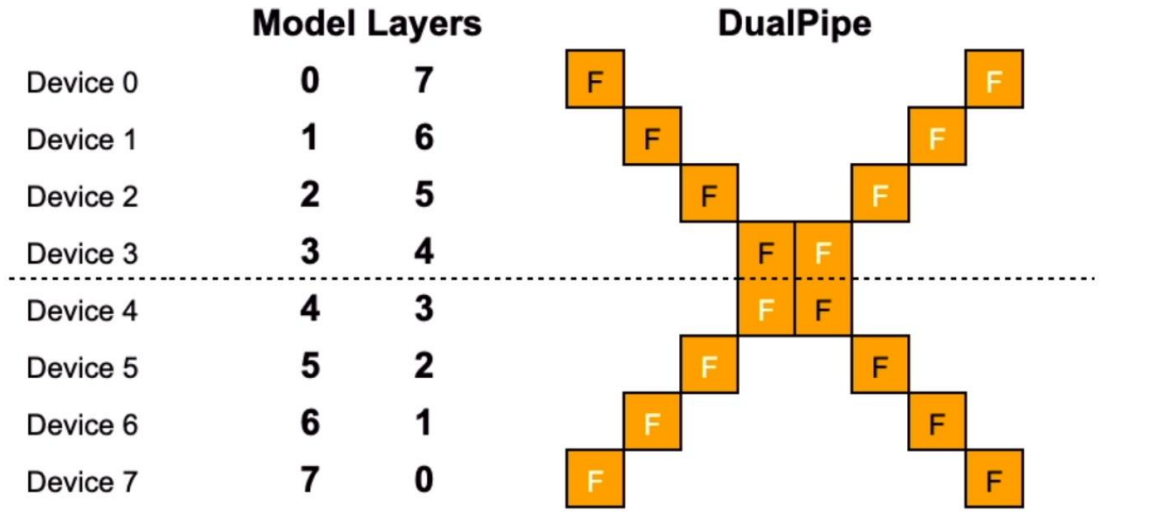
3.9b. Advanced PP: Zero-bubble Pipeline (ICLR'24)

- Small scale: Integer Linear Programming, for both memory and end-to-end makespan optimization
- Large scale: heuristics
- Less #PP communications than interleaved 1F1B, but similar/less bubbles.

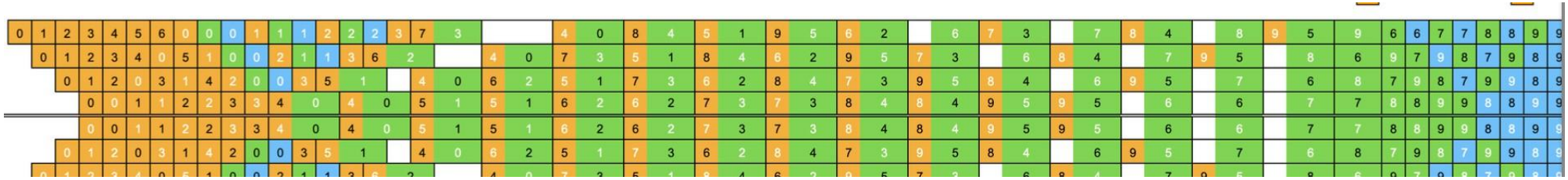


3.10. Advanced PP: DeepSeek-V3 DualPipe (2025)

- Sharding strategy



Memory overhead: now all devices together stores 2 copies of the #parameters.



DeepSeek-AI, DeepSeek-V3 Technical Report, arXiv:2412.19437, 2024.

3.11. Advanced PP

Approach	Bubble Ratio	Convergence	Weights Memory	Activations Memory	Extra Computation Overhead
GPipe	$(D-1)/(T+D-1)^*$	Excellent	M_θ	$T \times M_a$	
GEMS	$\approx (D-1)/(D+1/2)^*$	Excellent	$2M_\theta$	M_a	
DAPPLE	$(D-1)/(D+T-1)^*$	Excellent	M_θ	$[M_a, D \times M_a]$	
Chimera	$(D-2)/(2T+D-2)^*$	Excellent	$2M_\theta$	$[(D/2+1)M_a, D \times M_a]$	
Megatron-LM	$(D-1)/(v \times T)$	Excellent	M_θ	$T \times M_a$	
ZeroBubble (ZB-H2)	$\approx 0\%$	Excellent	M_θ	$(2D-1) \times M_a$	X
AMPNet	$\approx 0\%$	Poor	M_θ	$[M_a, D \times M_a]$	
PipeDream	$\approx 0\%$	Good	$[M_\theta, D \times M_\theta]$	$[M_a, D \times M_a]$	
XPipe	$\approx 0\%$	Good	M_θ	$[M_a, D \times M_a]$	X
SpecTrain	$\approx 0\%$	Good	M_θ	$[M_a, D \times M_a]$	X
PipeDream-2BW	$\approx 0\%$	Good	$2M_\theta$	$[M_a, D \times M_a]$	
PipeMare	$\approx 0\%$	Good	M_θ	$[M_a, D \times M_a]$	X
AvgPipe	$\approx 0\%$	Good	$P \times M_\theta$	$[1, D \times T] \times P \times M_a$	
WPipe	$\approx 0\%$	Good	$2M_\theta$	$T \times M_a$	

https://selamjie.medium.com/the-frontier-of-pipeline-parallelism-an-overview-d4264cc9f877

4. Hybrid and Automatic Parallelism

- 3D/5D Parallelism
- Automatic Model Parallelism

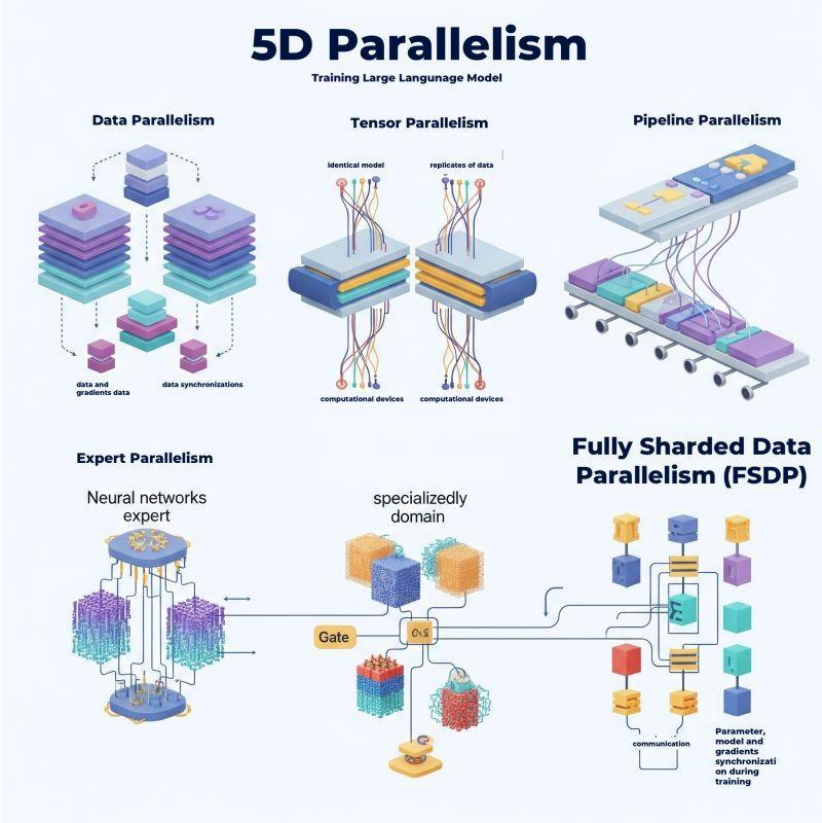
4.1. 3D Parallelism System Architecture

- **Hardware Symbiosis:** Nests DP, PP, and TP to match data center topologies.
- **Inner Loop (TP):** Confined to a single node via NVLink for heavy MatMul communication.
- **Intermediate Loop (PP):** Scales across nodes via RDMA for low-bandwidth stage boundaries.
- **Outer Loop (DP):** Wraps TP+PP logically, scaling globally for massive batch throughput.

Parallelism Dimension	Execution Granularity	Communication Primitive	Physical Mapping
Tensor (TP)	Intra-layer Operations	Dense All-Reduce	Intra-Node (NVLink)
Pipeline (PP)	Inter-layer Stages	Peer-to-Peer Send/ Recv	Inter-Node (InfiniBand)
Data (DP)	Global Batch Replicas	Sparse All-Reduce	Global Cluster

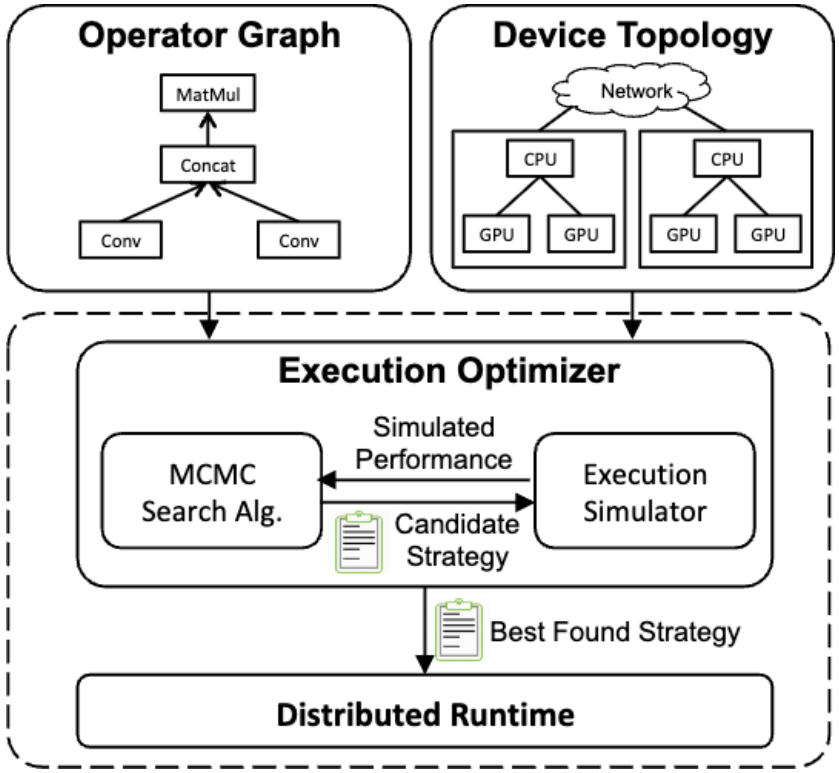
4.2. Expanding to 4D and 5D Parallelism: CP and EP

- **The Catalyst:** Massive sequence lengths and Mixture of Experts (MoE) break 3D parallelism.
- **Context Parallelism (CP):** Partitions tokens across GPUs, exchanging KV states via ring protocols to prevent sequence-induced OOM.
- **Expert Parallelism (EP):** Uses All-to-All routing to dynamically dispatch tokens to discrete MoE experts.
- **The 5D Formula:** $\mathrm{TotalGPUs} = \mathrm{TP} \times \mathrm{CP} \times \mathrm{EP} \times \mathrm{PP} \times \mathrm{DP}$
- **Outcome:** Fractures every bottleneck (dense weights, batch throughput, infinite context, sparse compute).



4.3. Defining the Search Space: The SOAP Framework

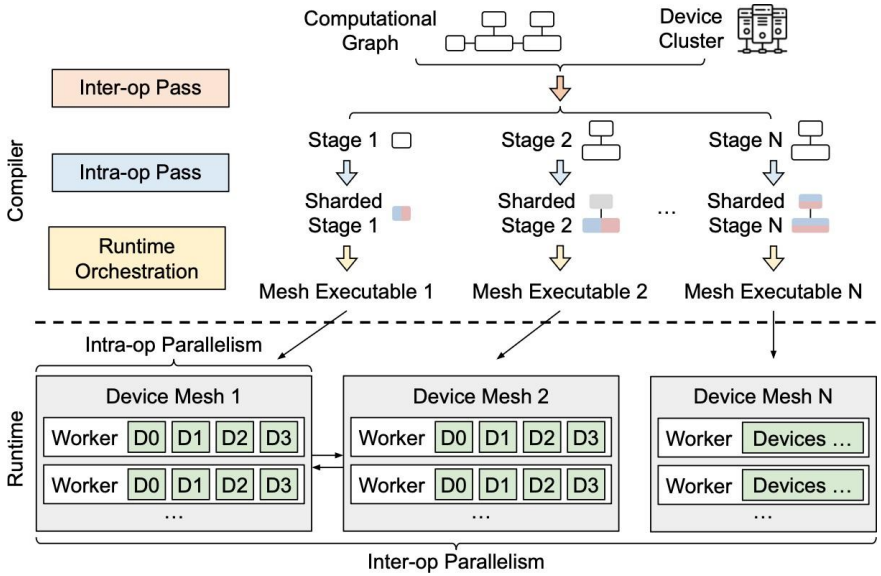
- **FlexFlow's SOAP Dimensions:**
  - **Sample:** DP (batch partitioning).
  - **Operation:** PP (graph node distribution).
  - **Attribute:** CP (sequence/image feature partitioning).
  - **Parameter:** TP (weight matrix slicing).
- **Search Engine:** Uses Markov Chain Monte Carlo (MCMC) to explore the vast configuration space.
- **Oracle Simulator:** Predicts strategy performance in milliseconds instead of hours, discovering non-intuitive plans up to 3.3× faster than manual tuning.



Z. Jia, M. Zaharia, and A. Aiken, Beyond Data and Model Parallelism for Deep Neural Networks, arXiv:1807.05358, 2018.

4.4. Hierarchical Space Search in Alpa

- **The Scaling Issue:** Flat MCMC searches fail at 100B+ parameters due to exponential complexity.
- **Alpa's Hierarchy:** Restructures the search to mirror physical data center bandwidths:
  - **Level 1 (Intra-op):** ILP solves TP/DP assignments within high-bandwidth "device meshes" (intra-node).
  - **Level 2 (Inter-op):** Dynamic Programming solves PP staging across distant meshes (inter-node).
- **Result:** Outperforms specialized manual systems (e.g., DeepSpeed) by up to 9.7× on complex MoE models.



L. Zheng et al., Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning, arXiv:2201.12023, 2022.

4.5. The Future of Distributed Training Frameworks

- **The Trajectory:** Moving from imperative manual engineering to declarative, fully automated compilation.
- **Advanced Scheduling:** Evolving beyond LLM-centric limits (e.g., FlexPipe for programmable PP, Pfeife for non-sequential models).
- **Heterogeneous Training:** Orchestrators will dynamically adapt to clusters with asymmetric memory/bandwidth profiles.
- **Ultimate Goal:** Virtualizing hardware to democratize trillion-parameter AI, making extreme-scale training an out-of-the-box process.

China-Africa AI Compute Faculty Development Program

Module 04b: Distributed Communication Primitives

Guojie Luo
罗国杰
gluo@pku.edu.cn

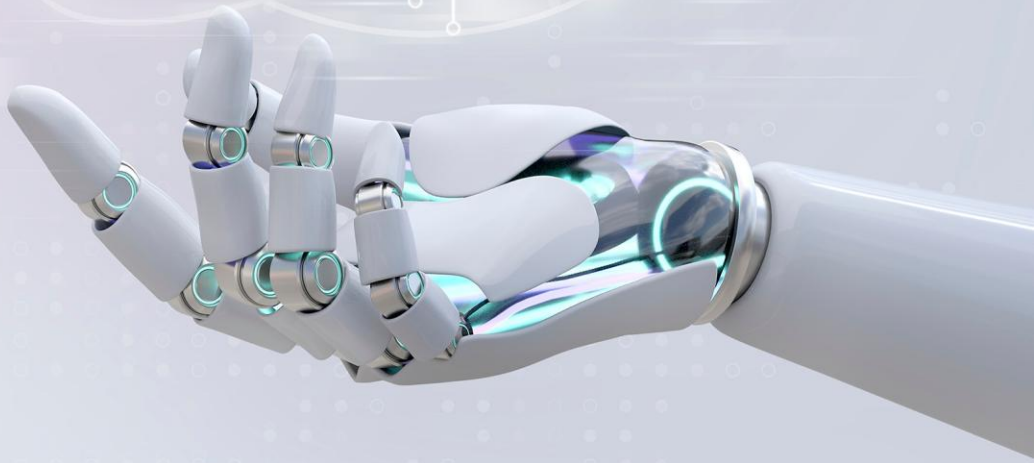
Jing Wang
王晶
jwang@ruc.edu.cn

African Scientific Research and Innovation Council

BAAI 智源研究院

北京大學 PEKING UNIVERSITY

中国人民大學 RENMIN UNIVERSITY OF CHINA



Outline

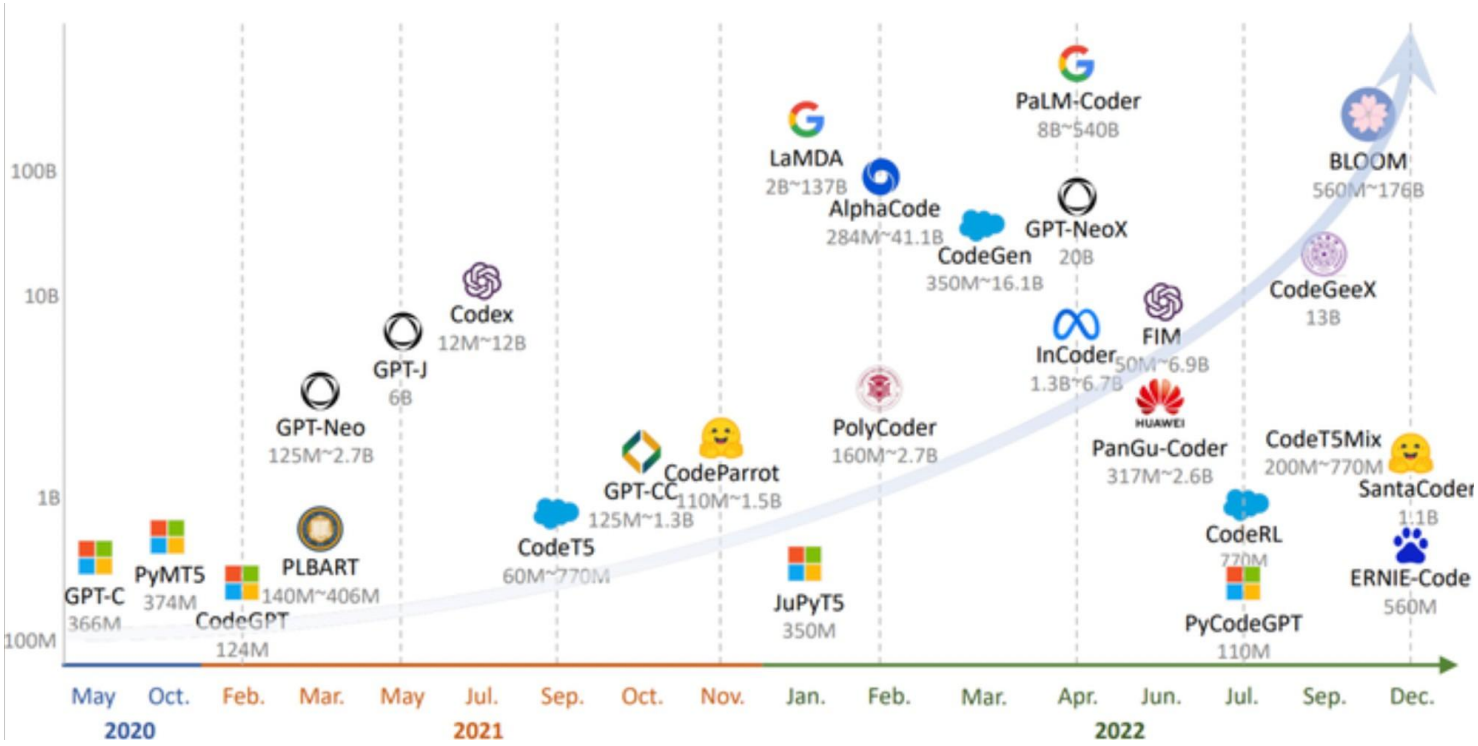
1. Communication Primitives
2. Communication Topology Optimization

1. Communication Primitives

- Scaling Law of LLMs
- AI Computing Systems Are Also Scaling
- Mapping LLMs on Multiple GPUs/NPUs
- Communication Datapath in the AI Systems
- Collective v.s. Point-to-Point Communications
- Collective Ops: Broadcast / Reduce / Gather / Scatter / AllReduce / AllGather / ReduceScatter / AlltoAll
- Take FSDP (PyTorch) as an Example
- Implementations of Collective Communication Op.s
- Topologies in the Modern AI Systems

1.1. Scaling Law of LLMs

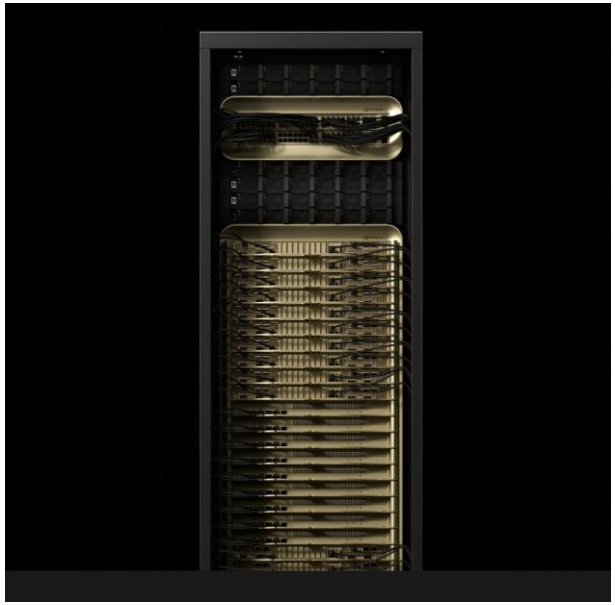
- Model performance improves predictably as a power-law function of compute, data size, and parameter count.



1 J. Kaplan et al., Scaling Laws for Neural Language Models, arXiv:2001.08361, 2020.
2 https://dnacap.fund/insights/exploring-the-landscape-of-large-language-models

1.2. AI Computing Systems Are Also Scaling

- Scale-up supernodes are on the rise…



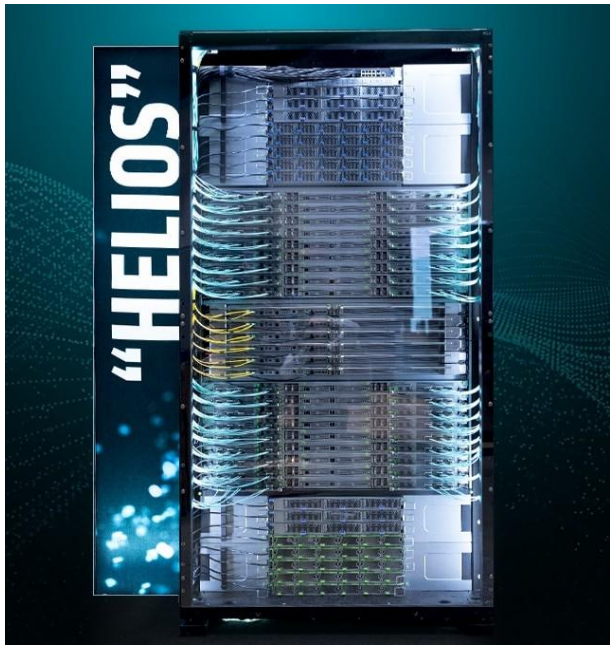
NVL72



NVL576



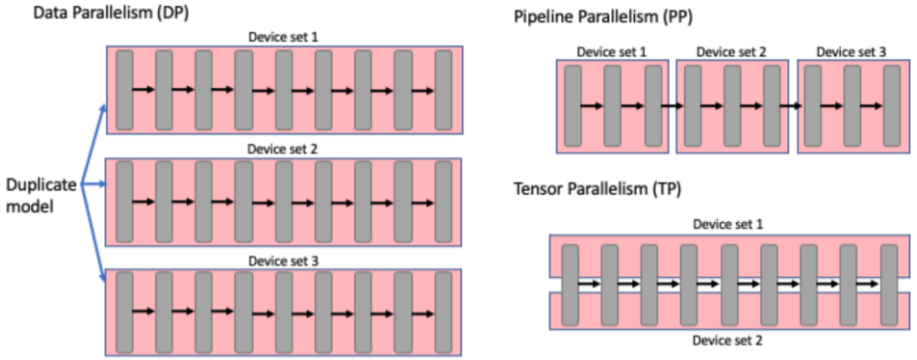
Huawei CloudMatrix384

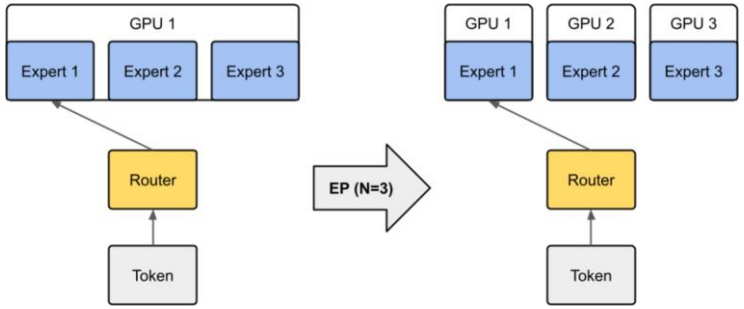


AMD Helios

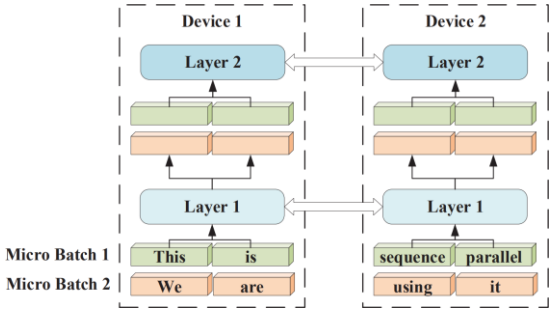
1.3. Mapping LLMs on Multiple GPUs/NPUs…

- Multi-dimension Parallelisms





Expert Parallelism

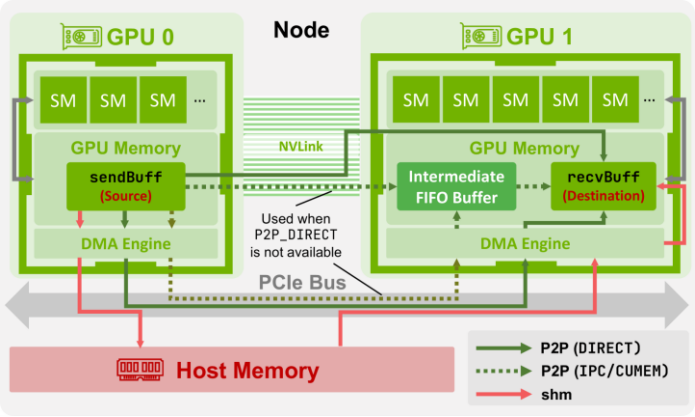


Sequence Parallelism

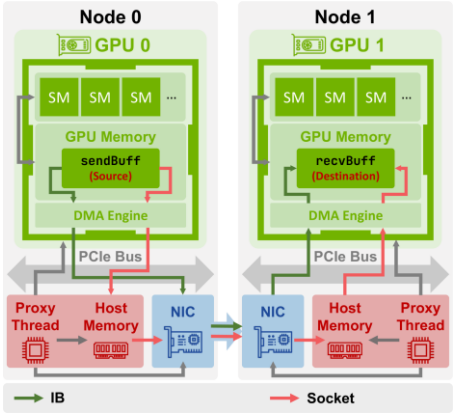
1 https://robotchinwag.com/posts/demystifying-tensor-parallelism/
2 https://docs.nvidia.com/nemo/megatron-bridge/0.2.0/parallelisms.html
3 https://aclanthology .org/2023.acl-long.134.pdf

1.4. Communication Datapath in the AI Systems

- Scale-up: NVLINK, HCCS, UnifiedBus
- Scale-out: InfiniBand (IB), RoCE



Intra-node GPU communications via NVLINK or PCIe

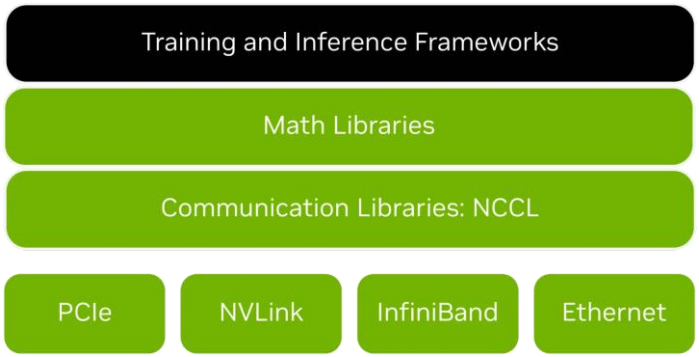


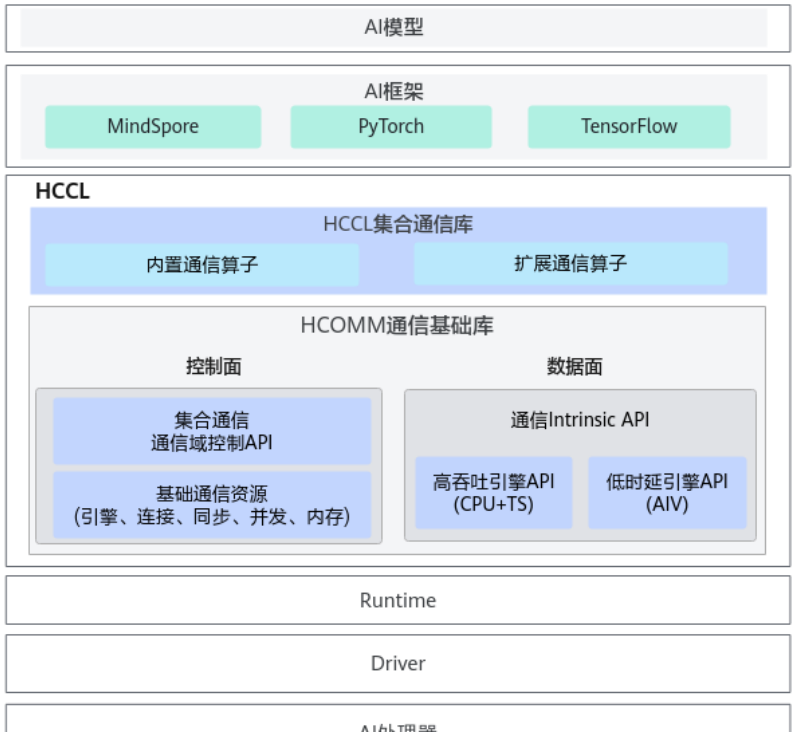
Inter-node communications via RDMA NIC (IB)

Z. Hu et al., Demystifying NCCL: An In-depth Analysis of GPU Communication Protocols and Algorithms, arXiv:2507.04786, 2025.

1.5. Collective v.s. Point-to-Point Communications

- Collective operations involve communication among a group of processes rather than just between pairs of processes. e.g., AllReduce, AllGather, Bcast
- Point-to-point: Send/Recv (Two-side), Put/Get (One-side)

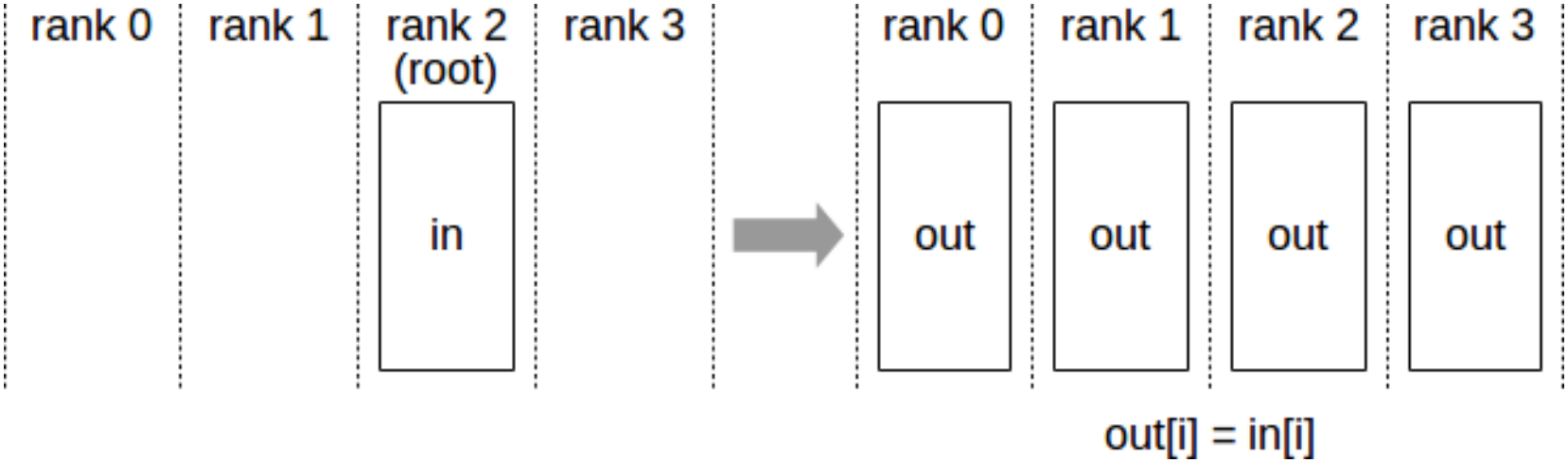




[1] https://developer.nvidia.com/blog/scaling-ai-inference-performance-and-flexibility-with-nvidia-nvlink-and-nvlink-fusion/
[2] https://www.hiascend.com/developer/techArticles/20240809-1

1.6. Broadcast

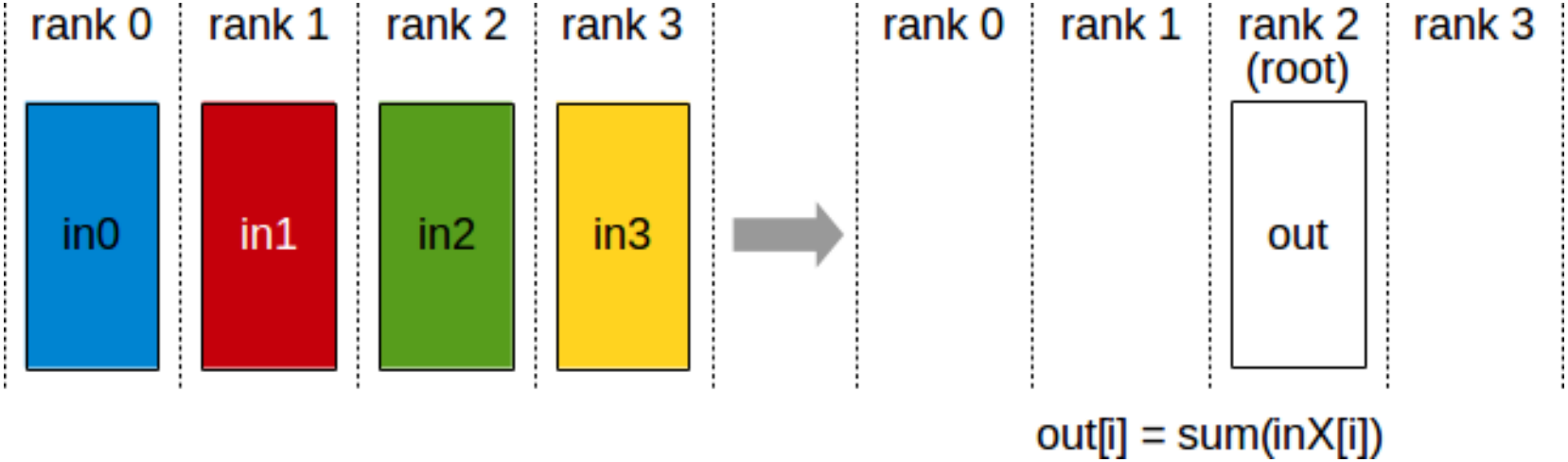
- Operations: copy an N-element buffer from the root rank (2) to all the ranks (0-3).



https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html

1.7. Reduce

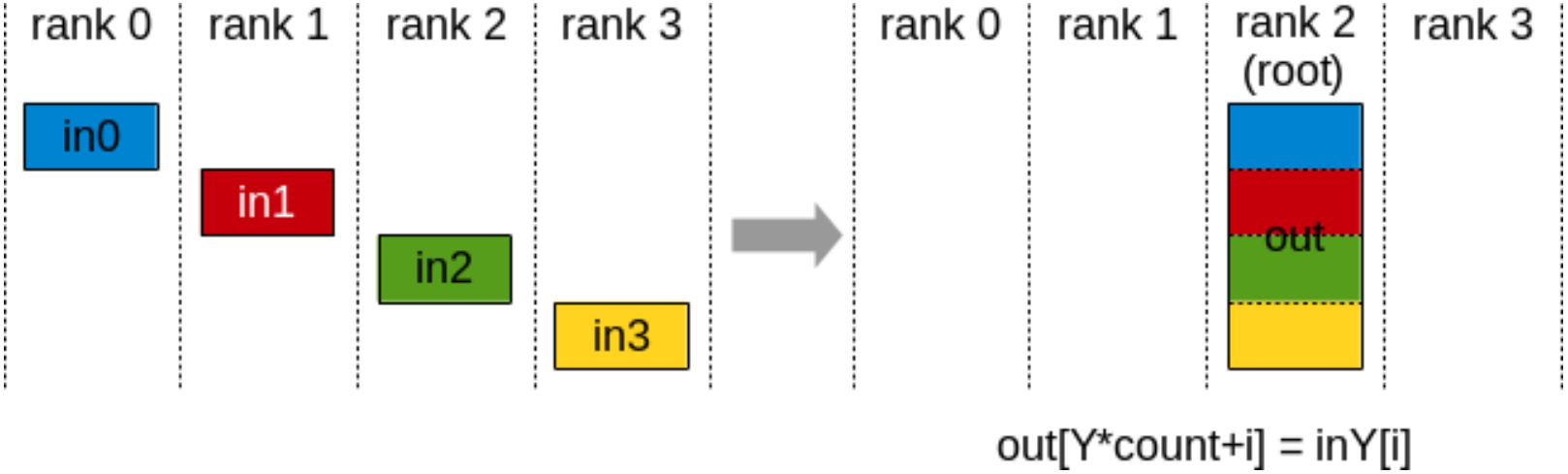
- Operations: performs reduction on data (for example, sum, min, max) across devices and stores the result in the receive buffer of a specified root rank.



https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html

1.8. Gather

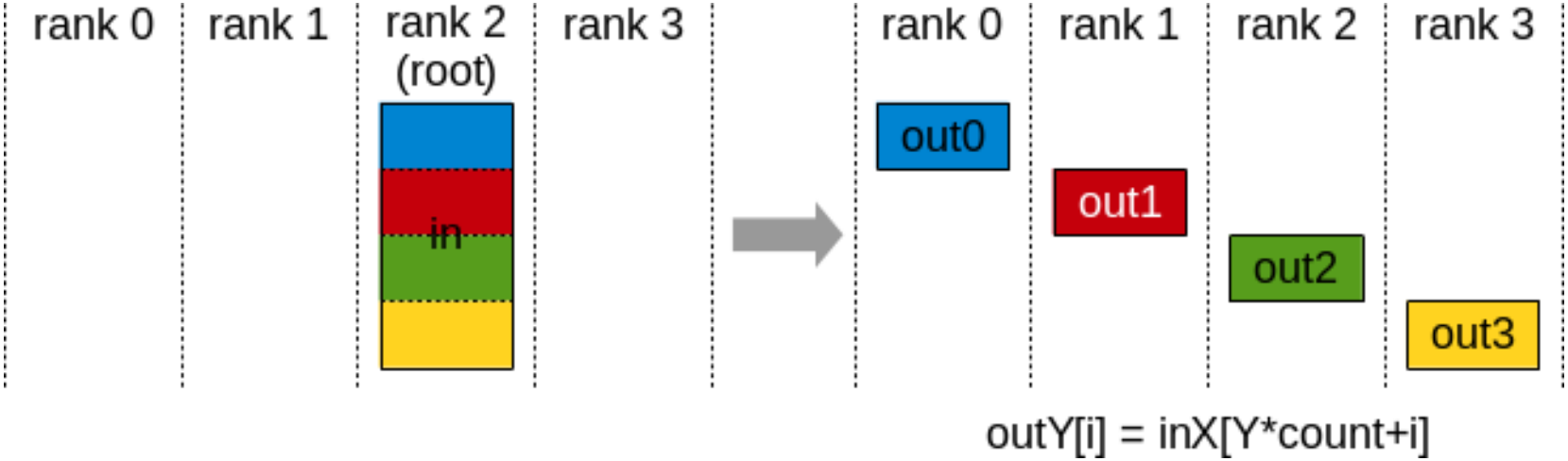
- Operations: gathers N values from k ranks into an output buffer on the root rank of size k*N.



https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html

1.9. Scatter

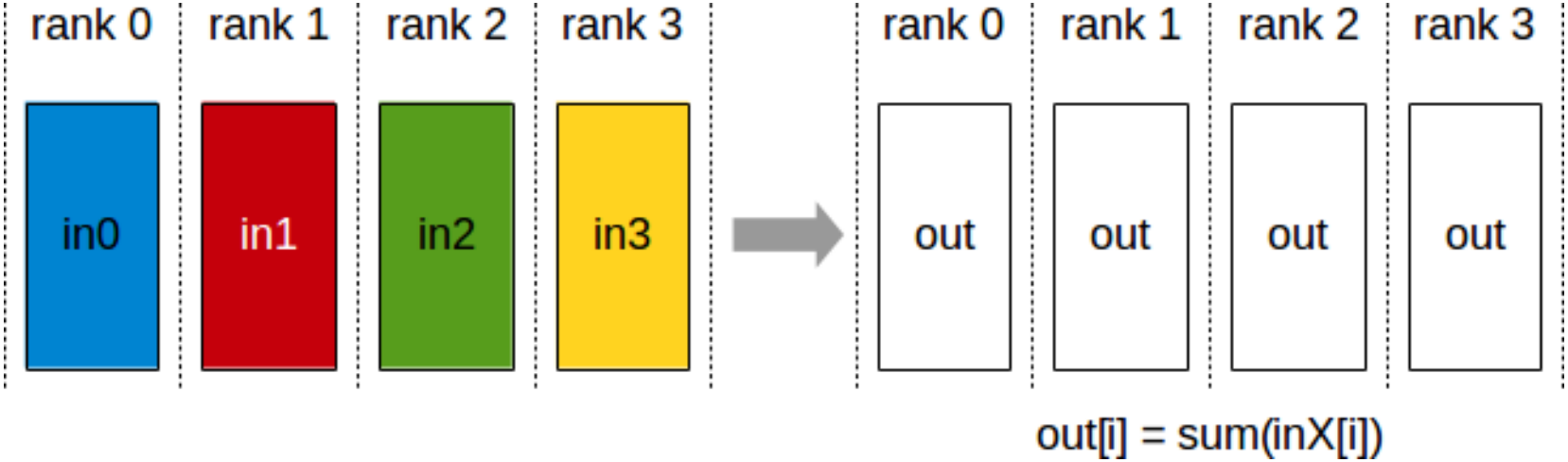
- Operations: distributes a total of N*k values from the root rank to k ranks, each rank receiving N values.



https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html

1.10. AllReduce

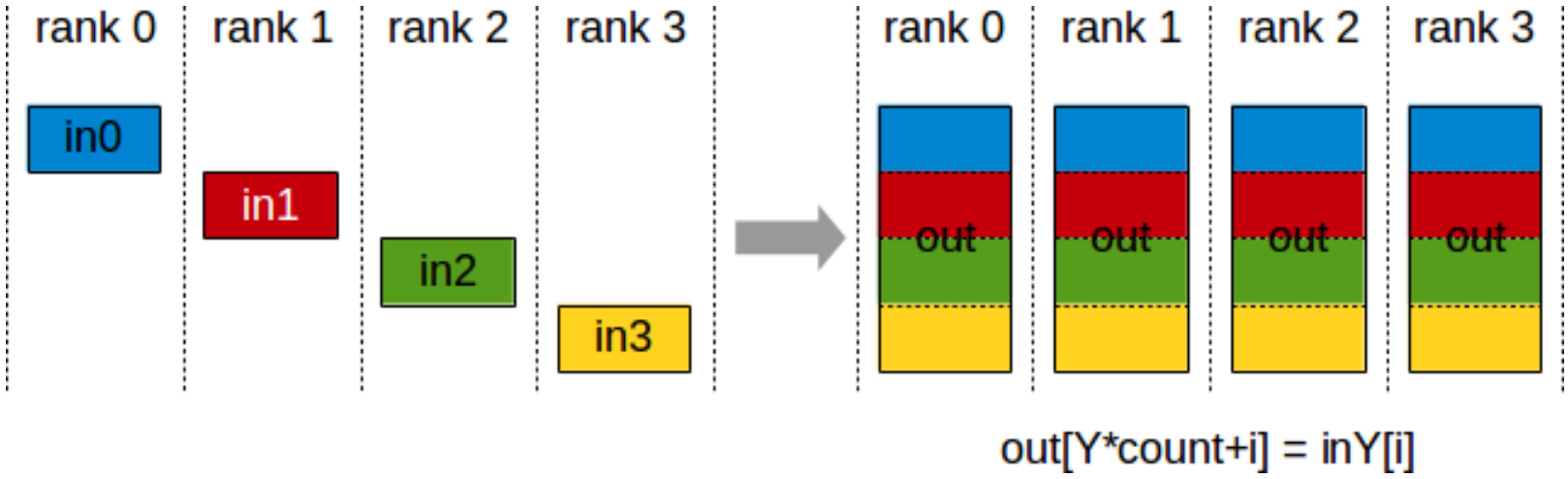
- Operations: performs reductions on data (for example, sum, min, max) across devices and stores the result in the receive buffer of every rank.



https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html

1.11. AllGather

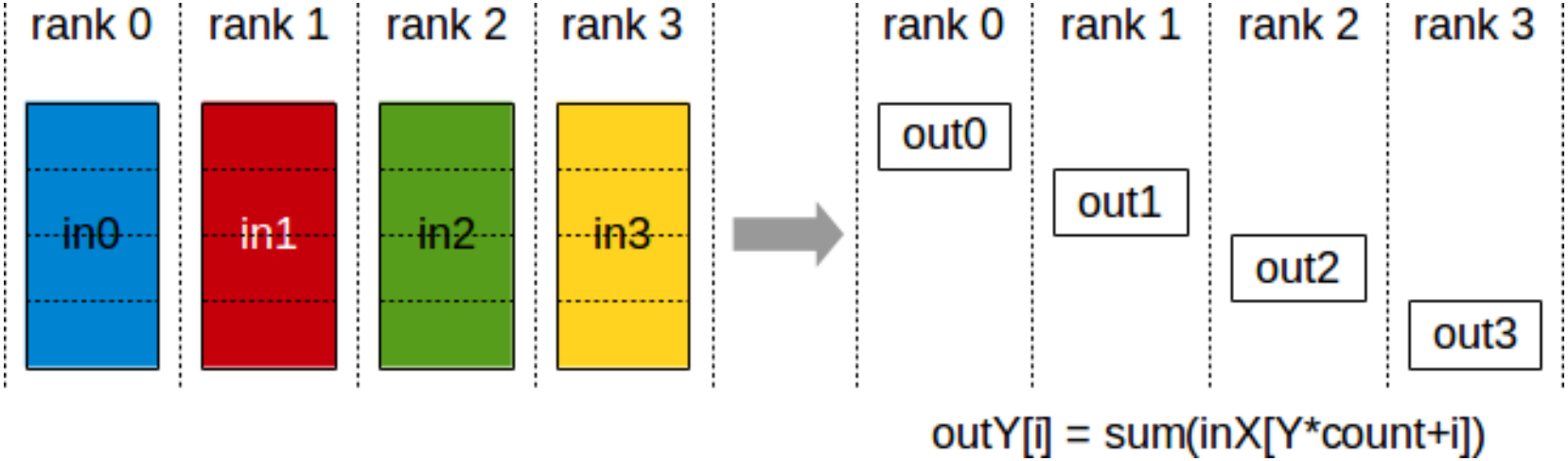
- Operations: gathers N values from k ranks into an output buffer of size k*N, and distributes that result to all ranks.
  - The output is ordered by the rank index.
  - This operation is therefore impacted by a different rank to device mapping.



https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html

1.12. ReduceScatter

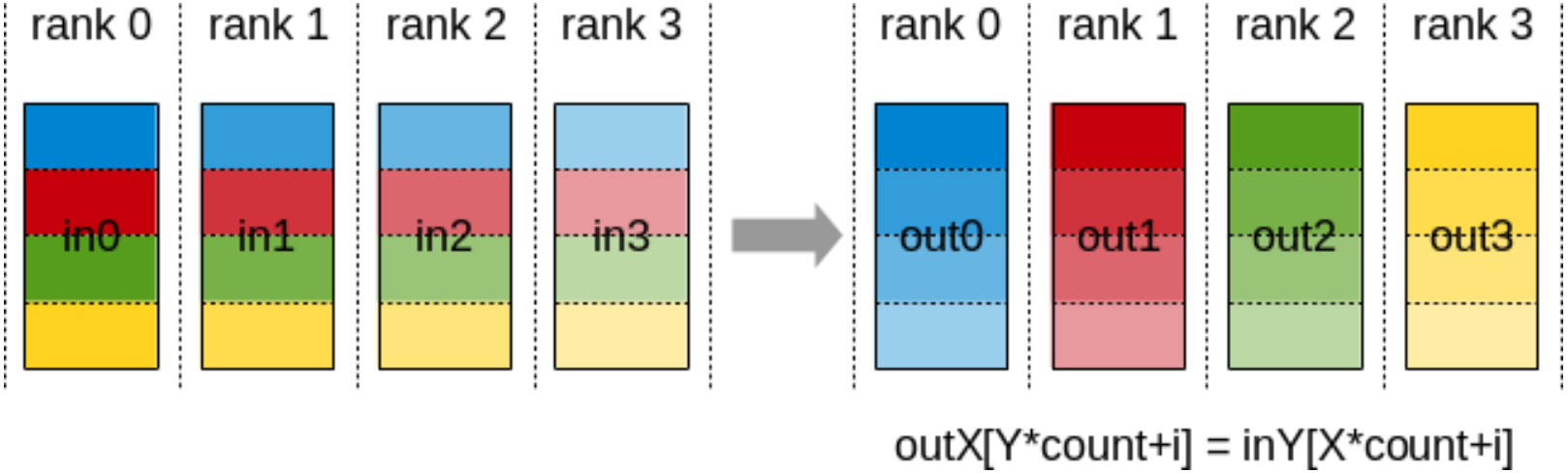
- Operations: performs the same operation as Reduce, except that the result is scattered in equal-sized blocks between ranks, each rank getting a chunk of data based on its rank index.
  - This operation is therefore impacted by a different rank to device mapping.



https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html

1.13. AlltoAll

- Operations: each rank provides an input buffer of size N values, where the j-th chunk of N/k values is sent to destination rank j. Each rank receives an output buffer of size N values, where the i-th chunk of N/k values comes from source rank i.



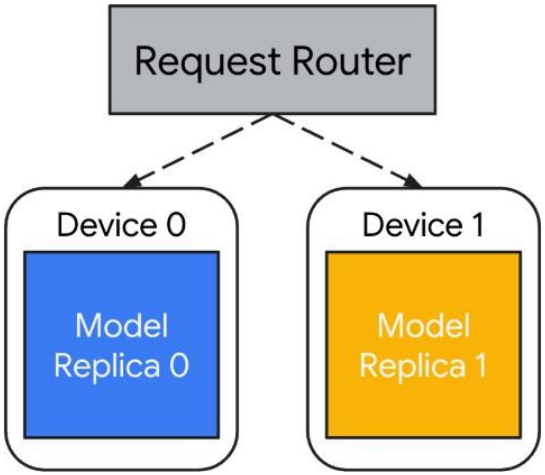
https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html

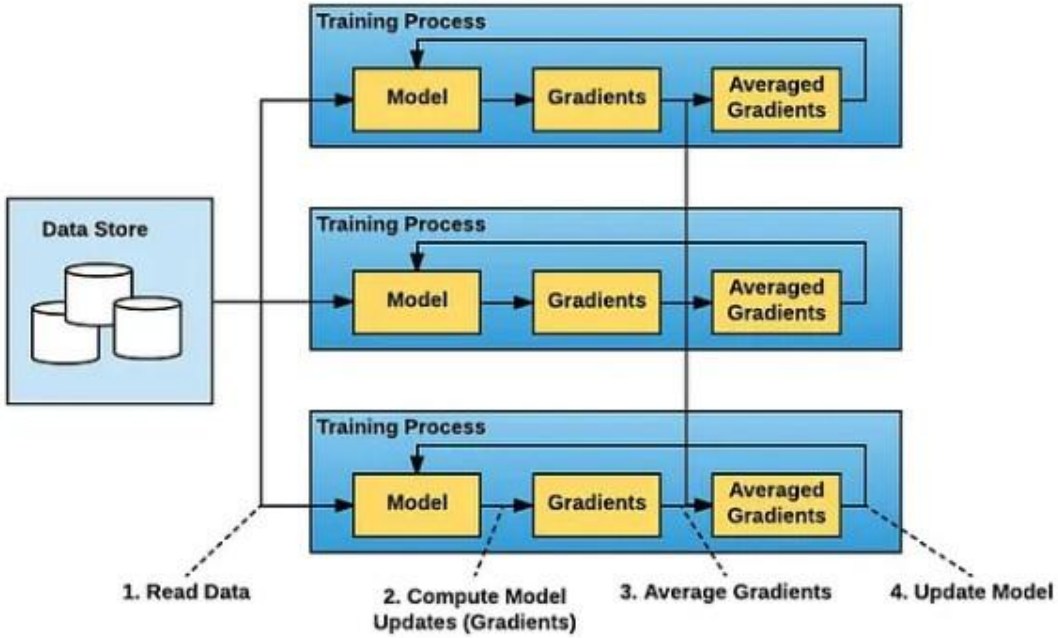
1.14a. Take FSDP (PyTorch) as an Example

- DP, DDP, FSDP: Data Parallelism, Distributed DP, Fully Sharded DP
- What is DP: multiple model copies on different devices.

DataParallel

```
class torch.nn.DataParallel(module, device_ids=None, output_device=None, dim=0)
```

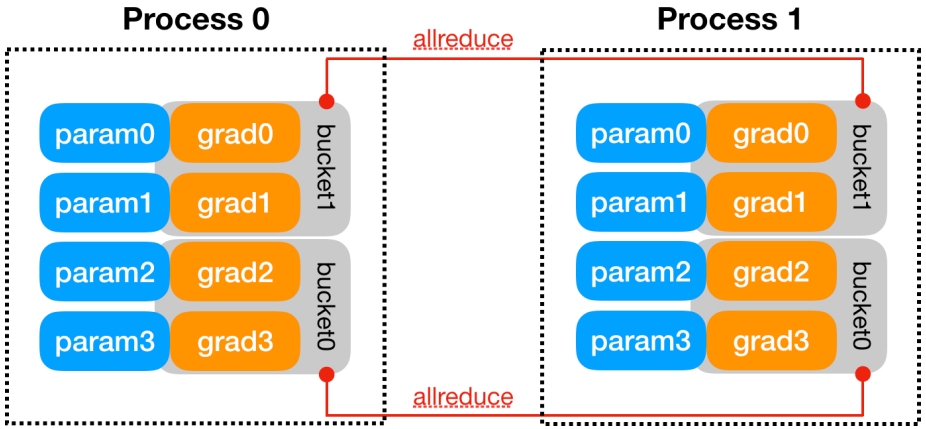


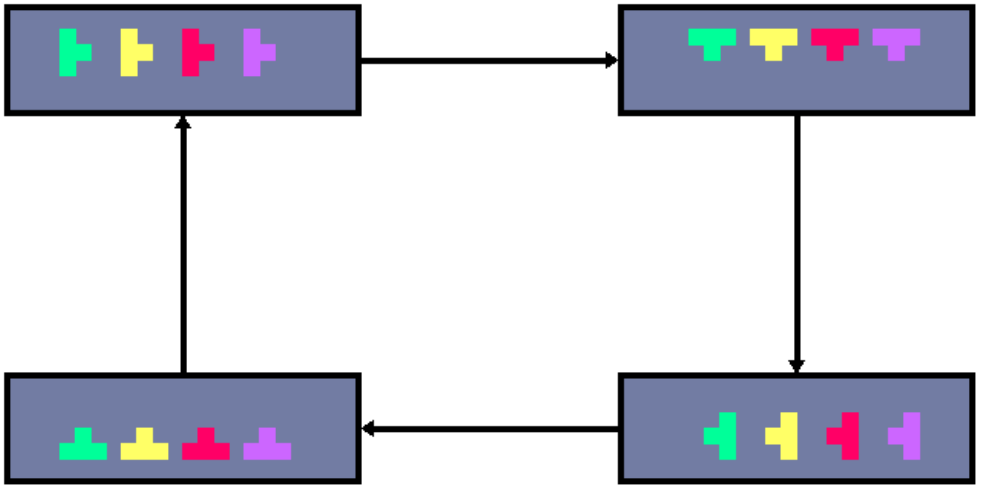


https://www.uber.com/en-JP/blog/horovod/

1.14b. Take FSDP (PyTorch) as an Example

- DP, DDP, FSDP: Data Parallelism, Distributed DP, Fully Sharded DP
- DDP is recommended: optimized all-reduce (ring-based) + distributed support
- DP: reduce op is executed on node 0 (imbalance); DDP: ring-based all-reduce.
- DDP: robust multi-process support.



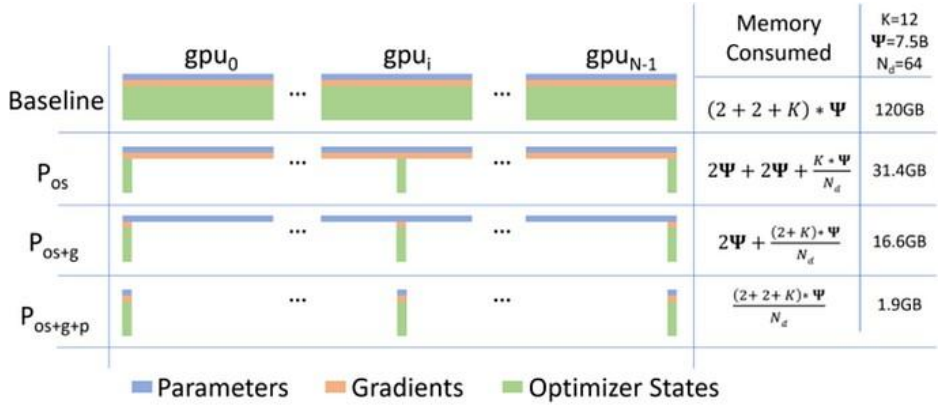


https://docs.pytorch.org/docs/stable/notes/ddp.html

1.14d. Take FSDP (PyTorch) as an Example

- DP, DDP, FSDP: Data Parallelism, Distributed DP, Fully Sharded DP
- ZeRO with different opt levels:
  - ZeRO-1: Shards the optimizer states (Optimizer States Sharding).
  - ZeRO-2: Shards both optimizer states and gradients (Optimizer States & Gradients Sharding).
  - ZeRO-3: Shards the optimizer states, gradients, and model parameters (Optimizer States & Gradients & Parameters Sharding).

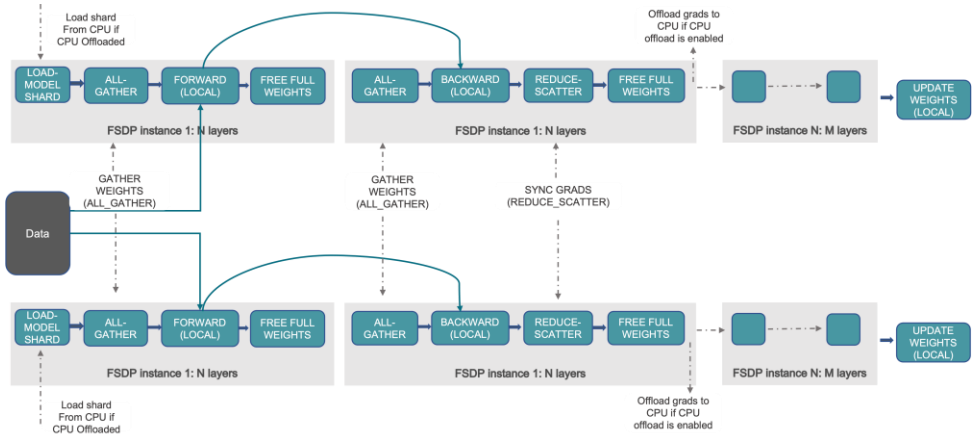
- ZeRO3 will introduce the extra communications because of the partitions of model parameters.
- K denotes the memory multiplier of the optimizer states

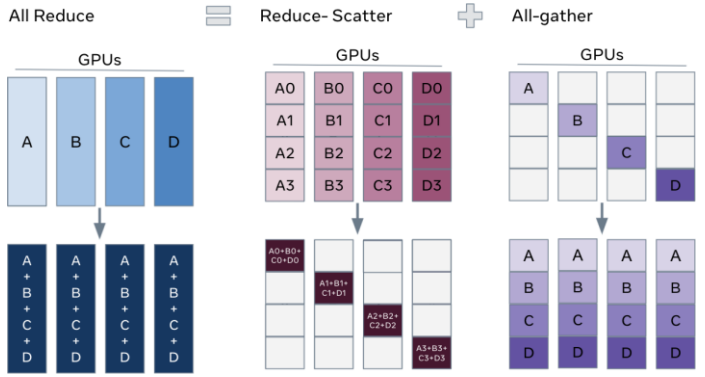


[1] https://docs.pytorch.org/docs/stable/notes/ddp.html
[2] https://www.youtube.com/watch?v=czgA-MbAdvA&t=2550s

1.14e. Take FSDP (PyTorch) as an Example

- DP, DDP, FSDP: Data Parallelism, Distributed DP, Fully Sharded DP
- FSDP: sharding model parameters/gradient/optimizer states for GPU memory reduction (ZeRO3).

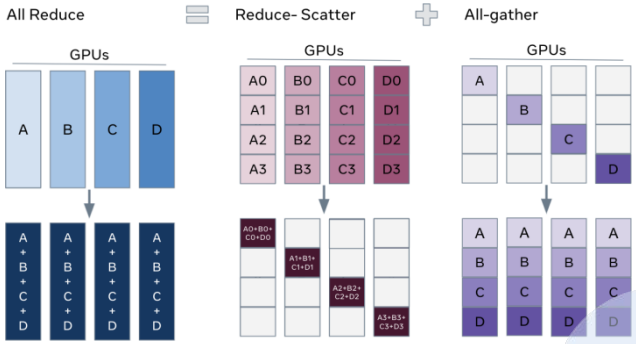




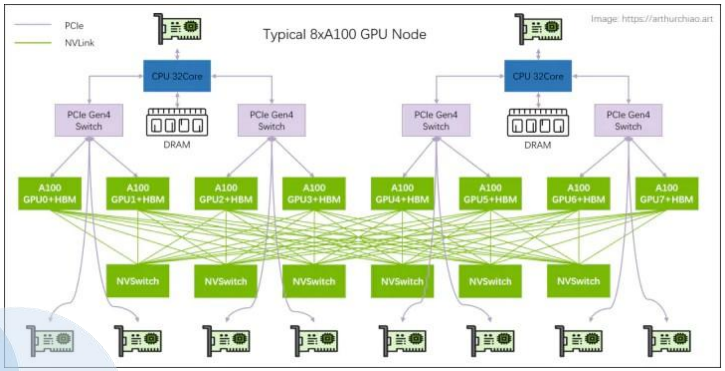
All-reduce = Reduce-scatter + all-gather

[1] https://docs.pytorch.org/tutorials/intermediate/FSDP_tutorial.html
[2] Y. Zhao et al., PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel, PVLDB, 16(12):3848–3860, 2023.

1.15. Implementations of Collective Communication Op.s

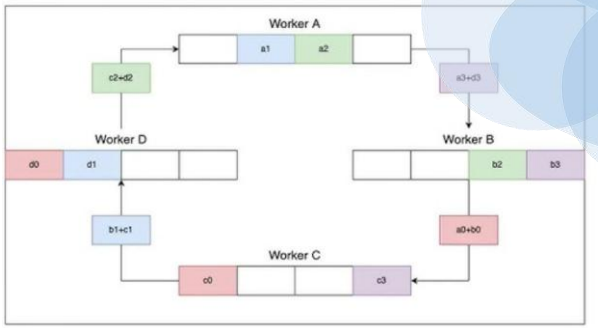


All-reduce = Reduce-scatter + all-gather

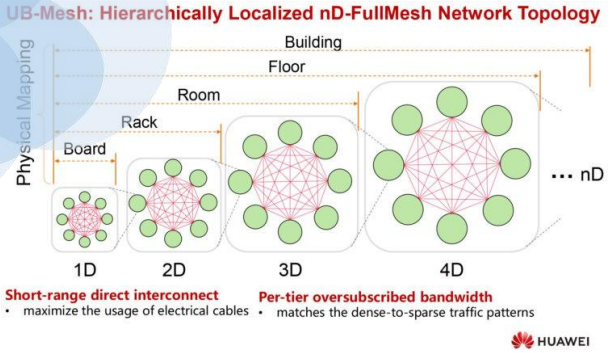


NVIDIA GPU CLOS Topology

The implementations highly depends on the hardware topology



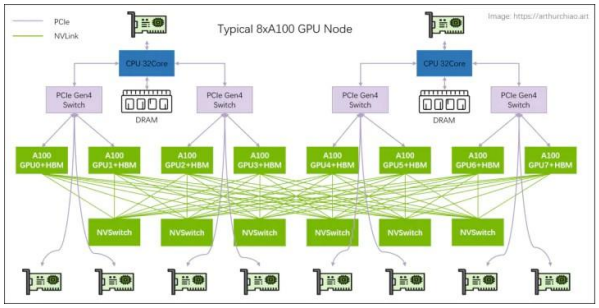
Ring All-reduce



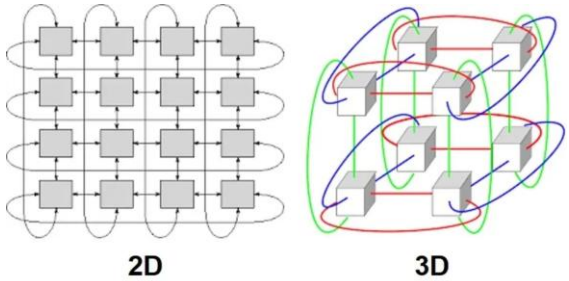
HUAWEI NPU Full-mesh Topology

[1] https://docs.pytorch.org/tutorials/intermediate/FSDP_tutorial.html
[2] Y. Zhao et al., PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel, PVLDB, 16(12):3848–3860, 2023.

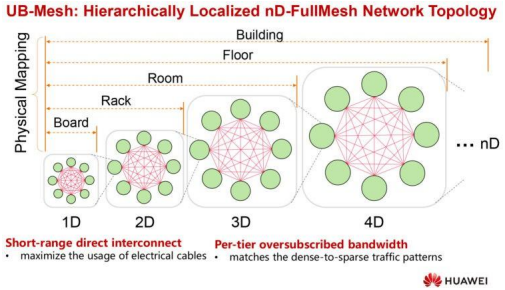
1.16. Topologies in the Modern AI Systems



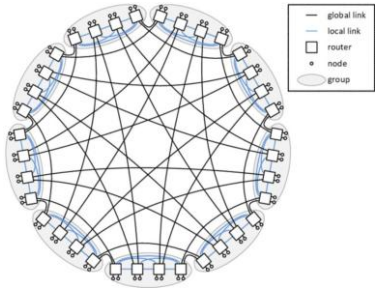
NVIDIA GPU Clos Topology



GOOGLE TPU Torus Topology



HUAWEI NPU Full-mesh Topology



Dragonfly Topology

J. Kim, W. J. Dally, S. Scott, and D. Abts, Technology-Driven, Highly-Scalable Dragonfly Topology, ISCA 2008, pp. 77–88.

2. Communication Topology Optimization

- Take AllReduce as an Example (Naive / Tree / Ring / Halving-Doubling)
- Double Binary Tree-Based AllReduce
- Protocol Design Matters
- Broadcast (a.k.a., One-to-All Broadcast)
- Broadcast: Assumptions / Lower Bounds / Tree-Based / MST / Pipelining / Simultaneous Trees
- Further Readings

2.1a. Take AllReduce as an Example

- Start from a naive implementation with root rank = 0.
  - Data transfer volume: 2 * (N-1) * K
- There are three disadvantages
  - The bandwidth and computation of Rank 0 become bottleneck.
  - There are many idle resources during the allreduce (Rank 1 – N-1).
  - Poor scalability and lacks the consideration of topology.
- How can we improve it?
  - Scatter the data transfer and computation workload to all of the ranks.
- Solutions:
  - Tree-based, ring-based, halving-doubling, etc.

```python
def naive_allreduce(local_data):
    rank = get_rank()
    num_procs = get_world_size()

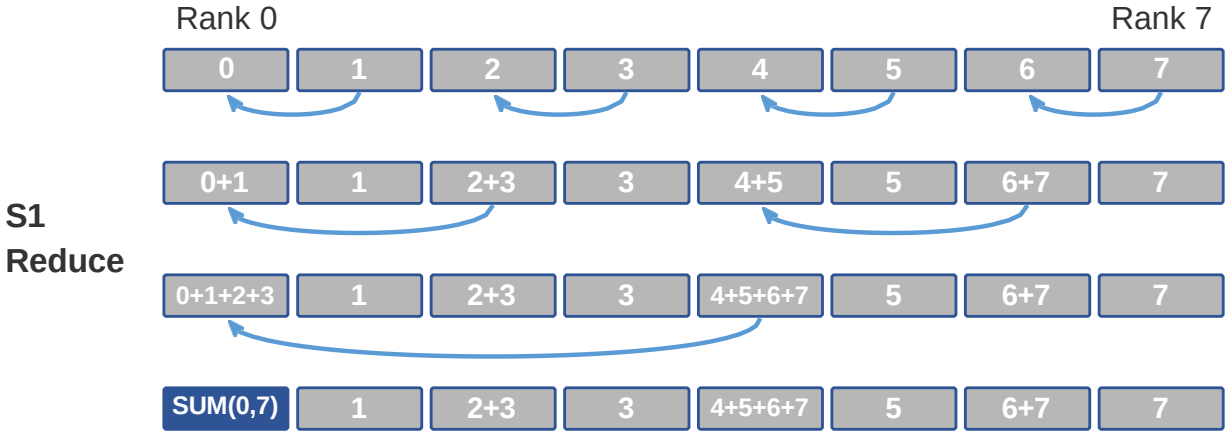
    if rank == 0:
        global_sum = local_data
        for i in range(1, num_procs):
            received_data = recv_from(i)
            global_sum += received_data
    else:
        send_to(rank=0, data=local_data)

    if rank == 0:
        for i in range(1, num_procs):
            send_to(rank=i, data=global_sum)
    else:
        global_sum = recv_from(0)

    return global_sum
```

2.1b. Take AllReduce as an Example

- Tree-based AllReduce Algorithm
  - Build a hierarchical relation with the root of tree (Rank 0).
  - Stage 1: Bottom-up Reduce
  - Stage 2: Top-down Broadcast




```python
def tree_allreduce(my_data, rank, N):
    # --- Stage 1: Reduce (to root Rank 0) ---
    step = 1
    while step < N:
        if (rank & step) != 0: # Sender
            target = rank - step
            send(my_data, to=target)
            break
        else: # Receiver
            source = rank + step
            if source < N:
                received_data = recv(from=source)
                my_data += received_data

        step *= 2

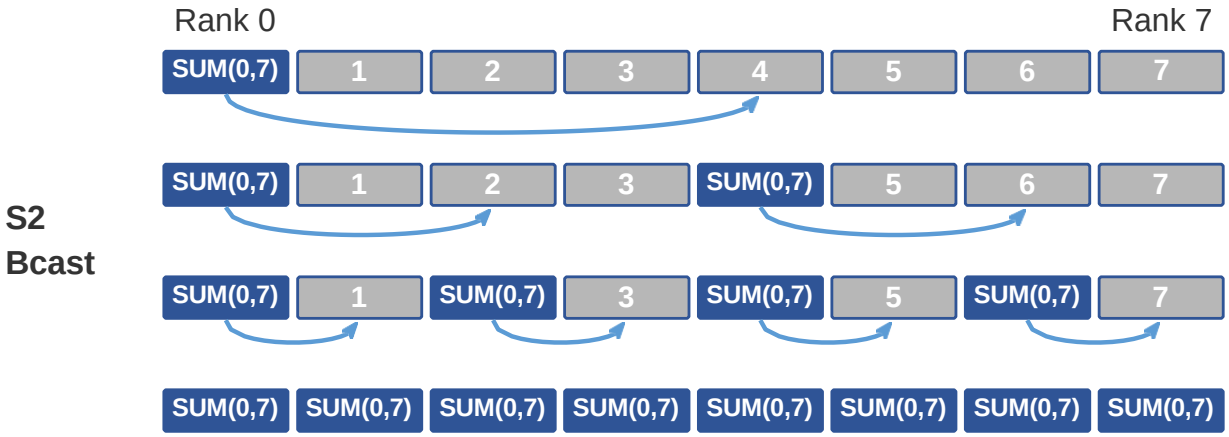
    # --- Stage 2: Broadcast (to all ranks) ---
    while step >= 1:
        if (rank & step) == 0: # Sender
            target = rank + step
            if target < N:
                send(my_data, to=target)
        else: # Receiver
            source = rank - step
            my_data = recv(from=source)

        step //= 2

    return my_data
```

2.1c. Take AllReduce as an Example

- Tree-based AllReduce Algorithm
  - Build a hierarchical relation with the root of tree (Rank 0).
  - Stage 1: Bottom-up Reduce
  - Stage 2: Top-down Broadcast




```python
def tree_allreduce(my_data, rank, N):
    # --- Stage 1: Reduce (to root Rank 0) ---
    step = 1
    while step < N:
        if (rank & step) != 0: # Sender
            target = rank - step
            send(my_data, to=target)
            break
        else: # Receiver
            source = rank + step
            if source < N:
                received_data = recv(from=source)
                my_data += received_data

        step *= 2

    # --- Stage 2: Broadcast (to all ranks) ---
    while step >= 1:
        if (rank & step) == 0: # Sender
            target = rank + step
            if target < N:
                send(my_data, to=target)
        else: # Receiver
            source = rank - step
            my_data = recv(from=source)

        step //= 2

    return my_data
```

2.1d. Take AllReduce as an Example

- Tree-based AllReduce Algorithm
  - Build a hierarchical relation with the root of tree (Rank 0).
  - Stage 1: Bottom-up Reduce
  - Stage 2: Top-down Broadcast
- Advantages
  - Parallel data transfer and reduce operations.
  - Logarithmic complexity (the naive version has linear complexity).
  - It excels with small data sizes because of the low latency.
- Disadvantages
  - It still has idle computing and bandwidth resources.
- Further Opt: Double Binary Tree (NCCL)

```python
def tree_allreduce(my_data, rank, N):
    # --- Stage 1: Reduce (to root Rank 0) ---
    step = 1
    while step < N:
        if (rank & step) != 0: # Sender
            target = rank - step
            send(my_data, to=target)
            break
        else: # Receiver
            source = rank + step
            if source < N:
                received_data = recv(from=source)
                my_data += received_data

        step *= 2

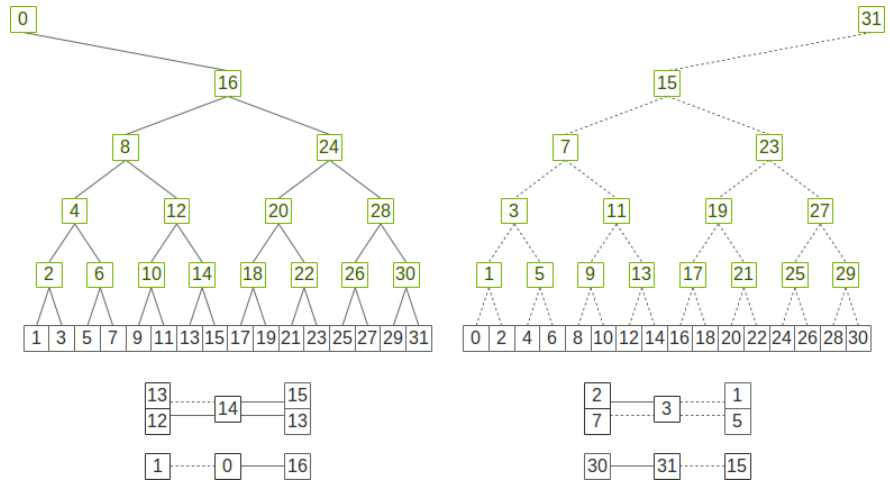
    # --- Stage 2: Broadcast (to all ranks) ---
    while step >= 1:
        if (rank & step) == 0: # Sender
            target = rank + step
            if target < N:
                send(my_data, to=target)
        else: # Receiver
            source = rank - step
            my_data = recv(from=source)

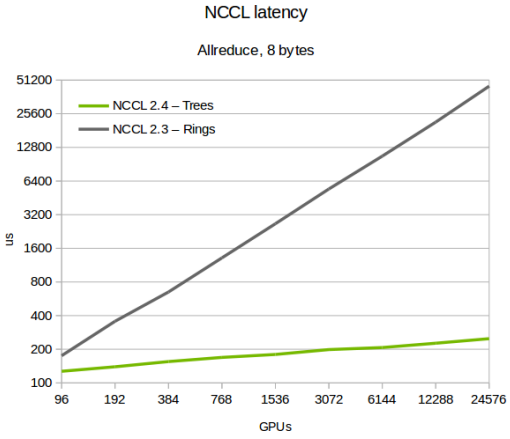
        step //= 2

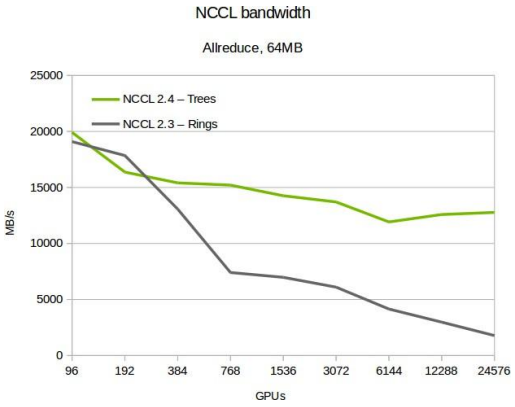
    return my_data
```

2.2. Double Binary Tree-Based AllReduce

- Observation
  - in a binary tree: internal/leaves ≈ 50-/50+
- Double binary tree
  - no rank is internal on both trees
  - all ranks have both two parents and two children except for the root
  - full bandwidth and logarithmic latency
  - good for small and medium size messages



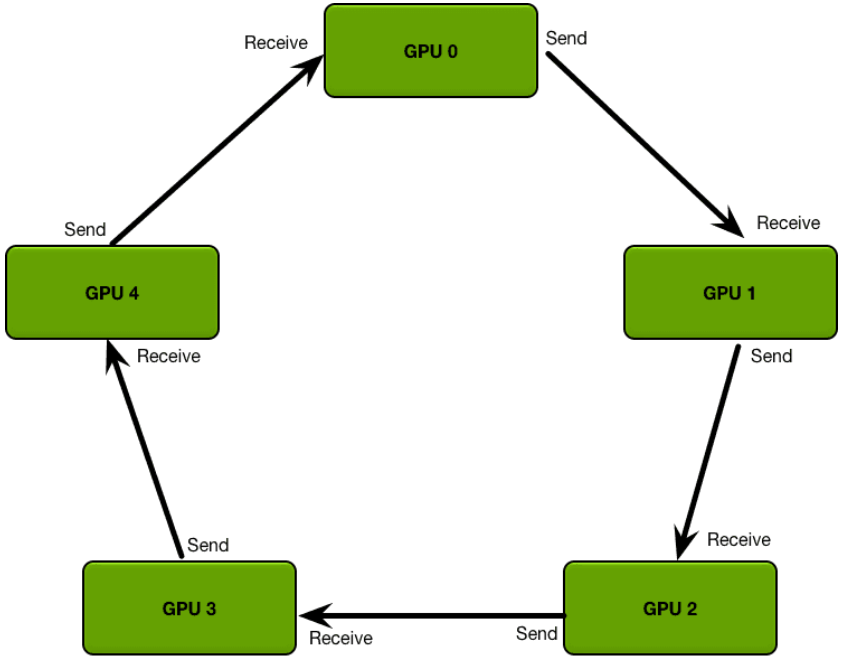




https://developer.nvidia.com/blog/massively-scale-deep-learning-training-nccl-2-4/

2.3a. Take AllReduce as an Example

- Ring-based AllReduce Algorithm
  - Stage 1: Iterative Reduce
  - Stage 2: Iterative Gather




```python
def ring_allreduce(local_data):
    rank = get_rank()
    N = get_world_size()

    chunks = split_data(local_data, N)

    send_to_rank = (rank + 1) % N
    recv_from_rank = (rank - 1 + N) % N

    # --- Stage 1: Scatter-Reduce ---
    for i in range(N - 1):
        send_idx = (rank - i + N) % N
        recv_idx = (rank - i - 1 + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        received_chunk = recv_data(src=recv_from_rank)

        chunks[recv_idx] += received_chunk

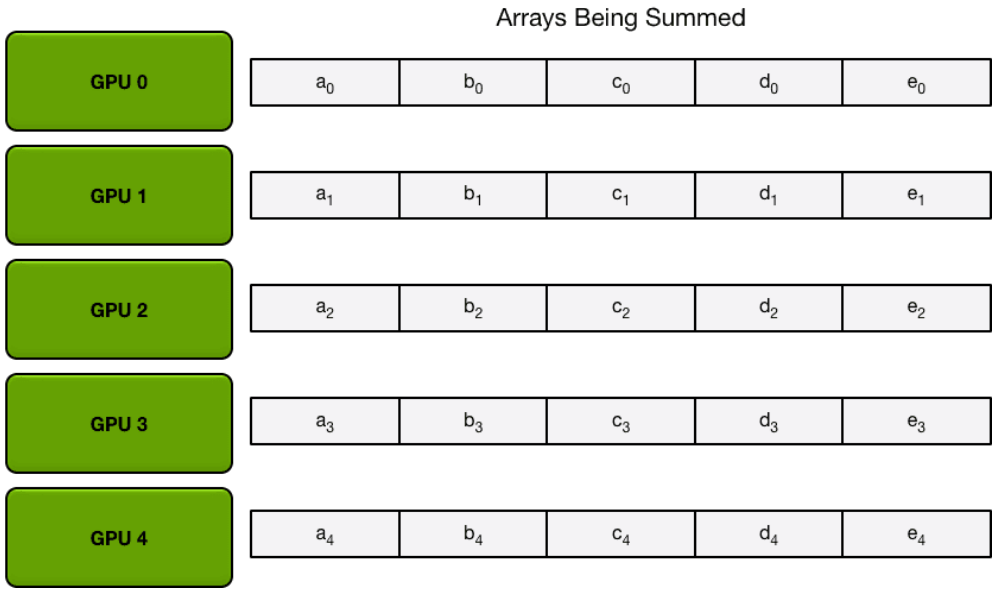
    # --- Stage 2: All-Gather ---
    for i in range(N - 1):
        send_idx = (rank - i + 1 + N) % N
        recv_idx = (rank - i + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        chunks[recv_idx] = recv_data(src=recv_from_rank)

    return flatten(chunks)
```

2.3b. Take AllReduce as an Example

- Ring-based AllReduce Algorithm
  - Stage 1: Iterative Reduce with N-1 steps
  - Stage 2: Iterative Gather with N-1 steps
- Split the data with N partitions




```python
def ring_allreduce(local_data):
    rank = get_rank()
    N = get_world_size()

    chunks = split_data(local_data, N)

    send_to_rank = (rank + 1) % N
    recv_from_rank = (rank - 1 + N) % N

    # --- Stage 1: Scatter-Reduce ---
    for i in range(N - 1):
        send_idx = (rank - i + N) % N
        recv_idx = (rank - i - 1 + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        received_chunk = recv_data(src=recv_from_rank)

        chunks[recv_idx] += received_chunk

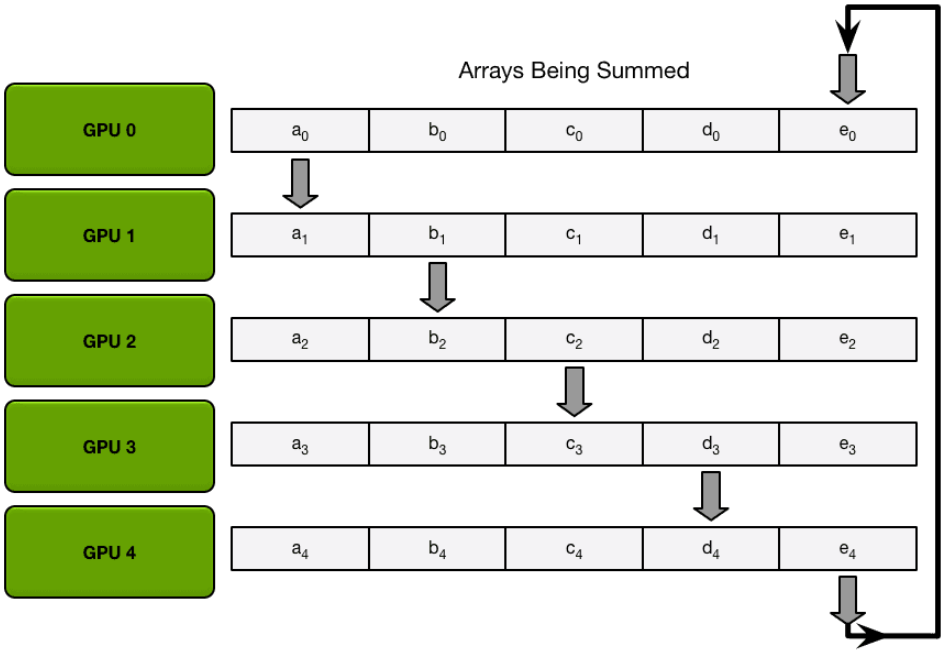
    # --- Stage 2: All-Gather ---
    for i in range(N - 1):
        send_idx = (rank - i + 1 + N) % N
        recv_idx = (rank - i + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        chunks[recv_idx] = recv_data(src=recv_from_rank)

    return flatten(chunks)
```

2.3c. Take AllReduce as an Example

- Ring-based AllReduce Algorithm
  - Stage 1: Iterative Reduce with N-1 steps
  - Stage 2: Iterative Gather with N-1 steps




```python
def ring_allreduce(local_data):
    rank = get_rank()
    N = get_world_size()

    chunks = split_data(local_data, N)

    send_to_rank = (rank + 1) % N
    recv_from_rank = (rank - 1 + N) % N

    # --- Stage 1: Scatter-Reduce ---
    for i in range(N - 1):
        send_idx = (rank - i + N) % N
        recv_idx = (rank - i - 1 + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        received_chunk = recv_data(src=recv_from_rank)

        chunks[recv_idx] += received_chunk

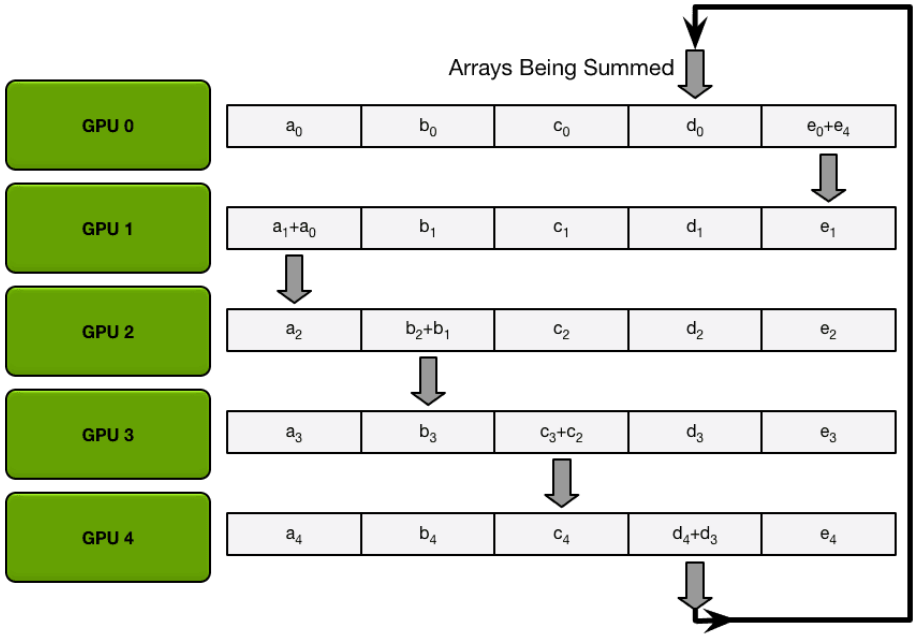
    # --- Stage 2: All-Gather ---
    for i in range(N - 1):
        send_idx = (rank - i + 1 + N) % N
        recv_idx = (rank - i + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        chunks[recv_idx] = recv_data(src=recv_from_rank)

    return flatten(chunks)
```

2.3d. Take AllReduce as an Example

- Ring-based AllReduce Algorithm
  - Stage 1: Iterative Reduce with N-1 steps
  - Stage 2: Iterative Gather with N-1 steps




```python
def ring_allreduce(local_data):
    rank = get_rank()
    N = get_world_size()

    chunks = split_data(local_data, N)

    send_to_rank = (rank + 1) % N
    recv_from_rank = (rank - 1 + N) % N

    # --- Stage 1: Scatter-Reduce ---
    for i in range(N - 1):
        send_idx = (rank - i + N) % N
        recv_idx = (rank - i - 1 + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        received_chunk = recv_data(src=recv_from_rank)

        chunks[recv_idx] += received_chunk

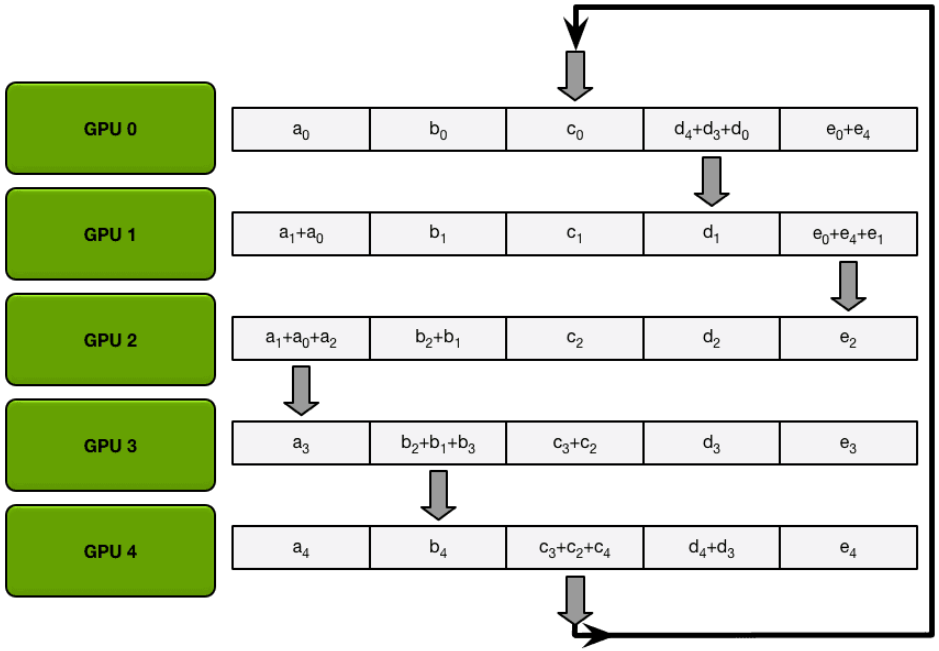
    # --- Stage 2: All-Gather ---
    for i in range(N - 1):
        send_idx = (rank - i + 1 + N) % N
        recv_idx = (rank - i + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        chunks[recv_idx] = recv_data(src=recv_from_rank)

    return flatten(chunks)
```

2.3e. Take AllReduce as an Example

- Ring-based AllReduce Algorithm
  - Stage 1: Iterative Reduce with N-1 steps
  - Stage 2: Iterative Gather with N-1 steps




```python
def ring_allreduce(local_data):
    rank = get_rank()
    N = get_world_size()

    chunks = split_data(local_data, N)

    send_to_rank = (rank + 1) % N
    recv_from_rank = (rank - 1 + N) % N

    # --- Stage 1: Scatter-Reduce ---
    for i in range(N - 1):
        send_idx = (rank - i + N) % N
        recv_idx = (rank - i - 1 + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        received_chunk = recv_data(src=recv_from_rank)

        chunks[recv_idx] += received_chunk

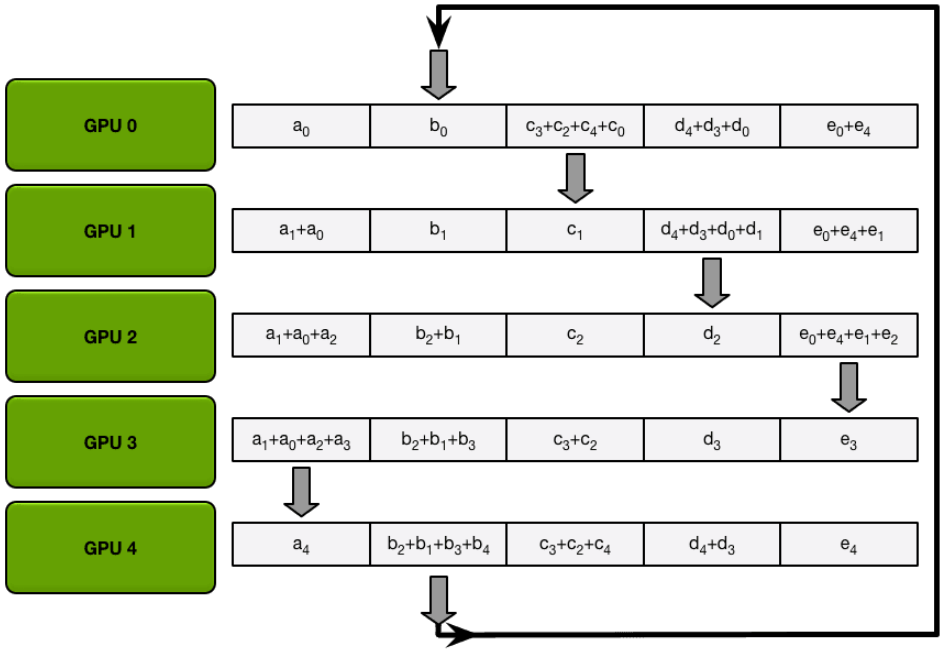
    # --- Stage 2: All-Gather ---
    for i in range(N - 1):
        send_idx = (rank - i + 1 + N) % N
        recv_idx = (rank - i + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        chunks[recv_idx] = recv_data(src=recv_from_rank)

    return flatten(chunks)
```

2.3f. Take AllReduce as an Example

- Ring-based AllReduce Algorithm
  - Stage 1: Iterative Reduce with N-1 steps
  - Stage 2: Iterative Gather with N-1 steps




```python
def ring_allreduce(local_data):
    rank = get_rank()
    N = get_world_size()

    chunks = split_data(local_data, N)

    send_to_rank = (rank + 1) % N
    recv_from_rank = (rank - 1 + N) % N

    # --- Stage 1: Scatter-Reduce ---
    for i in range(N - 1):
        send_idx = (rank - i + N) % N
        recv_idx = (rank - i - 1 + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        received_chunk = recv_data(src=recv_from_rank)

        chunks[recv_idx] += received_chunk

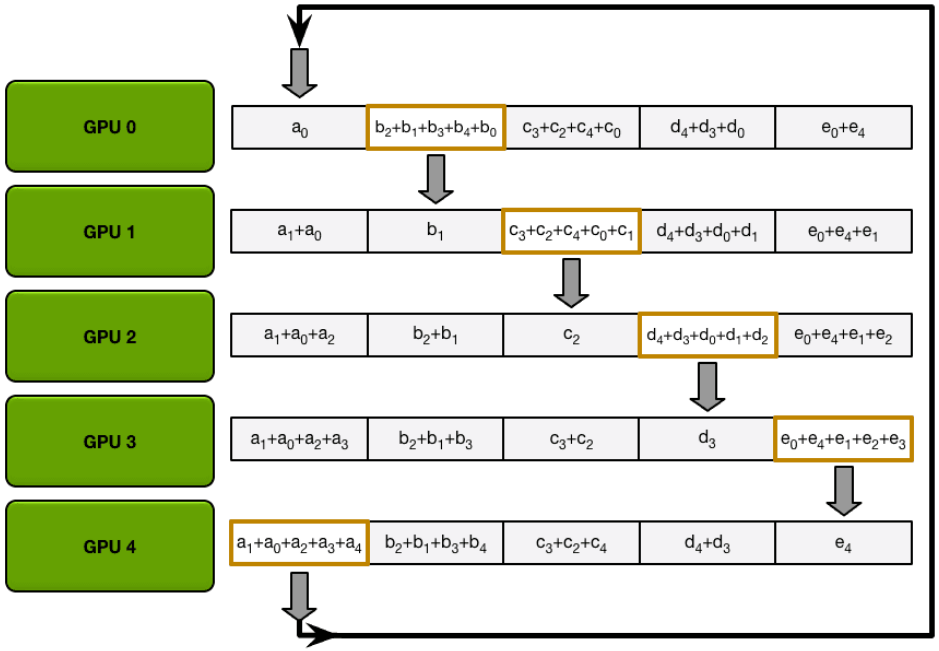
    # --- Stage 2: All-Gather ---
    for i in range(N - 1):
        send_idx = (rank - i + 1 + N) % N
        recv_idx = (rank - i + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        chunks[recv_idx] = recv_data(src=recv_from_rank)

    return flatten(chunks)
```

2.3g. Take AllReduce as an Example

- Ring-based AllReduce Algorithm
  - Stage 1: Iterative Reduce with N-1 steps
  - Stage 2: Iterative Gather with N-1 steps




```python
def ring_allreduce(local_data):
    rank = get_rank()
    N = get_world_size()

    chunks = split_data(local_data, N)

    send_to_rank = (rank + 1) % N
    recv_from_rank = (rank - 1 + N) % N

    # --- Stage 1: Scatter-Reduce ---
    for i in range(N - 1):
        send_idx = (rank - i + N) % N
        recv_idx = (rank - i - 1 + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        received_chunk = recv_data(src=recv_from_rank)

        chunks[recv_idx] += received_chunk

    # --- Stage 2: All-Gather ---
    for i in range(N - 1):
        send_idx = (rank - i + 1 + N) % N
        recv_idx = (rank - i + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        chunks[recv_idx] = recv_data(src=recv_from_rank)

    return flatten(chunks)
```

2.3h. Take AllReduce as an Example

- Ring-based AllReduce Algorithm
  - Stage 1: Iterative Reduce with N-1 steps
  - Stage 2: Iterative Gather with N-1 steps
- Quantitative Analysis
  - Data Transfer Volume: 2 * (N-1) * K / N
  - Overall Time: 2 * (N-1) * alpha + 2 * (N-1) * K / N / beta
  - Tree-based AllReduce Time : 2 * log(N) * (alpha + K / beta)
  - alpha: latency; beta: bandwidth
- Advantages: Optimal bandwidth utilization

```python
def ring_allreduce(local_data):
    rank = get_rank()
    N = get_world_size()

    chunks = split_data(local_data, N)

    send_to_rank = (rank + 1) % N
    recv_from_rank = (rank - 1 + N) % N

    # --- Stage 1: Scatter-Reduce ---
    for i in range(N - 1):
        send_idx = (rank - i + N) % N
        recv_idx = (rank - i - 1 + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        received_chunk = recv_data(src=recv_from_rank)

        chunks[recv_idx] += received_chunk

    # --- Stage 2: All-Gather ---
    for i in range(N - 1):
        send_idx = (rank - i + 1 + N) % N
        recv_idx = (rank - i + N) % N

        send_data(chunks[send_idx], dest=send_to_rank)
        chunks[recv_idx] = recv_data(src=recv_from_rank)

    return flatten(chunks)
```

2.3i. Take AllReduce as an Example

- Halving-doubling AllReduce Algorithm
  - Stage 1: Recursive halving reduce
  - Stage 2: Recursive doubling gather

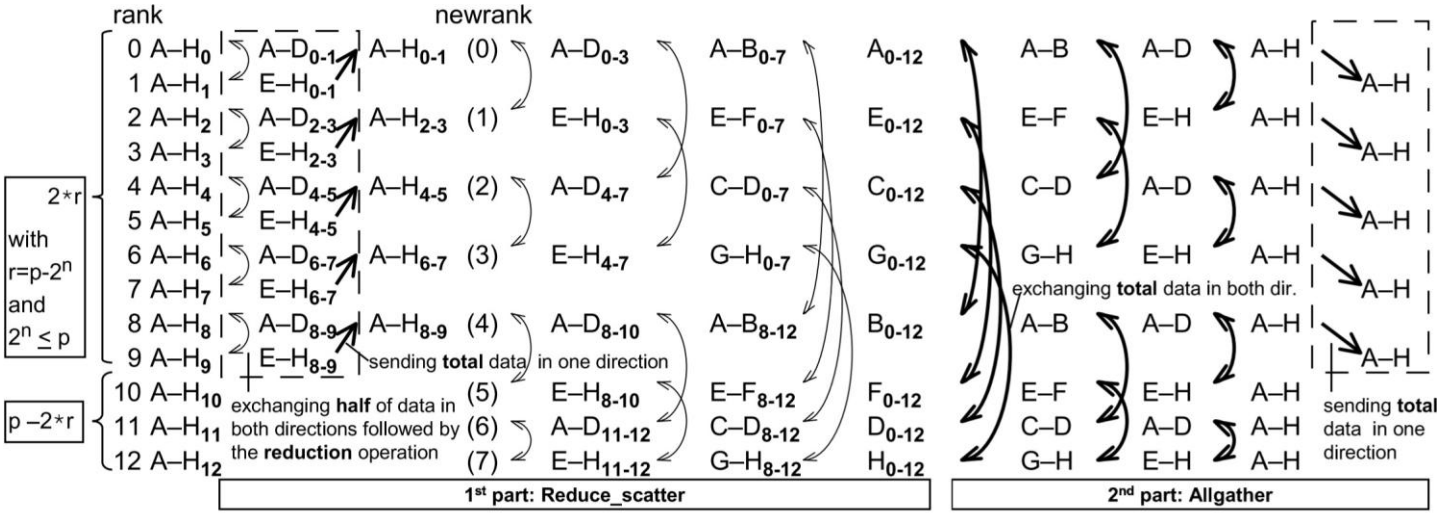
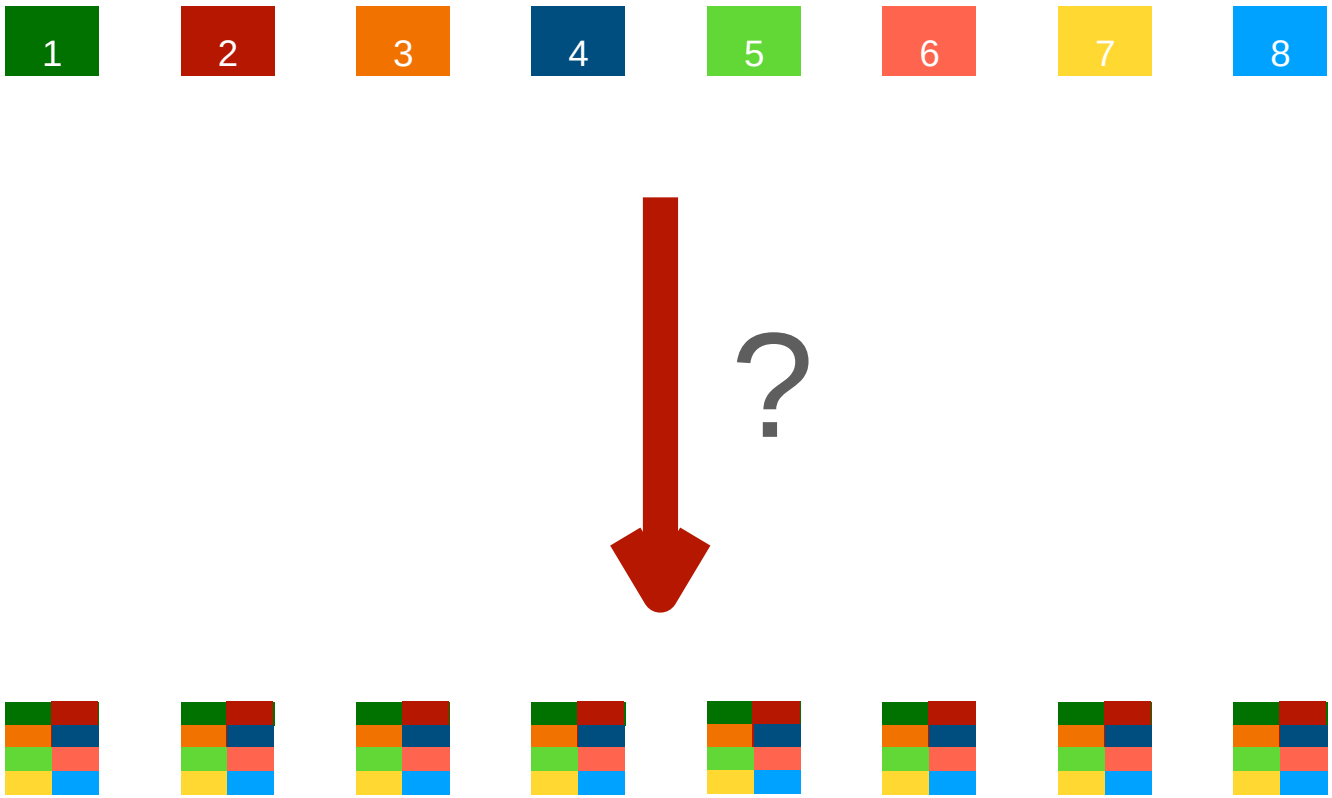


Figure 12: Allreduce using the recursive halving and doubling algorithm. The intermediate results after each communication step, including the reduction operation in the reduce-scatter phase, are shown. The dotted frames show the additional overhead caused by a non-power-of-two number of processes.

https://www.semanticscholar.org/paper/Optimization-of-Collective-Reduction-Operations-Rabenseifner/e2584fdc3f9e696423d96522038f3fa35137ab92

Communication Primitives

Recursive Halving-Doubling AllReduce

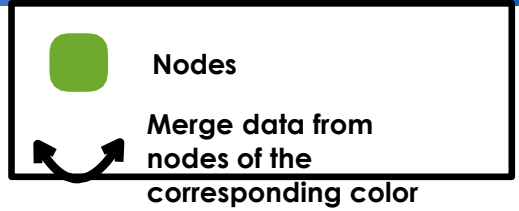


[1] Thakur, Rajeev, Rolf Rabenseifner, and William Gropp. "Optimization of Collective Communication Operations in MPICH."

Communication Primitives

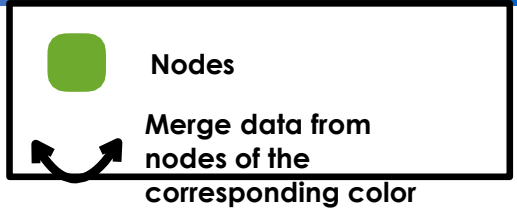
Recursive Halving-Doubling AllReduce





[1] Thakur, Rajeev, Rolf Rabenseifner, and William Gropp. "Optimization of Collective Communication Operations in MPICH."

Communication Primitives



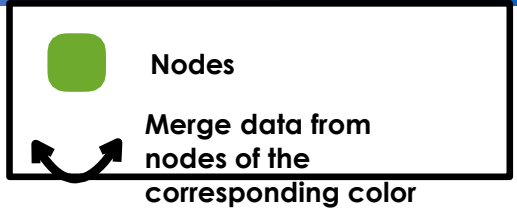
Recursive Halving-Doubling AllReduce

Step 1: Each node exchanges with a neighbor offset by 1



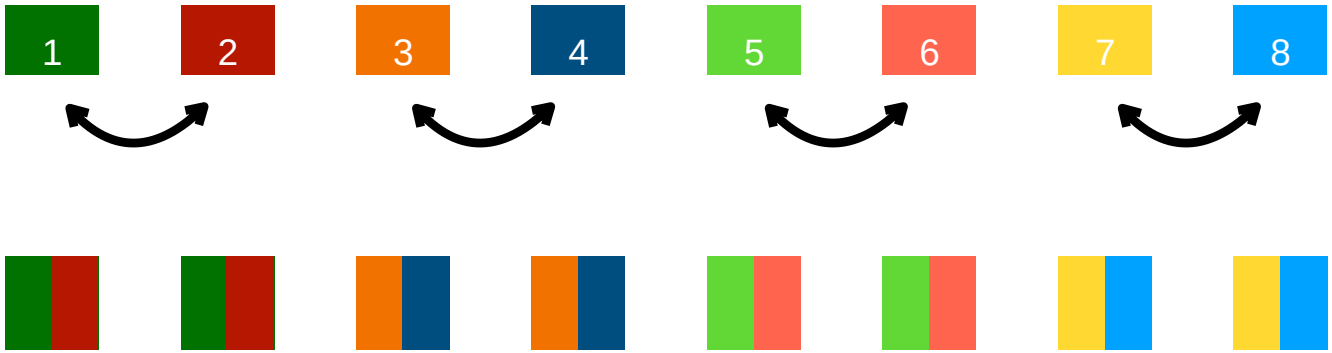
[1] Thakur, Rajeev, Rolf Rabenseifner, and William Gropp. "Optimization of Collective Communication Operations in MPICH."

Communication Primitives



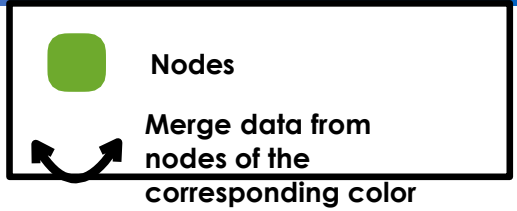
Recursive Halving-Doubling AllReduce

Step 1: Each node exchanges with a neighbor offset by 1



[1] Thakur, Rajeev, Rolf Rabenseifner, and William Gropp. "Optimization of Collective Communication Operations in MPICH."

Communication Primitives

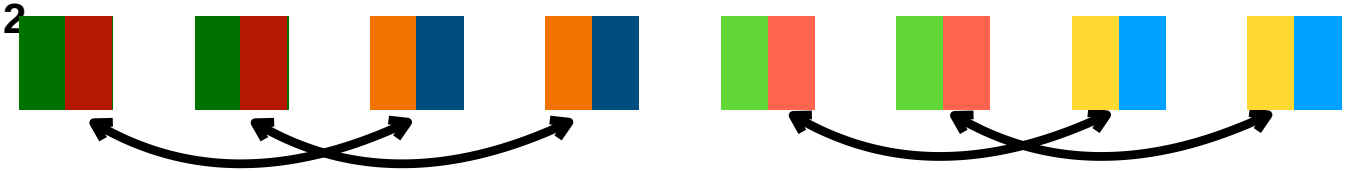


Recursive Halving-Doubling AllReduce

Step 1: Each node exchanges with a neighbor offset by 1

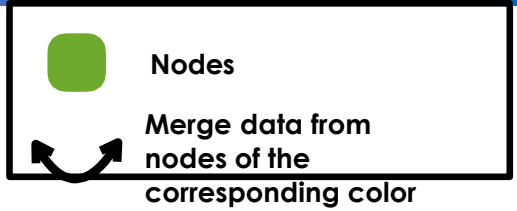


Step 2: Each node exchanges with a neighbor offset by 2



[1] Thakur, Rajeev, Rolf Rabenseifner, and William Gropp. "Optimization of Collective Communication Operations in MPICH."

Communication Primitives

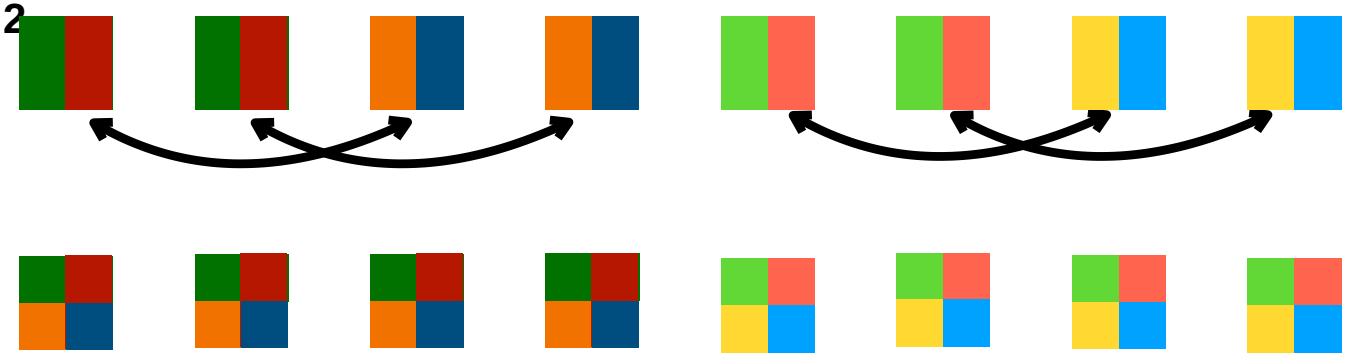


Recursive Halving-Doubling AllReduce

Step 1: Each node exchanges with a neighbor offset by 1

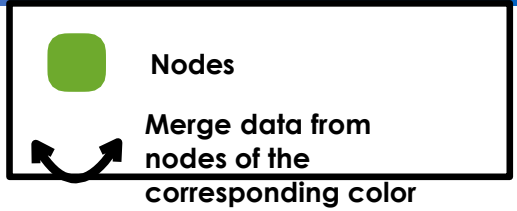


Step 2: Each node exchanges with a neighbor offset by 2



[1] Thakur, Rajeev, Rolf Rabenseifner, and William Gropp. "Optimization of Collective Communication Operations in MPICH."

Communication Primitives

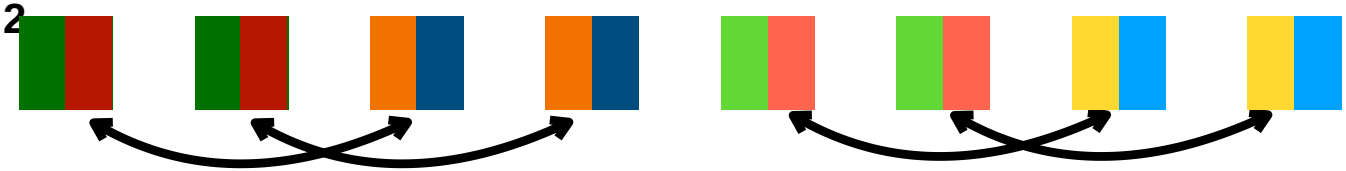


Recursive Halving-Doubling AllReduce

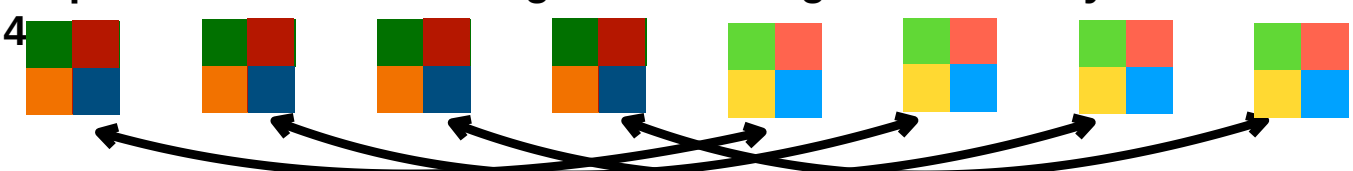
Step 1: Each node exchanges with a neighbor offset by 1



Step 2: Each node exchanges with a neighbor offset by 2

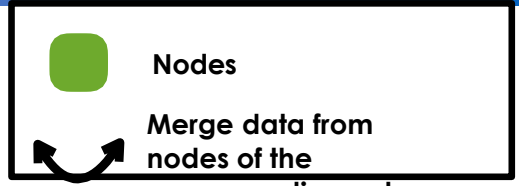


Step 3: Each node exchanges with a neighbor offset by 4



[1] Thakur, Rajeev, Rolf Rabenseifner, and William Gropp. "Optimization of Collective Communication Operations in MPICH."

Communication Primitives

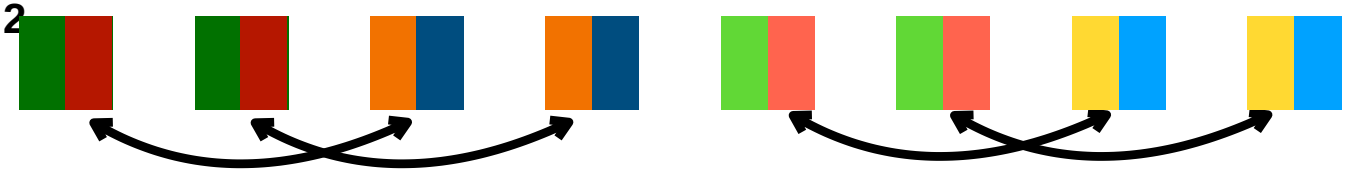


Recursive Halving-Doubling AllReduce

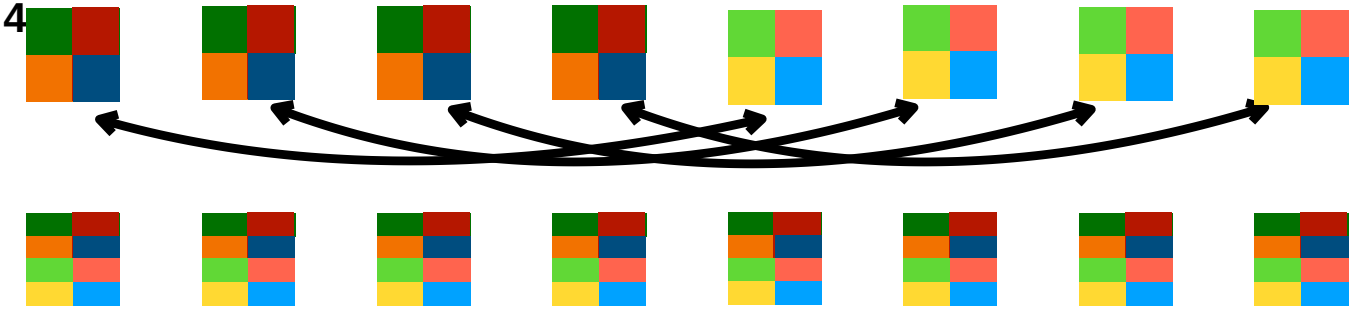
Step 1: Each node exchanges with a neighbor offset by 1



Step 2: Each node exchanges with a neighbor offset by 2

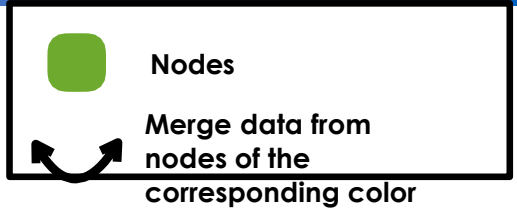


Step 3: Each node exchanges with a neighbor offset by 4



[1] Thakur, Rajeev, Rolf Rabenseifner, and William Gropp. "Optimization of Collective Communication Operations in MPICH."

Communication Primitives

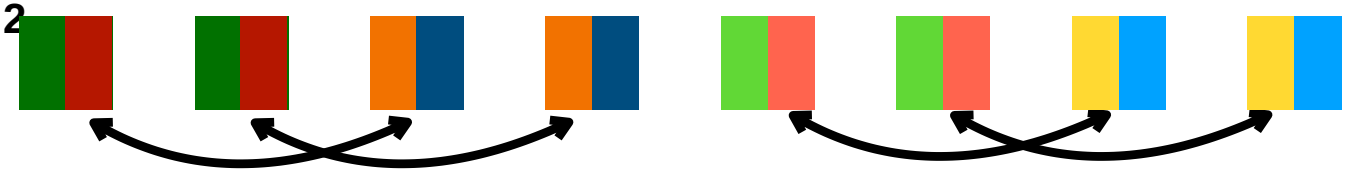


Recursive Halving-Doubling AllReduce

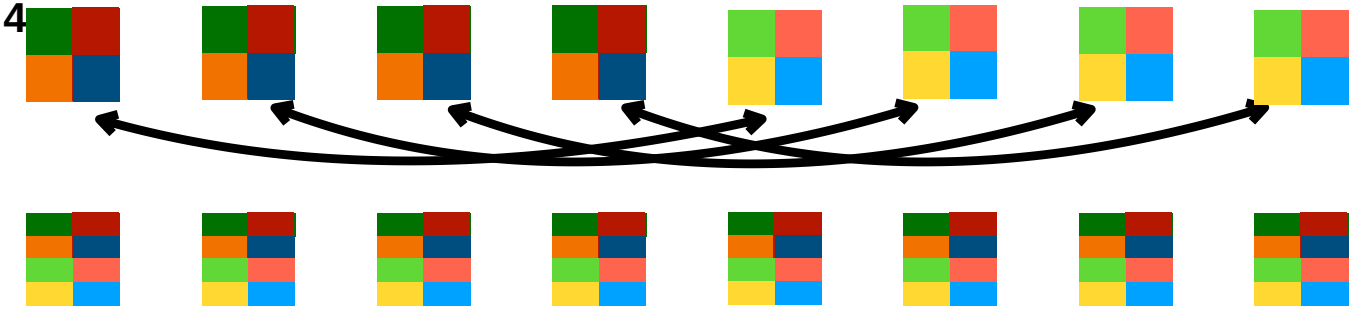
Step 1: Each node exchanges with a neighbor offset by 1



Step 2: Each node exchanges with a neighbor offset by 2



Step 3: Each node exchanges with a neighbor offset by 4



With N worker nodes, AllReduce completes in log(N) steps.

[1] Thakur, Rajeev, Rolf Rabenseifner, and William Gropp. "Optimization of Collective Communication Operations in MPICH."

2.3j. Take AllReduce as an Example

- Comparisons among ring-based, tree-based, and halving-doubling Allreduce algorithms.
- Modern CCLs can select an efficient strategy according to the topology and the communication domain.

Algorithm	Steps (Complexity)	Latency	Bandwidth Efficiency	Ideal Data Size
Ring-based	O(N)	High	Optimal	Large Data (> 64MB)
Tree-based	O(log N)	Lowest	Moderate	Small-Medium (< 16MB)
Halving-Doubling	O(log N)	Low	Moderate	Very Small (< 1MB)

2.4. Protocol Design Matters

- Take NCCL as an example.

TABLE I
COMPARISON OF NCCL COMMUNICATION PROTOCOLS

	Simple	LL	LL128
Design Goal	High bandwidth	Low latency	Low latency and high bandwidth
Synchronization Mechanism	Memory fences (high overhead)	Flag-based synchronization	Flag-based synchronization
Payload	Data chunks	4B data + 4B flag	120B data + 8B flag
Bandwidth Utilization	Near peak	25 ∼ 50% of peak [11]	∼ 95% of peak [11]
Latency Per-hop	$\sim 6\mu s$	$\sim 1\mu s$	$\sim 2\mu s$

TABLE II
NCCL COMMUNICATION CHARACTERISTICS AND TRANSPORTS

	Intra-Node	Inter-Node
Transport	P2P `p2p.cc` SHM `shm.cc` NVLS `nvls.cc`	NET `net_ib.cc` `net_socket.cc` COLLNET `coll_net.cc`
Physical Interconnect	NVLink PCIe	InfiniBand RoCE TCP/IP (Socket)
Optimizations	GPUDirect P2P `P2P_DIRECT`	GPUDirect RDMA

TABLE III
SUPPORTED ALGORITHMS AND PROTOCOLS FOR NCCL COLLECTIVE OPERATIONS

	AllReduce			Broadcast			Reduce			ReduceScatter			AllGather		
Algorithm	Simple	LL	LL128	Simple	LL	LL128	Simple	LL	LL128	Simple	LL	LL128	Simple	LL	LL128
Ring	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Tree	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
CollNet Direct	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
CollNet Chain	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
NVLS	✓	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗
NVLS Tree	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗

Legend: ✓= Supported, ✗= Not supported.

Z. Hu et al., Demystifying NCCL: An In-depth Analysis of GPU Communication Protocols and Algorithms, arXiv:2507.04786, 2025.

2.5. Besides, the Protocol Designs Are Also Important

- Take NCCL as an example.

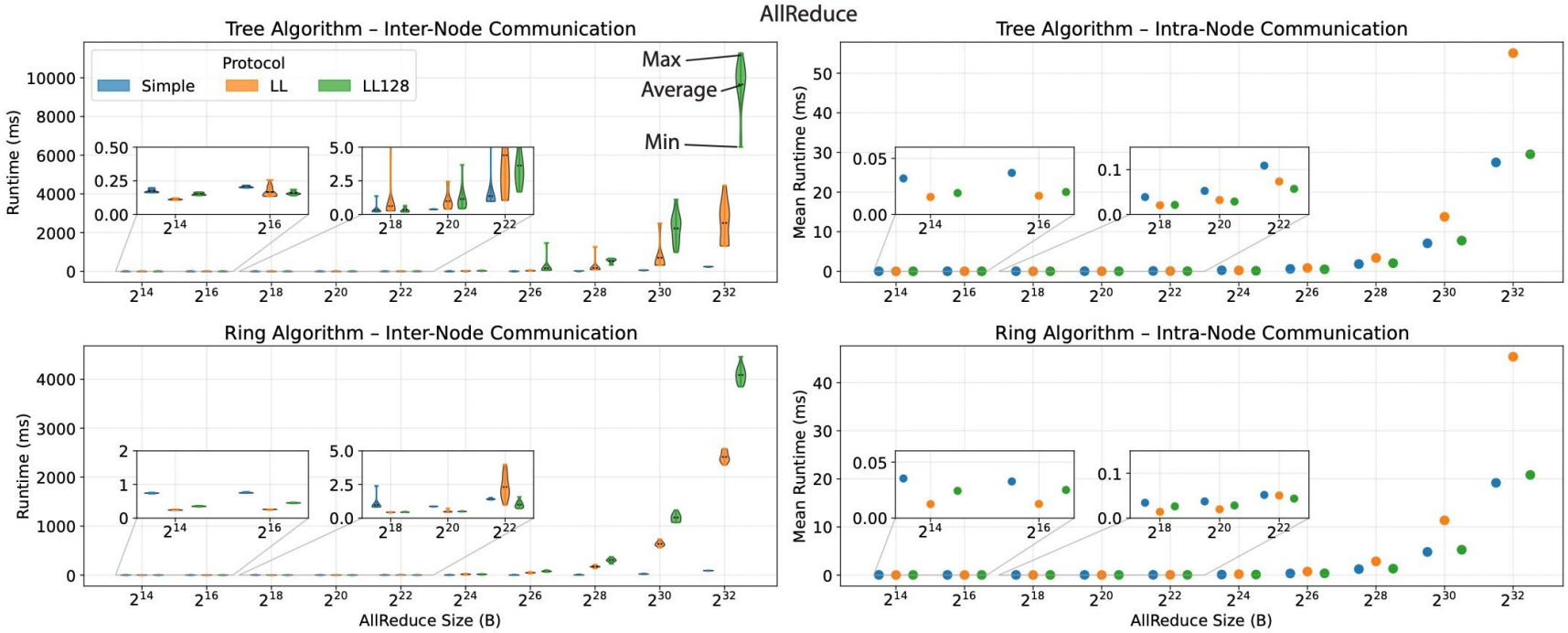


Fig. 6. Runtime comparison of protocols for Ring and Tree AllReduce when running inter- and intra-node. Each data point consists of 20 runs with a warm-up phase. For intra-node communication we report only the median value for readability as the variance is very low.

Z. Hu et al., Demystifying NCCL: An In-depth Analysis of GPU Communication Protocols and Algorithms, arXiv:2507.04786, 2025.

2.6. Broadcast (a.k.a., One-to-All Broadcast)

- Among a group of processing elements (nodes), a designated root node has a data item to be communicated (copied) to all other nodes.

Before			After		
Node *r*	Node 1	Node 2	Node *r*	Node 1	Node 2
x			*x*	x	*x*

  - All nodes are assumed to explicitly take part in the broadcast operation.
  - It is generally assumed that before its execution all nodes know the index of the designated root node as well as the amount n of data to be broadcast.
  - The data item x may be either a single, atomic unit or divisible into smaller, disjoint pieces.
    - The latter can be exploited algorithmically when n is large.

2.7. Broadcast: Assumptions

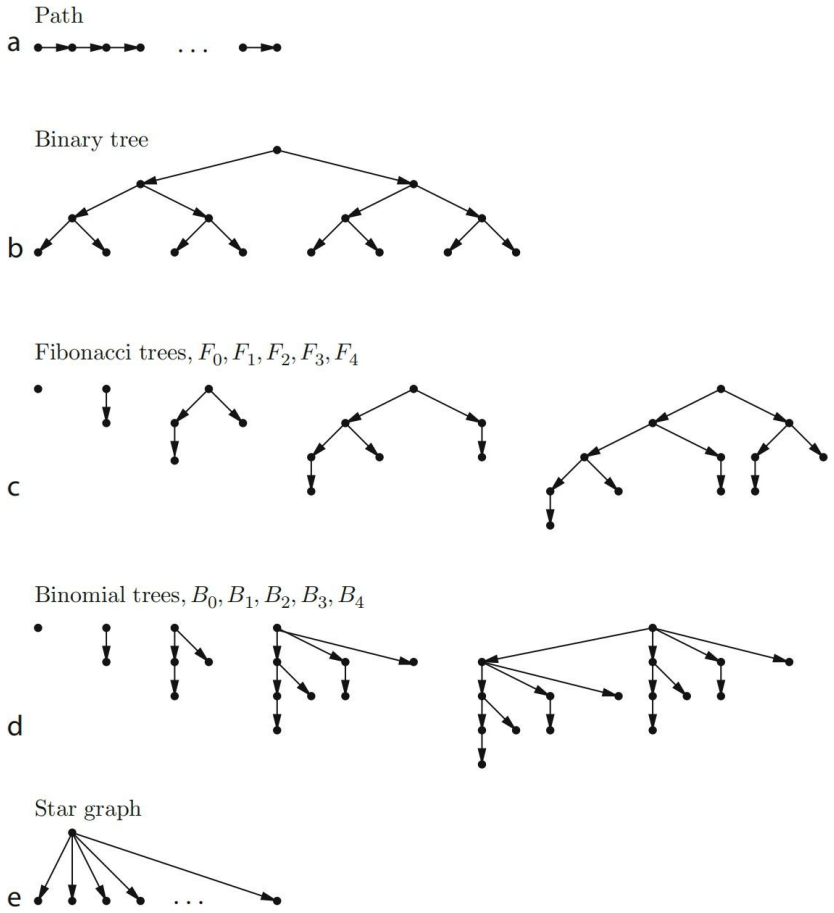
- Assumptions in the following analysis (for this lecture ONLY)
  - All nodes can communicate through a communication network.
  - Individual nodes perform communication operations that send and/or receive individual messages.
  - Communication is through a single port
    - (such that a node can be involved in at most one communication operation at a time)
    - Such an operation can be either a send to or a receive from another node (unidirectional communication),
    - a combined send to and receive from another node (bidirectional), or
    - a send to and receive from two possibly different nodes (simultaneous sendreceive, fully bidirectional communication).
  - The communication medium is homogeneous and fully connected
  - A first approximation for the transferring time between two nodes: $\alpha + n\beta$
    - n is the message size
    - α is the start-up cost (latency)
    - β is the cost per item transferred (inverse of the bandwidth)

2.8. Broadcast: Two Lower Bounds

- Two lower bounds
  - for the α term: $\lceil \log_2 p \rceil \alpha$
    - a round: during which each node can send at most one message and receive at most one message.
    - In each round, the number of nodes that know message x can at most double.
    - Thus, a minimum of $\lceil \log_2 p \rceil$ rounds are needed to broadcast the message.
    - Each round costs at least α.
  - for the β term: $n\beta$
    - If $p > 1$ then the message must leave the root node, requiring a time of at least $n\beta$.
- Notes
  - When assumptions about the communication system change, these lower bounds change as well.
  - Lower bounds for mesh and torus networks, hypercubes, and many other topologies are known [WGe95].
  - Determining the minimum broadcast time in an arbitrary, given graph is NP-hard [JaM95]

[WGe95] Watts J, van de Geijn RA (1995) A pipelined broadcast for multi-dimensional meshes. Parallel Process Lett 5: 281–292
[JaM95] Jansen K, Müller H (1995) The minimum broadcast time problem for several processor networks. Theor Comput Sci 147(1&2): 69–85

2.9. Broadcast: Tree-Based Algorithms



2.10. Broadcast: The “MST” Algorithm

- The so-called* Minimum Spanning Tree (MST) algorithm
  - Partition the set of nodes into two roughly equalsized subsets.
  - Send x from the root to a node in the subset that does not include the root.
    - The receiving node will become a local root in that subset.
  - Recursively broadcast x from the (local) root nodes in the two subsets.
- The total cost of the MST algorithm: $\lceil \log_2 p \rceil (\alpha + n\beta)$
  - It achieves the lower bound for the α term
  - but not for the β term for which it is a logarithmic factor off from the lower bound.
- The MST algorithm constructs a binomial spanning tree over the set of nodes
  - Figure (d) in the previous slide

* which is a misnomer, since all broadcast trees are spanning trees and minimum for homogeneous communication media

2.11. Broadcast: Pipelining

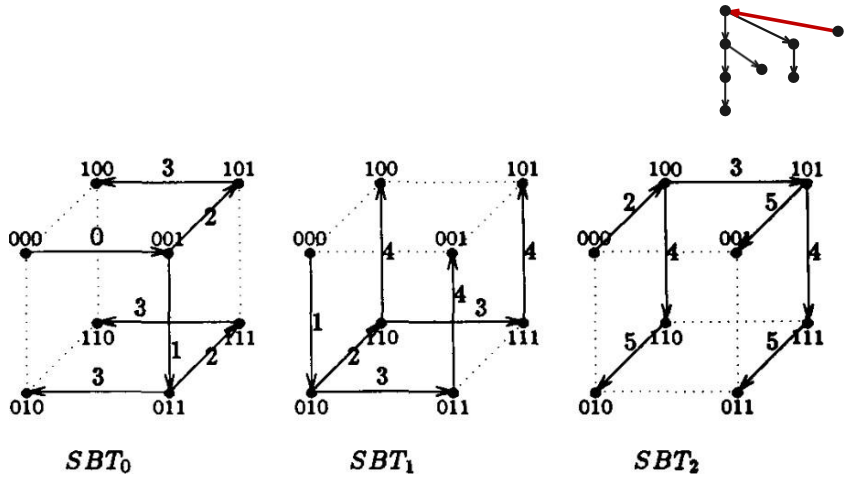
- For large n, pipelining improves the broadcast cost
  - the message is split into k blocks of size $\frac{n}{k}$ each
- In the "path" broadcast tree:
  - The first piece from node 0 takes $p-1$ rounds to reach node $p-1$
  - an additional $k-1$ rounds for the remaining $k-1$ pieces
  - total time cost = $(k+p-2)(\alpha+\beta\frac{n}{k}) \geq (p-2)\alpha + 2\sqrt{(p-2)n\alpha\beta} + \beta n$
    - with optimal number of blocks $k = \sqrt{(p-2)\beta\frac{n}{\alpha}}$
- Pipelining can be applied on fixed degree trees
  - Binary tree: best-effort time cost = $2(\log_2 p-1)\alpha + 4\sqrt{(\log_2 p-1)n\alpha\beta} + 2\beta n$
  - Fibonacci tree: best-effort time cost = $(\log_\Theta p-1)\alpha + 2\sqrt{2(\log_\Theta p-1)n\alpha\beta} + 2\beta n$ where $\Theta = \frac{\sqrt{5}+1}{2}$
- Pipelining is not helpful on nonconstant degrees trees (e.g., binomial tree and star graph)

2.12. Broadcast: The Third Lower Bound

- The third lower bound on the number of rounds: $k - 1 + \lceil \log_2 p \rceil$
  - first piece $\lceil \log_2 p \rceil$ rounds plus k-1 additional rounds
  - total time cost = $(k - 1 + \lceil \log_2 p \rceil)(\alpha + \beta \frac{n}{k}) \geq (\log_2 p - 1)\alpha + 2\sqrt{(\log_2 p - 1)\alpha \beta n} + \beta n$
    - with optimal number of blocks $k = \sqrt{(\log_2 p - 1)\beta \frac{n}{\alpha}}$

2.13. Broadcast: Simultaneous Trees

- optimal k - 1 + log2p rounds of pipelining
  - when the message is split into k blocks
- p=2d nodes in a d-dimensional hypercube
  - node id: i ∈ { 0, 1, …, 2d-1 } with ij as the j-th bit
- d edge-disjoint spanning binomial trees (ESBT)
  - all trees are rooted at node 0
  - the j-th tree has 2d-1-1 leaves with ij=0
  - the j-th tree broadcasts blocks j, j+d, j+2d, …
- Leaves in tree j = t%d receive block t-d in round t
  - block -d, 1-d, …, −1, 0, 1, …, k-1
- BitDistancei[j]: distance to the next 1 to the left of ij
  - BitDistance0[k] ≡ k for 0 < k ≤ d

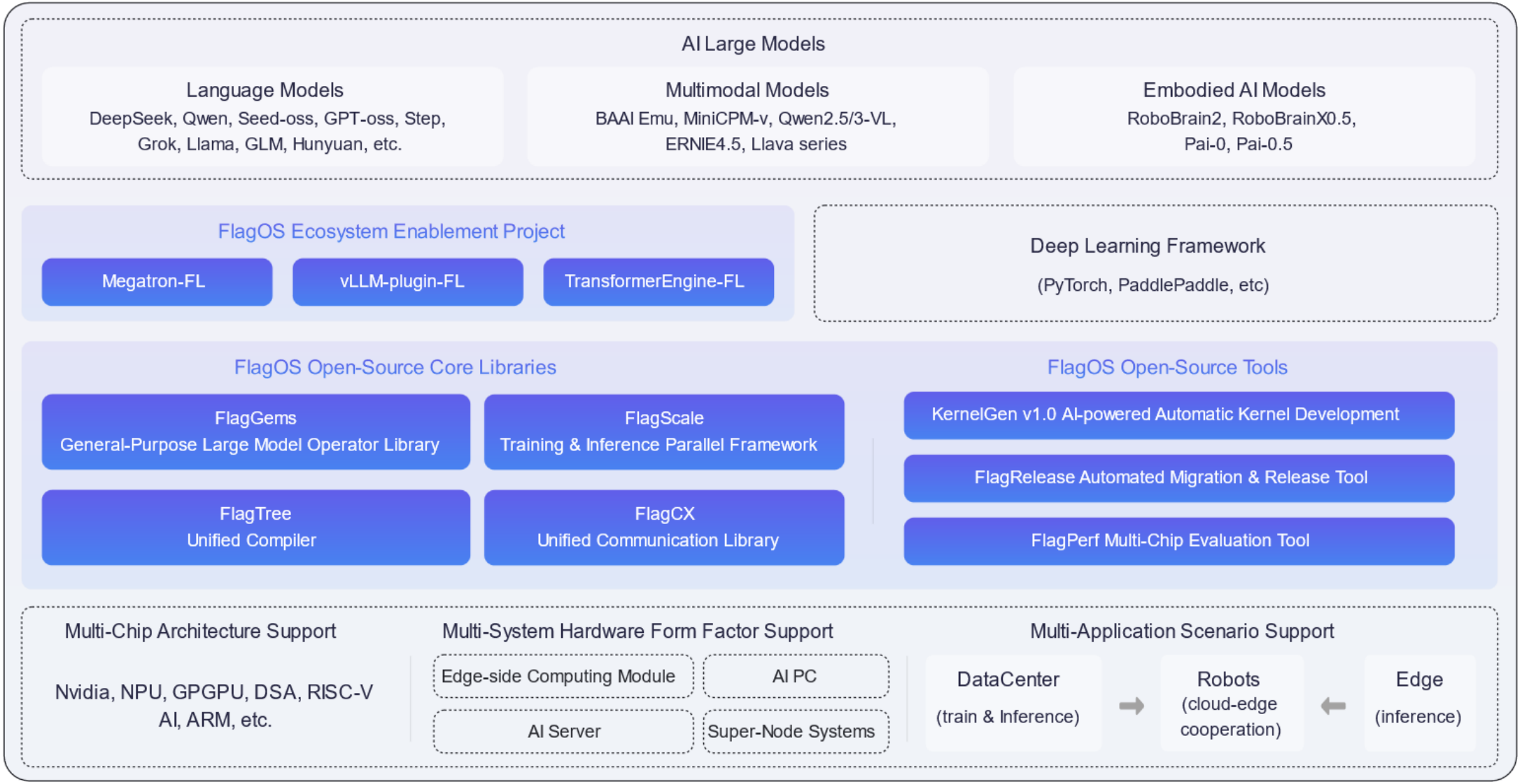



```
f ← ((k mod d) + d − 1) mod d /* Start round for first phase */
t ← 0
while t < k + d − 1 do
      /* New phase consisting of (up to) d rounds */
      for j ← f to d − 1
          s ← t − d + (1 − i_j) ∗ BitDistance_i[j] /* block to send */
          r ← t − d + i_j ∗ BitDistance_i[j]       /* block to receive */
          if s ≥ k then s ← k − 1
          if r ≥ k then r ← k − 1
          par/* simultaneous send-receive with neighbor */
              if s ≥ 0 then Send block s to node (i xor 2^j)
              if r ≥ 0 then Receive block r from node (i xor 2^j)
          end par
          t ← t + 1 /* next round */
      end for
      f ← 0 /* next phases start from 0 */
endwhile
```

S. Lennart Johnsson and C. T. Ho, Optimum Broadcasting and Personalized Communication in Hypercubes, IEEE TC, 1989.

2.14. Further Readings

- van de Geijn, R., Träff, J. (2011). Collective Communication. In: Padua, D. (eds) Encyclopedia of Parallel Computing. Springer, Boston, MA.
- Träff, J.L., van de Geijn, R.A. (2011). Broadcast. In: Padua, D. (eds) Encyclopedia of Parallel Computing. Springer, Boston, MA.
- Träffe, J.L., van de Geijn, R.A. (2011). Allgather. In: Padua, D. (eds) Encyclopedia of Parallel Computing. Springer, Boston, MA.
- Träff, J.L., vande Geijn, R.A. (2011). All-to-All. In: Padua, D. (eds) Encyclopedia of Parallel Computing. Springer, Boston, MA.


AI Large Models
Language Models
DeepSeek, Qwen, Seed-oss, GPT-oss, Step,
Grok, Llama, GLM, Hunyuan, etc.
Multimodal Models
BAAI Emu, MiniCPM-v, Qwen2.5/3-VL,
ERNIE4.5, Llava series
Embodied AI Models
RoboBrain2, RoboBrainX0.5,
Pai-0, Pai-0.5
FlagOS Ecosystem Enablement Project
Megatron-FL
vLLM-plugin-FL
TransformerEngine-FL
Deep Learning Framework
(PyTorch, PaddlePaddle, etc)
FlagOS Open-Source Core Libraries
FlagGems
General-Purpose Large Model Operator Library
FlagScale
Training & Inference Parallel Framework
FlagTree
Unified Compiler
FlagCX
Unified Communication Library
FlagOS Open-Source Tools
KernelGen v1.0 AI-powered Automatic Kernel Development
FlagRelease Automated Migration & Release Tool
FlagPerf Multi-Chip Evaluation Tool
Multi-Chip Architecture Support
Nvidia, NPU, GPGPU, DSA, RISC-V
AI, ARM, etc.
Multi-System Hardware Form Factor Support
Edge-side Computing Module
AI PC
AI Server
Super-Node Systems
Multi-Application Scenario Support
DataCenter
(train & Inference)
Robots
(cloud-edge
cooperation)
Edge
(inference)

African Scientific Research and Innovation Council

BAAI 智源研究院

北京大学 PEKING UNIVERSITY

中国人民大学 RENMIN UNIVERSITY OF CHINA

Thanks

Questions & further discussion welcome

Guojie Luo
罗国杰
gluo@pku.edu.cn

Jing Wang
王晶
jwang@ruc.edu.cn

BAAI Team
智源团队
github.com/flagos-ai

